

LFS458 — Kubernetes Administration Lab Guide

Welcome to the **Vega Training** lab guide for the LFS458 Kubernetes Administration course.

Lab Topology

Each student environment consists of **three VMs** on a shared `/24` network. The hostnames are fixed; only the IP addresses differ per student.

Role	Hostname	IP Address
Control Plane	<code>controller</code>	<i>assigned by your instructor</i>
Worker 1	<code>worker1</code>	<i>assigned by your instructor</i>
Worker 2	<code>worker2</code>	<i>assigned by your instructor</i>

Finding your IPs

Run `ip addr show` on each node to find its IP. Use `/etc/hosts` to map hostnames so you can `ssh controller`, `ssh worker1`, etc. without remembering IPs:

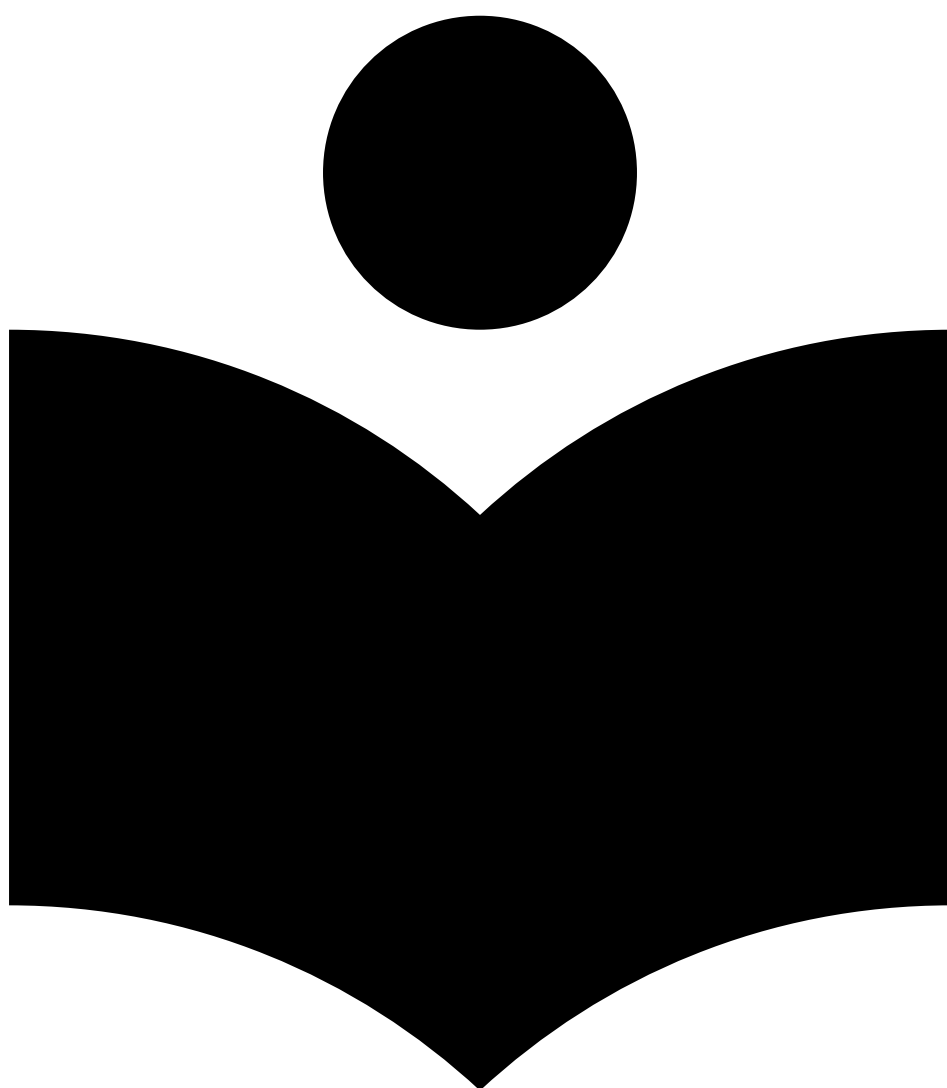
```
<controller-ip> controller
<worker1-ip>    worker1
<worker2-ip>    worker2
```

Lab Environment

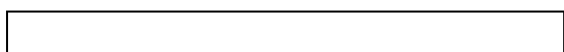
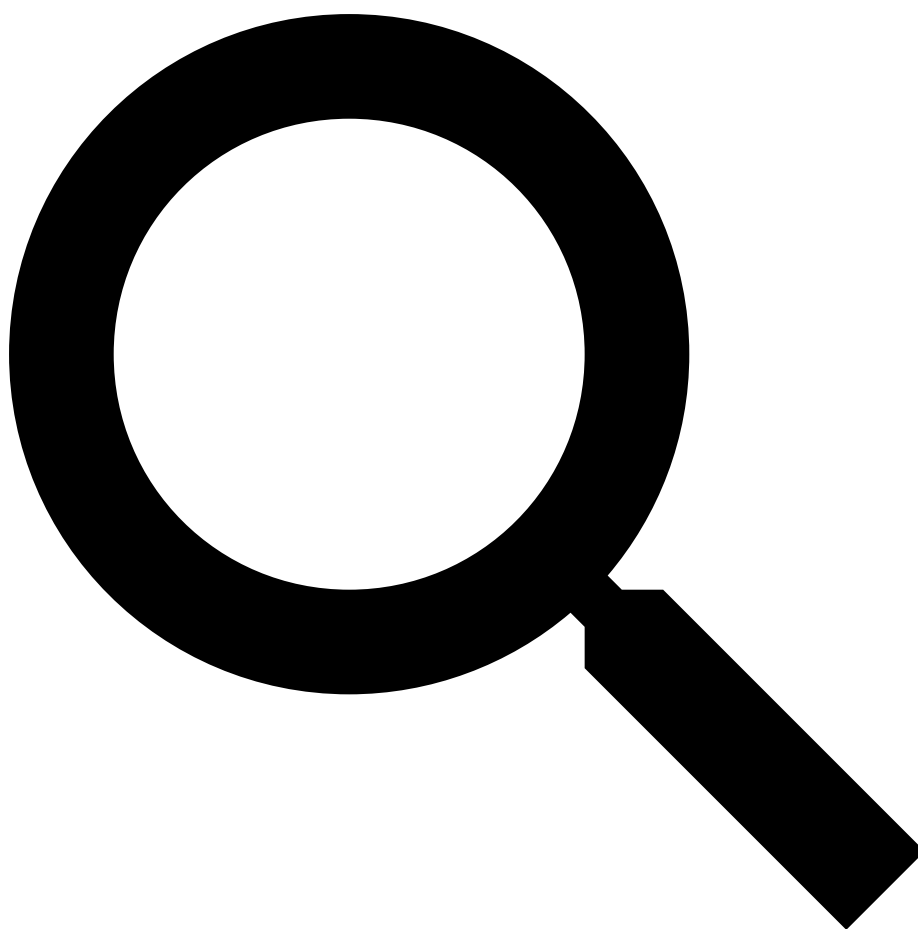
Component	Version
OS	Ubuntu 22.04 LTS

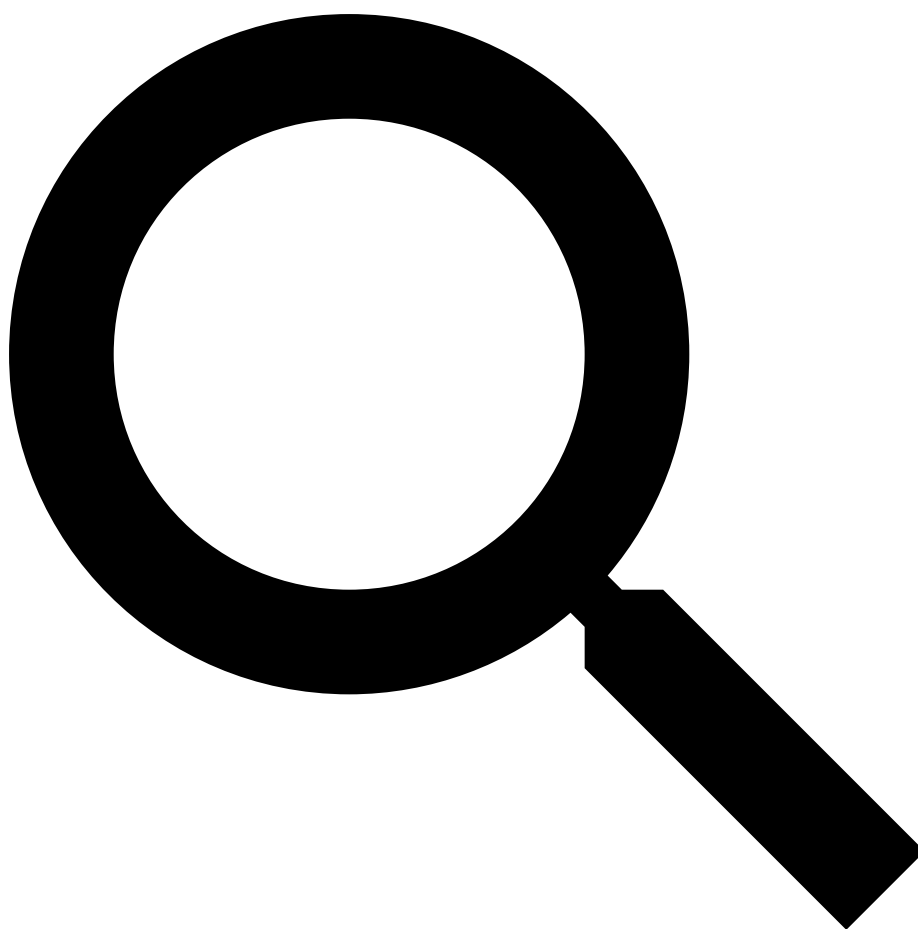


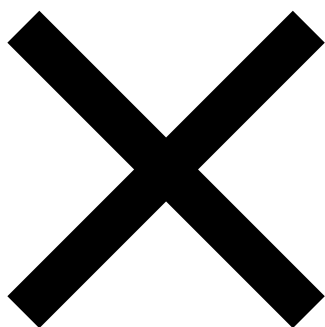
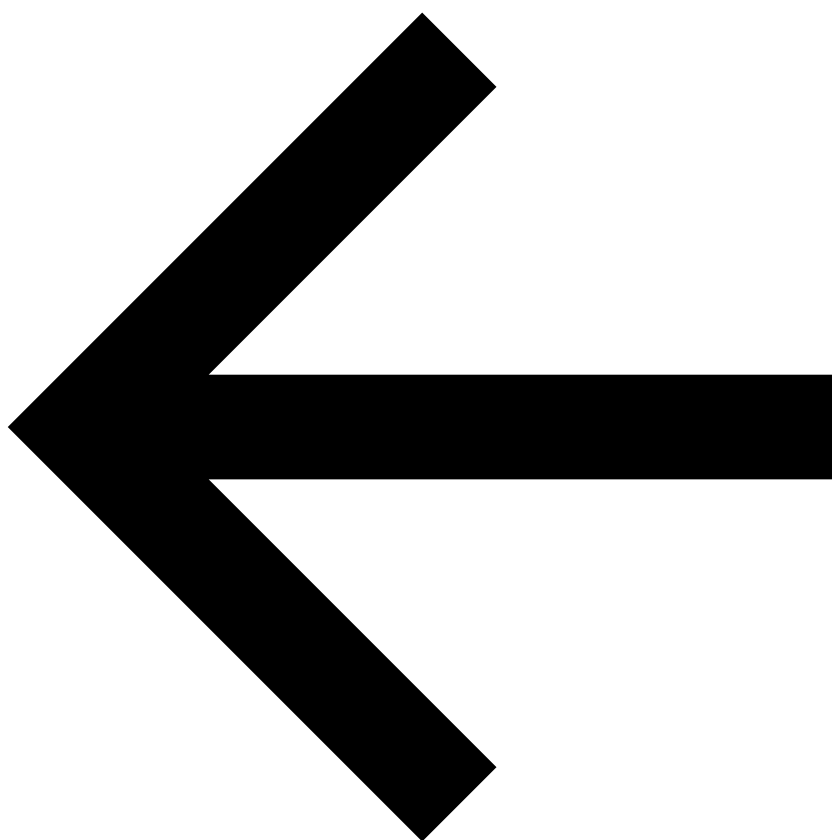
[Skip to content](#)





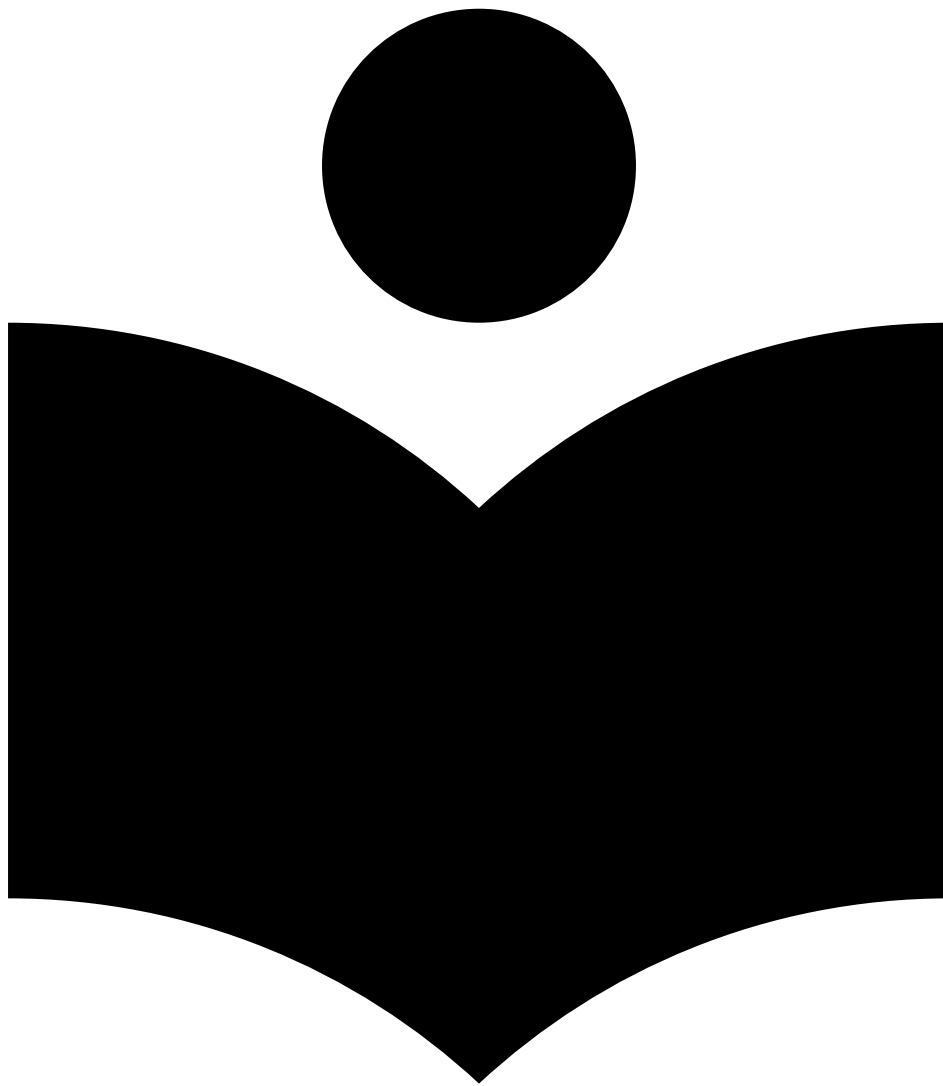






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- ☐ Ch 03: Install Kubernetes [Ch 03: Install Kubernetes](#)
Table of contents
 - [Overview](#)
 - [Lab 3.1 — Prepare All Nodes](#)
 - [Disable swap \(All nodes\)](#)
 - [Load required kernel modules \(All nodes\)](#)
 - [Configure sysctl \(All nodes\)](#)
 - [Lab 3.2 — Install containerd \(All nodes\)](#)
 - [Lab 3.3 — Install Kubernetes Packages \(All nodes\)](#)
 - [Lab 3.4 — Initialise the Control Plane \(controller only\)](#)
 - [Lab 3.5 — Install Calico CNI \(controller only\)](#)
 - [Lab 3.6 — Join Worker Nodes \(worker1 and worker2\)](#)
 - [Lab 3.7 — Test the Cluster](#)
- [Ch 04: Architecture](#)

- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 3.1 — Prepare All Nodes](#)
 - [Disable swap \(All nodes\)](#)
 - [Load required kernel modules \(All nodes\)](#)
 - [Configure sysctl \(All nodes\)](#)
- [Lab 3.2 — Install containerd \(All nodes\)](#)
- [Lab 3.3 — Install Kubernetes Packages \(All nodes\)](#)
- [Lab 3.4 — Initialise the Control Plane \(controller only\)](#)
- [Lab 3.5 — Install Calico CNI \(controller only\)](#)
- [Lab 3.6 — Join Worker Nodes \(worker1 and worker2\)](#)
- [Lab 3.7 — Test the Cluster](#)

Chapter 3: Install Kubernetes

Overview

In this lab you will build a three-node Kubernetes cluster from scratch using kubeadm. You will:

- Install the container runtime (containerd) on all nodes
- Install Kubernetes packages (kubeadm, kubelet, kubectl)
- Initialise the control plane with `kubeadm init`
- Install **Calico** as the CNI (pod CIDR 10.244.0.0/16)
- Join both worker nodes to the cluster
- Deploy a test nginx workload

All steps marked "**All nodes**" must be run on controller, worker1, and worker2 unless stated otherwise.

Lab 3.1 — Prepare All Nodes

Disable swap (All nodes)

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

Load required kernel modules (All nodes)

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

Configure sysctl (All nodes)

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
```

```
sudo sysctl --system
```

Lab 3.2 — Install containerd (All nodes)

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release
```

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
```

```
sudo apt-get update
sudo apt-get install -y containerd.io
```

Generate the default config and enable SystemdCgroup:

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
```

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Lab 3.3 — Install Kubernetes Packages (All nodes)

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.33/deb/Release.key \
| sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /' \
| sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl

sudo systemctl enable kubelet
```

Lab 3.4 — Initialise the Control Plane (controller only)

Copy the provided config file to the home directory:

```
cp ~/lfs458/ch03-install/kubeadm-config.yaml ~/kubeadm-config.yaml
```

Review it:

```
cat ~/kubeadm-config.yaml
```

The podSubnet is set to 10.244.0.0/16 to match the Calico CNI we will install.

Initialise the cluster:

```
sudo kubeadm init --config=kubeadm-config.yaml --upload-certs \
| sudo tee /var/log/kubeadm-init.log
```

Note

This will take 2-3 minutes. Save the kubeadm join command printed at the end — you will need it for the worker nodes.

Set up kubectl access for your user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Verify the control plane is up (the node will be NotReady until the CNI is installed):

```
kubectl get nodes
kubectl get pods -n kube-system
```

Lab 3.5 — Install Calico CNI (controller only)¶

Install the Tigera Operator:

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v
```

Wait for the operator to be ready:

```
kubectl rollout status deployment tigera-operator -n tigera-operator --tim
```

Apply the custom resources to configure Calico with the 10.244.0.0/16 pod CIDR:

```
kubectl apply -f ~/lfs458/ch03-install/calico-custom-resources.yaml
```

Watch Calico come up:

```
watch kubectl get pods -n calico-system
```

Wait until all pods show Running. Then verify the node is Ready:

```
kubectl get nodes
```

Lab 3.6 — Join Worker Nodes (worker1 and worker2)¶

From your kubeadm init output, copy the kubeadm join command and run it on each worker with sudo. It looks like:

```
sudo kubeadm join controller:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Lost the join command?

Regenerate it on the controller:

```
kubeadm token create --print-join-command
```

Back on the **controller**, verify all nodes joined:

```
kubectl get nodes -o wide
```

All three nodes should be Ready within a minute or two.

Lab 3.7 — Test the Cluster¶

Deploy a simple nginx pod:

```
kubectl create deployment nginx --image=nginx  
kubectl get pods -w
```

Wait until the pod shows Running, then:

```
POD=$(kubectl get pod -l app=nginx -o jsonpath='{.items[0].metadata.name}')  
kubectl exec -it $POD -- nginx -v
```

Expose it and test connectivity:

```
kubectl expose deployment nginx --port=80 --type=NodePort
```

```
NODE_PORT=$(kubectl get svc nginx -o jsonpath='{.spec.ports[0].nodePort}')  
echo "NodePort: $NODE_PORT"
```

```
curl http://controller:$NODE_PORT
```

You should see the nginx welcome page HTML.

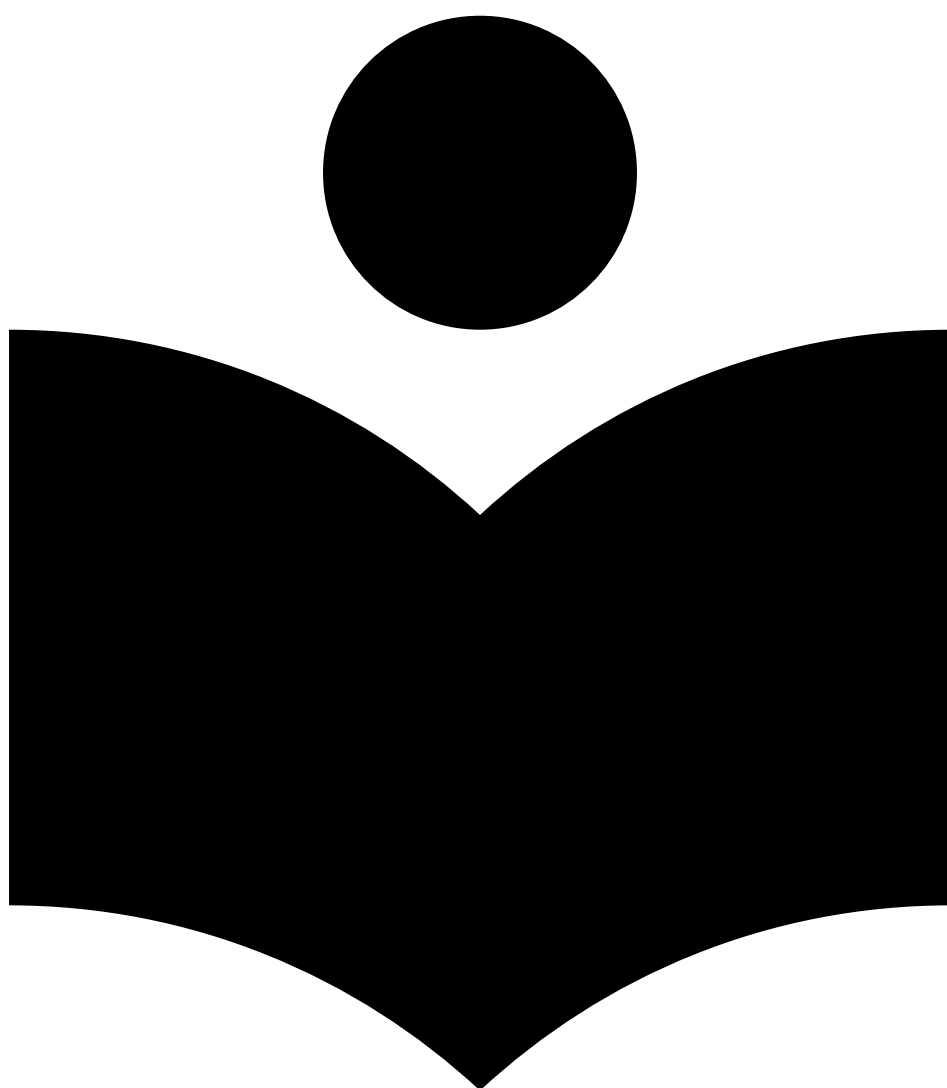
Clean up:

```
kubectl delete deployment nginx  
kubectl delete svc nginx
```

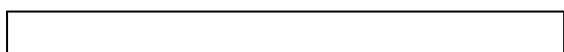
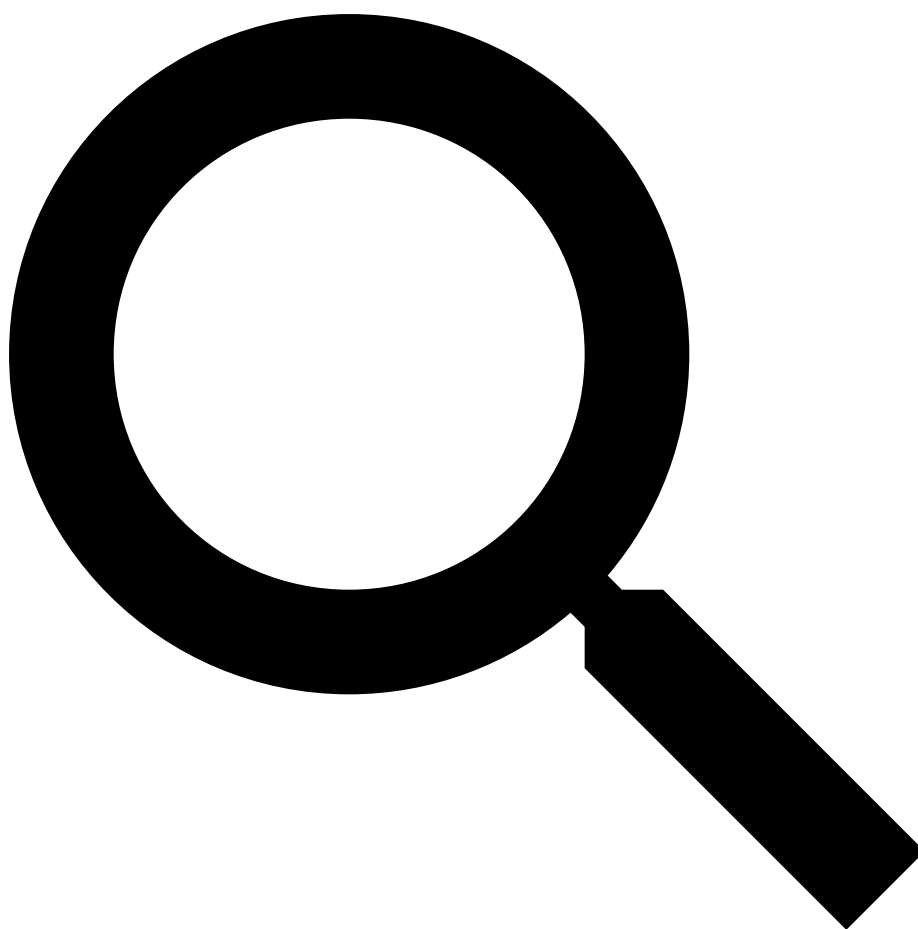
© 2025 VEGA TRAINING

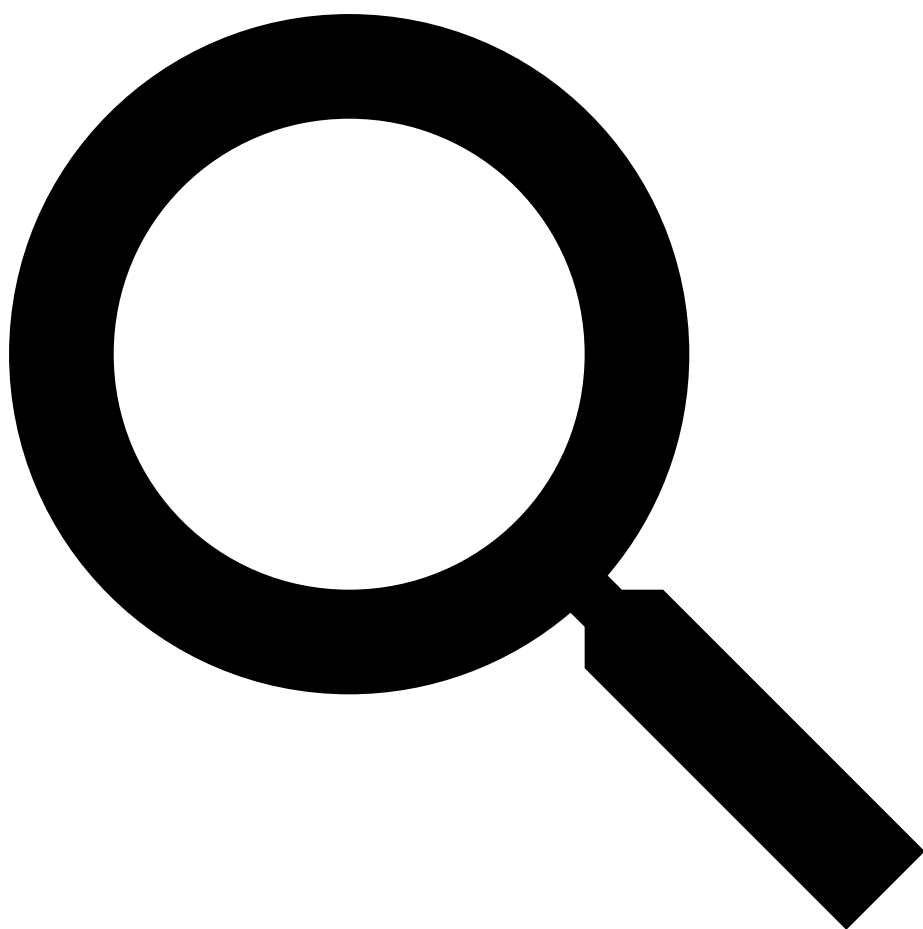


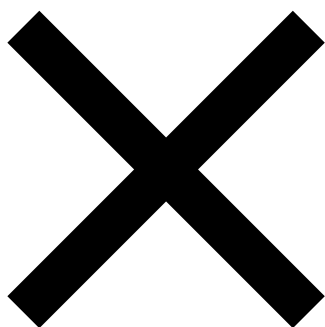
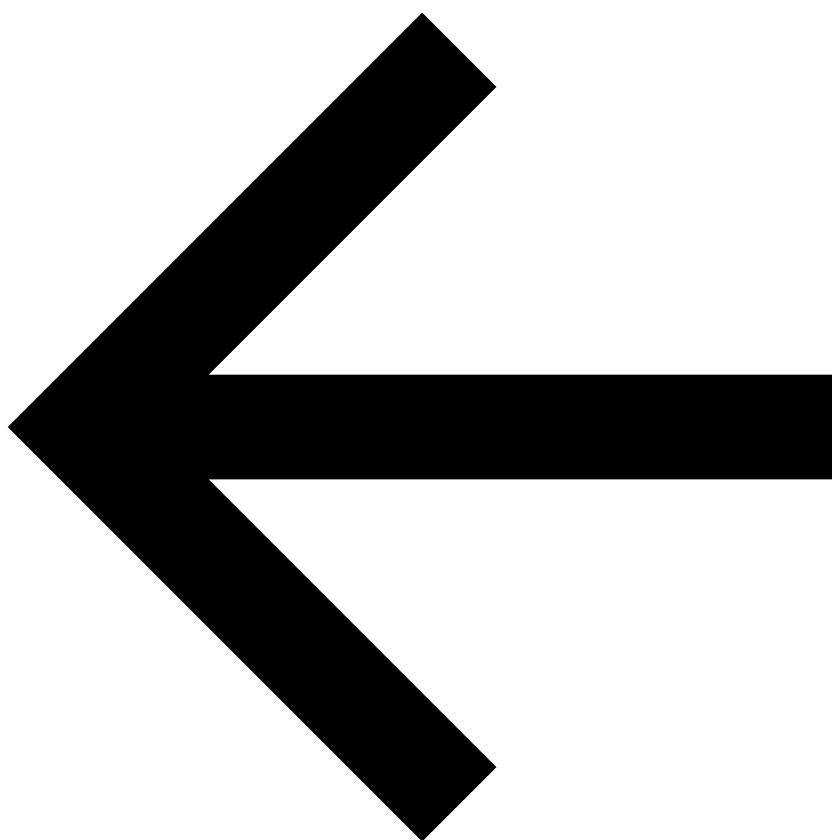
[Skip to content](#)





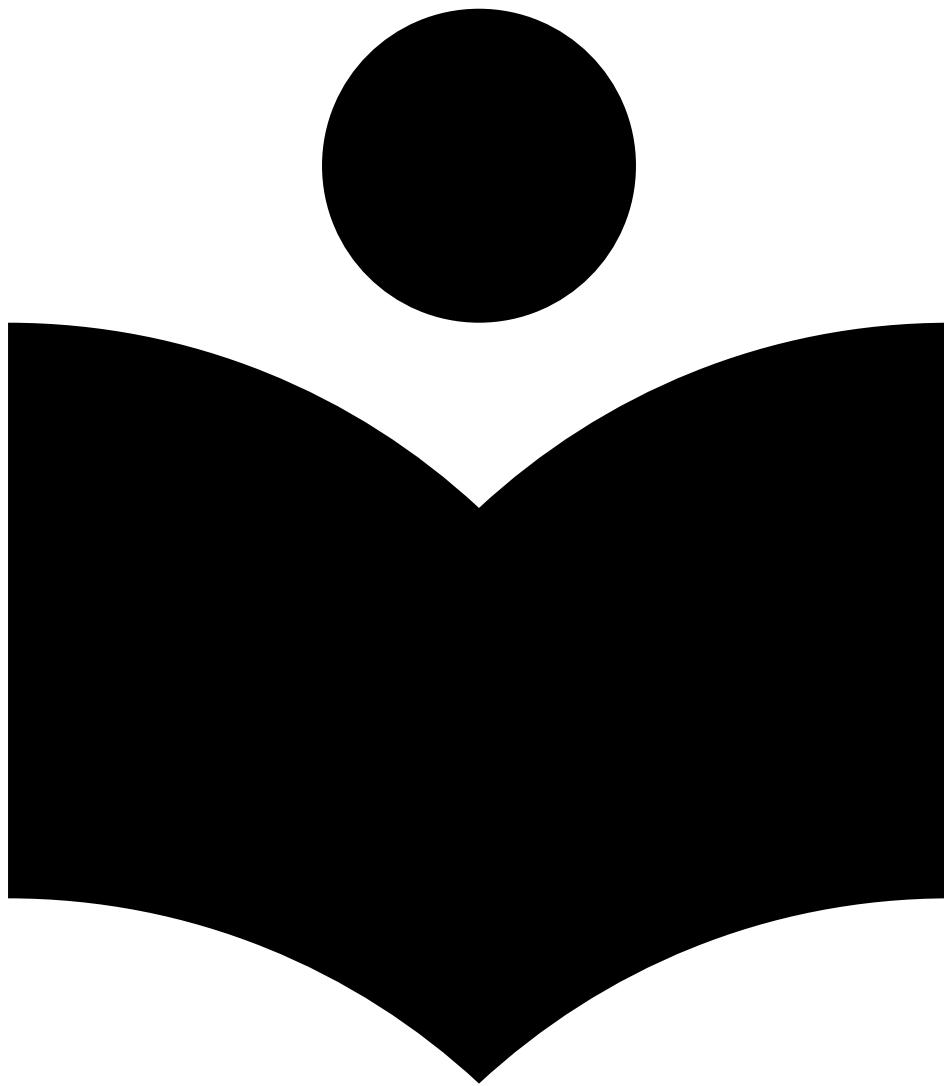






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- ☐ Ch 04: Architecture [Ch 04: Architecture](#)
Table of contents
 - [Overview](#)
 - [Lab 4.1 — etcd Backup and Restore](#)
 - [Backup etcd \(controller only\)](#)
 - [Restore etcd](#)
 - [Lab 4.2 — Cluster Upgrade \(1.33 → 1.34\)](#)
 - [Upgrade the control plane \(controller only\)](#)
 - [Upgrade worker nodes \(worker1 and worker2\)](#)
 - [Lab 4.3 — Resource Requests and Limits](#)
 - [Deploy the hog pod](#)
 - [Clean up](#)
 - [Lab 4.4 — LimitRange](#)

- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 4.1 — etcd Backup and Restore](#)
 - [Backup etcd \(controller only\)](#)
 - [Restore etcd](#)
- [Lab 4.2 — Cluster Upgrade \(1.33 → 1.34\)](#)
 - [Upgrade the control plane \(controller only\)](#)
 - [Upgrade worker nodes \(worker1 and worker2\)](#)
- [Lab 4.3 — Resource Requests and Limits](#)
 - [Deploy the hog pod](#)
 - [Clean up](#)
- [Lab 4.4 — LimitRange](#)

Chapter 4: Kubernetes Architecture¶

Overview¶

In this lab you will:

- Back up and restore the etcd database
- Upgrade the Kubernetes cluster from 1.33 to 1.34
- Set resource requests and limits on a pod
- Configure a LimitRange to enforce default limits on a namespace

Lab 4.1 — etcd Backup and Restore¶

Backup etcd (controller only)¶

Find the etcd pod and inspect its configuration:

```
ETCD_POD=$(kubectl get pod -n kube-system -l component=etcd \
  -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $ETCD_POD -n kube-system | grep -E "listen-client|cer
```


Install etcdctl:

```
sudo apt-get install -y etcd-client
```

Take a snapshot:

```
sudo ETCDCTL_API=3 etcdctl snapshot save /var/lib/etcd/snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Verify the snapshot:

```
sudo ETCDCTL_API=3 etcdctl snapshot status /var/lib/etcd/snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --write-out=table
```

Restore etcd

To restore, stop the API server, restore the data directory, then restart:

```
# Move the API server manifest to temporarily stop the pod
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
```

```
# Restore snapshot to a new data directory
sudo ETCDCTL_API=3 etcdctl snapshot restore /var/lib/etcd/snapshot.db \
  --data-dir=/var/lib/etcd-restore \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

```
# Point etcd to the restored directory (edit the etcd static pod manifest)
sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-restore|g' /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
# Restore the API server
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
```

```
# Wait for the cluster to recover
kubectl get nodes
```

Warning

In production, a full restore requires additional care (member peer URLs, cluster state). This exercise demonstrates the mechanics; the LFS458 course book covers production restore procedures in detail.

Lab 4.2 — Cluster Upgrade (1.33 → 1.34)[¶](#)

Upgrade the control plane (controller only)[¶](#)

Check available versions:

```
sudo apt-get update  
apt-cache madison kubeadm | head -5
```

Upgrade kubeadm:

```
sudo apt-mark unhold kubeadm  
sudo apt-get install -y kubeadm=1.34.0-*  
sudo apt-mark hold kubeadm
```

Plan the upgrade:

```
sudo kubeadm upgrade plan
```

Apply the upgrade:

```
sudo kubeadm upgrade apply v1.34.0
```

Drain the control plane node and upgrade kubelet:

```
kubectrl drain controller --ignore-daemonsets --delete-emptydir-data
```

```
sudo apt-mark unhold kubelet kubectrl  
sudo apt-get install -y kubelet=1.34.0-* kubectrl=1.34.0-*  
sudo apt-mark hold kubelet kubectrl
```

```
sudo systemctl daemon-reload  
sudo systemctl restart kubelet
```

```
kubectrl uncordon controller  
kubectrl get nodes
```

Upgrade worker nodes (worker1 and worker2)[¶](#)

Run on each worker:

```
sudo apt-mark unhold kubeadm  
sudo apt-get install -y kubeadm=1.34.0-*  
sudo apt-mark hold kubeadm
```

```
sudo kubeadm upgrade node
```

On the **controller**, drain the worker:

```
kubectrl drain worker1 --ignore-daemonsets --delete-emptydir-data
```

Back on **worker1**:

```
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y kubelet=1.34.0-* kubect1=1.34.0-*
sudo apt-mark hold kubelet kubect1
```

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

On the **controller**, uncordon:

```
kubect1 uncordon worker1
```

Repeat for worker2. Verify all nodes show v1.34.0:

```
kubect1 get nodes
```

Lab 4.3 — Resource Requests and Limits[¶](#)

Deploy the hog pod[¶](#)

The hog.yaml manifest is a starting point — you will add resource constraints to it.

```
cp ~/lfs458/ch04-architecture/hog.yaml ~/hog.yaml
cat ~/hog.yaml
```

Edit the file to add resources with both requests and limits, and pass `--vm 1 --vm-bytes 100M` to the stress command. The solution is in `~/lfs458/ch04-architecture/solutions/hog-with-resources.yaml` if needed.

Apply it:

```
kubect1 apply -f ~/hog.yaml
```

Watch the pod start:

```
kubect1 get pods -w
```

Once running, view resource usage:

```
HOG_POD=$(kubect1 get pod -l app=hog -o jsonpath='{.items[0].metadata.name}')
kubect1 top pod $HOG_POD
kubect1 logs $HOG_POD
```

Try creating a pod that exceeds the node's available memory — Kubernetes should schedule it only if resources are available, or leave it Pending.

Clean up[¶](#)

```
kubect1 delete -f ~/hog.yaml
```

Lab 4.4 — LimitRange

Create a LimitRange that sets default CPU and memory limits for new pods in a namespace:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: default
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "64Mi"
    max:
      cpu: "2"
      memory: "1Gi"
EOF
```

Verify:

```
kubectl describe limitrange default-limits
```

Deploy a pod without explicit resources and confirm it inherits the defaults:

```
kubectl run testlimit --image=nginx
TEST_POD=$(kubectl get pod testlimit -o jsonpath='{.metadata.name}')
kubectl get pod $TEST_POD -o jsonpath='{.spec.containers[0].resources}' |
```

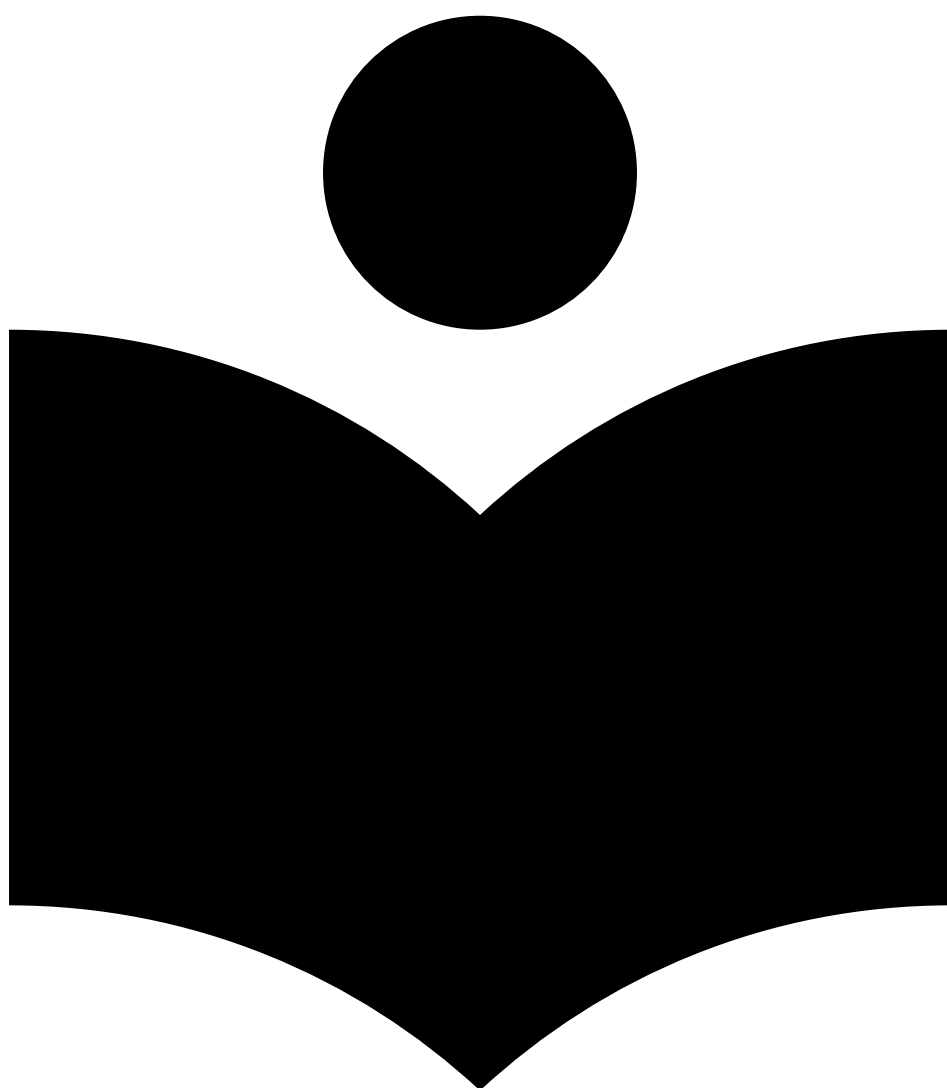
Clean up:

```
kubectl delete pod testlimit
kubectl delete limitrange default-limits
```

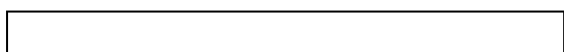
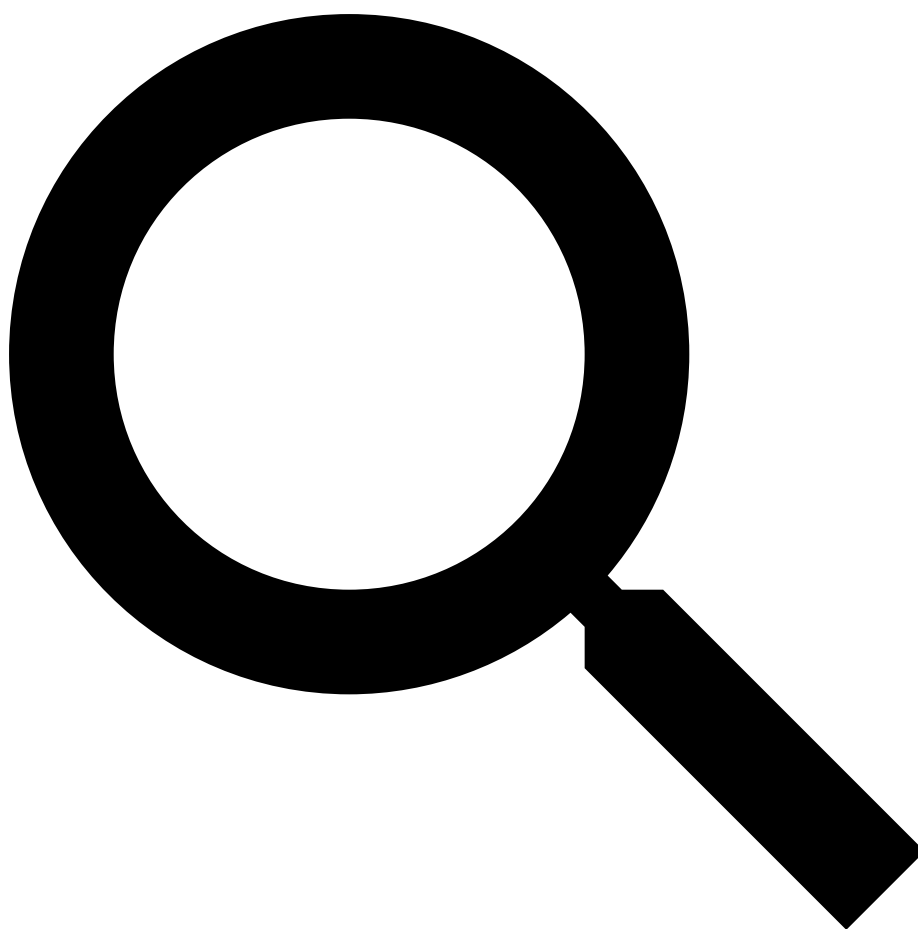
© 2025 VEGA TRAINING

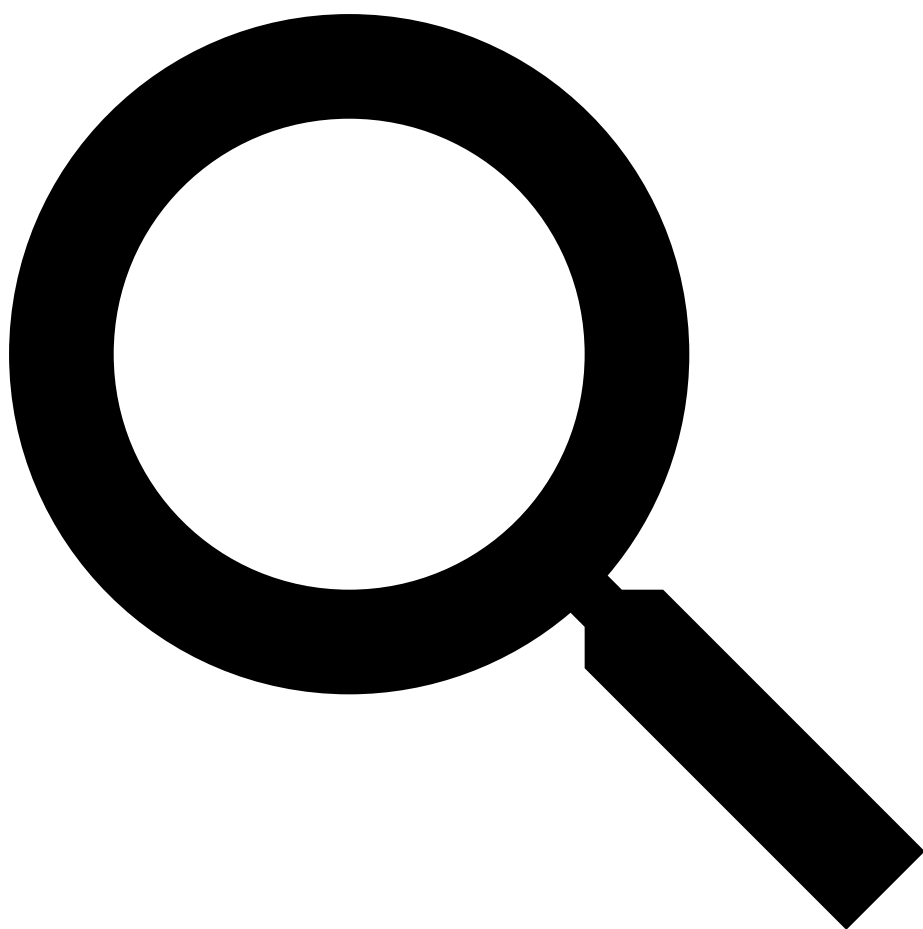


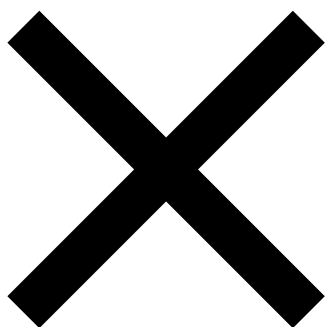
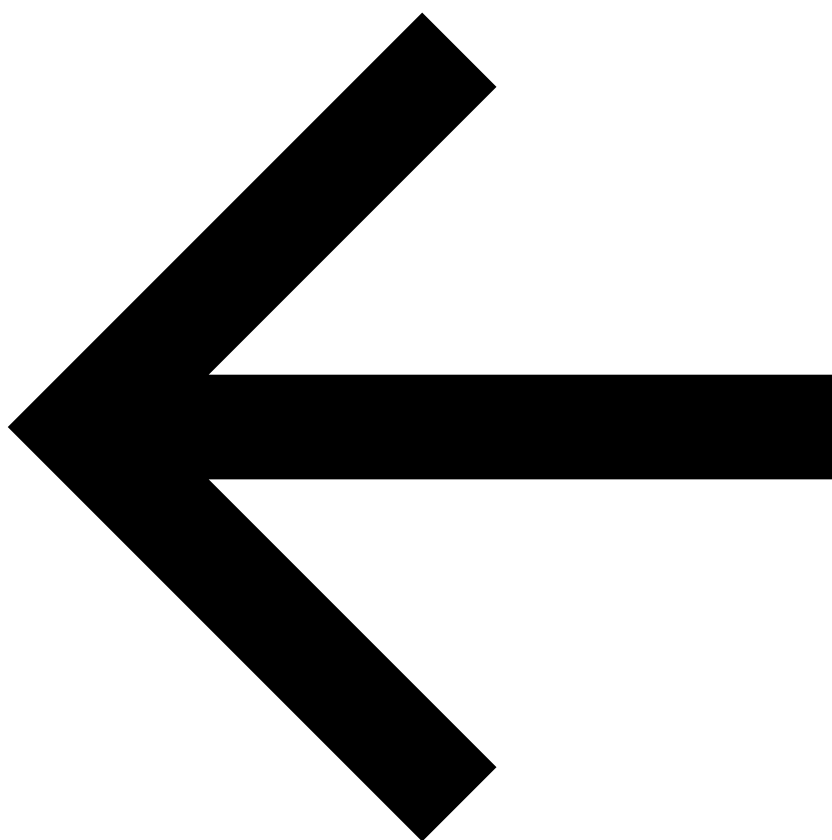
[Skip to content](#)





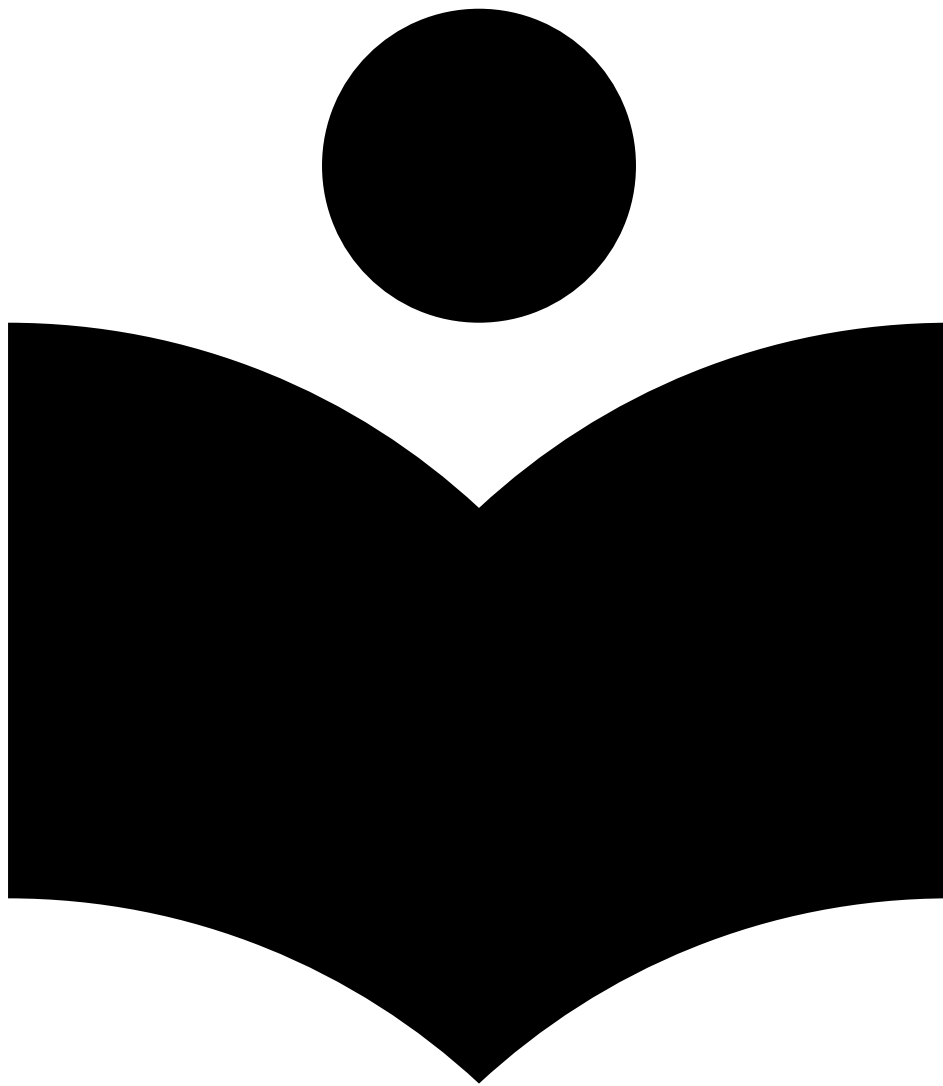






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- ☐ Ch 05: API Access [Ch 05: API Access](#)
 - Table of contents
 - [Overview](#)
 - [Lab 5.1 — Authenticate with Certificates](#)
 - [Locate the cluster CA and client certs](#)
 - [Get the API server URL](#)
 - [Query the API with curl](#)
 - [Lab 5.2 — Bearer Token Authentication](#)
 - [Lab 5.3 — kubectl proxy](#)
 - [Lab 5.4 — Trace kubectl with strace](#)
 - [Lab 5.5 — API Discovery](#)
- [Ch 06: API Objects](#)

- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 5.1 — Authenticate with Certificates](#)
 - [Locate the cluster CA and client certs](#)
 - [Get the API server URL](#)
 - [Query the API with curl](#)
- [Lab 5.2 — Bearer Token Authentication](#)
- [Lab 5.3 — kubectl proxy](#)
- [Lab 5.4 — Trace kubectl with strace](#)
- [Lab 5.5 — API Discovery](#)

Chapter 5: API Access¶

Overview¶

In this lab you will:

- Use TLS certificates to authenticate directly to the Kubernetes API
- Make raw HTTPS requests with curl using PEM certificates
- Use kubectl proxy and kubectl port-forward to access the API
- Trace API calls with strace

Lab 5.1 — Authenticate with Certificates¶

Locate the cluster CA and client certs¶

```
# Your kubeconfig already contains base64-encoded certs; decode them to find the raw certs
kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' \
  | base64 -d > /tmp/ca.crt

kubectl config view --raw -o jsonpath='{.users[0].user.client-certificate-data}' \
  | base64 -d > /tmp/client.crt

kubectl config view --raw -o jsonpath='{.users[0].user.client-key-data}' \
  | base64 -d > /tmp/client.key
```

Get the API server URL

```
API_SERVER=$(kubectl config view -o jsonpath='{.clusters[0].cluster.server'
echo "API server: $API_SERVER"
```

Query the API with curl

```
# List nodes
curl --cacert /tmp/ca.crt \
      --cert /tmp/client.crt \
      --key /tmp/client.key \
      "$API_SERVER/api/v1/nodes" | python3 -m json.tool | head -30

# List namespaces
curl --cacert /tmp/ca.crt \
      --cert /tmp/client.crt \
      --key /tmp/client.key \
      "$API_SERVER/api/v1/namespaces" | python3 -m json.tool | head -30
```

Lab 5.2 — Bearer Token Authentication

Find the token for the default service account:

```
SA_SECRET=$(kubectl get secret -n default \
  -o jsonpath='{.items[?(@.type=="kubernetes.io/service-account-token")].m
  | awk '{print $1}')
```

```
TOKEN=$(kubectl get secret $SA_SECRET -n default \
  -o jsonpath='{.data.token}' | base64 -d)
echo "Token (first 60 chars): ${TOKEN:0:60}..."
```

Use the token to query the API:

```
curl --cacert /tmp/ca.crt \
      -H "Authorization: Bearer $TOKEN" \
      "$API_SERVER/api/v1/namespaces" | python3 -m json.tool | head -20
```

Note

The default service account has limited permissions. You will see 403 Forbidden for most resources — this is expected and demonstrates RBAC in action.

Lab 5.3 — kubectl proxy

`kubectl proxy` opens a local HTTP proxy to the API server, handling authentication automatically:

```
# Start proxy in the background
kubectl proxy --port=8001 &
PROXY_PID=$!

# Query through the proxy (no auth needed)
curl http://localhost:8001/api/v1/nodes | python3 -m json.tool | head -20

# List available API groups
curl http://localhost:8001/apis | python3 -m json.tool | head -30

# Stop the proxy
kill $PROXY_PID
```

Lab 5.4 — Trace kubectl with strace

Use strace to observe the system calls (including network calls) kubectl makes:

```
sudo apt-get install -y strace
```

```
# Trace kubectl get nodes and capture output
strace -e trace=network kubectl get nodes 2>&1 | head -40
```

Observe the `connect()` calls to the API server's IP and port (6443).

Lab 5.5 — API Discovery

Explore the API resource groups available in your cluster:

```
# All API resources
kubectl api-resources | head -20

# Available API versions
kubectl api-versions | sort

# Explain a resource schema
kubectl explain pod.spec.containers.resources
```

Check the API server's health endpoints:

```
curl --cacert /tmp/ca.crt \
     --cert /tmp/client.crt \
     --key /tmp/client.key \
     "$API_SERVER/healthz"

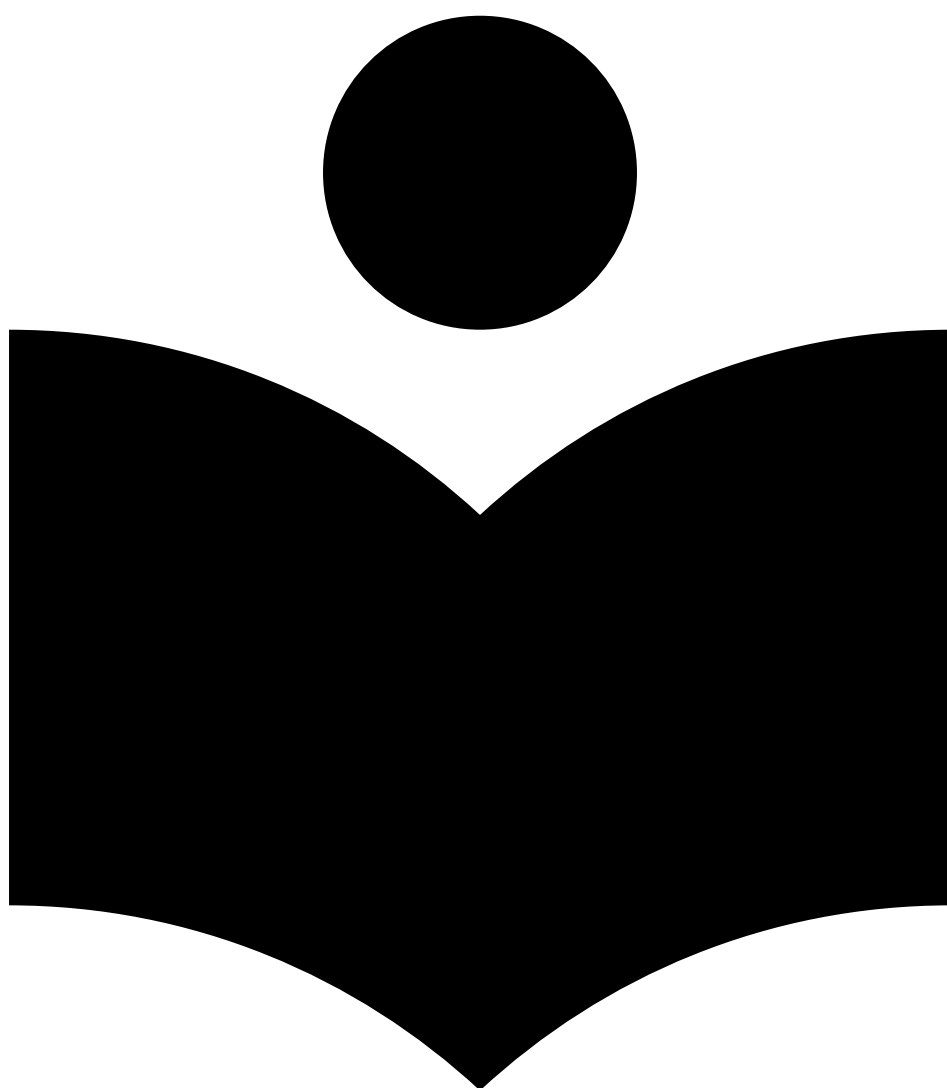
curl --cacert /tmp/ca.crt \
     --cert /tmp/client.crt \
     --key /tmp/client.key \
     "$API_SERVER/readyz"
```

Both should return ok.

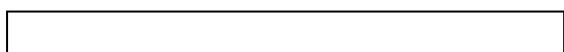
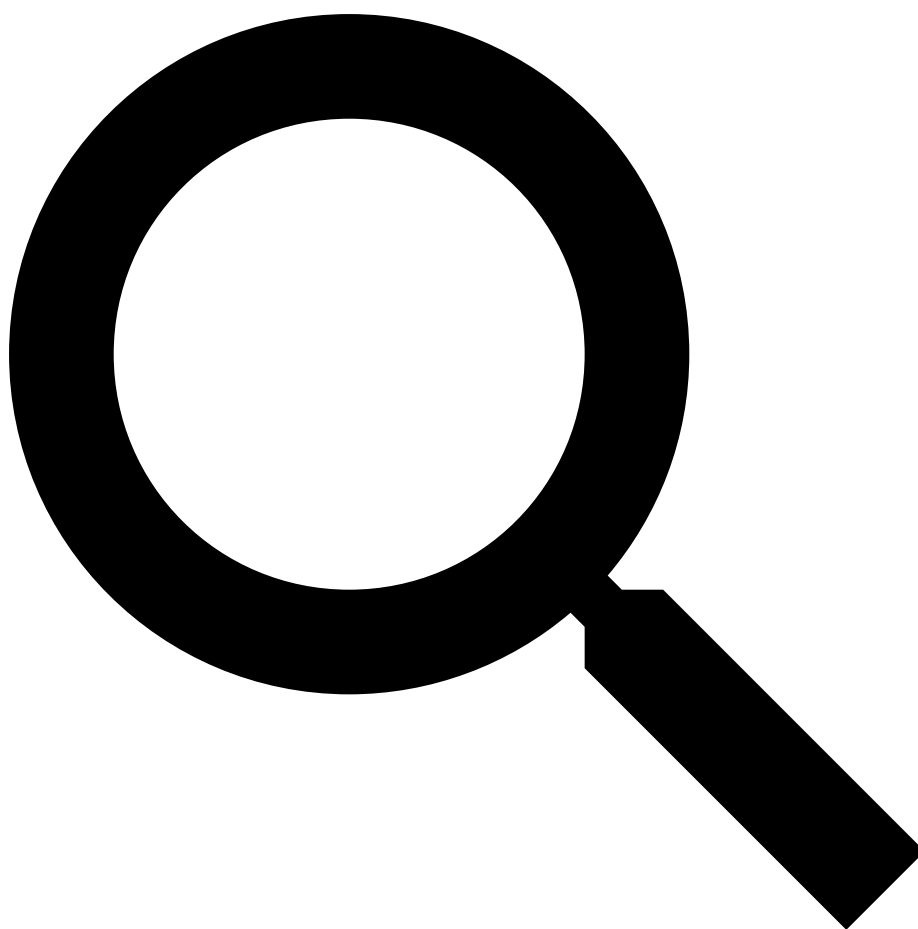
© 2025 VEGA TRAINING

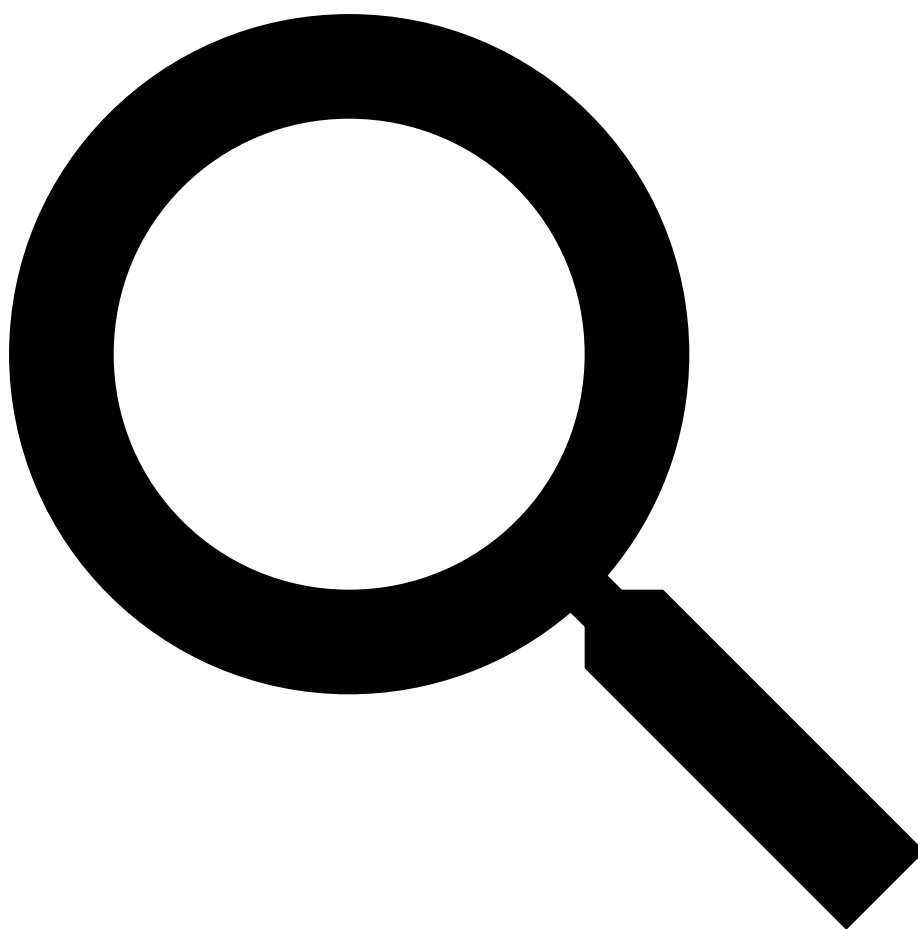


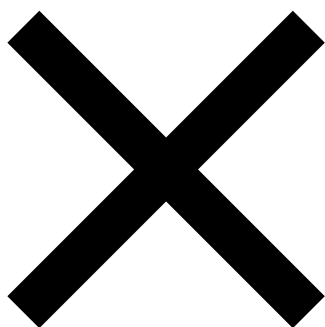
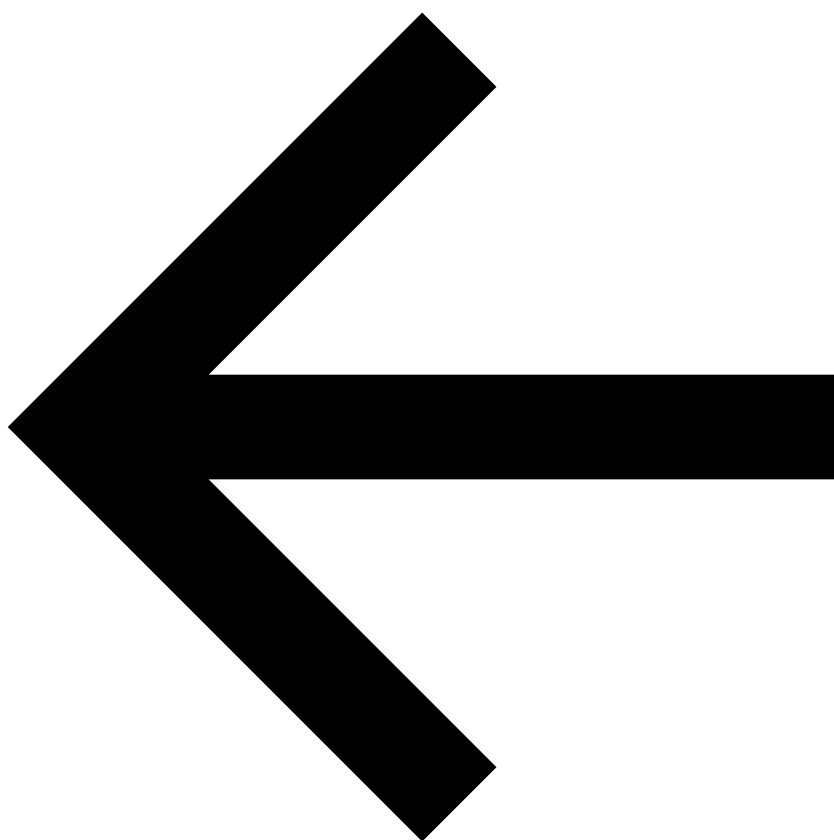
[Skip to content](#)





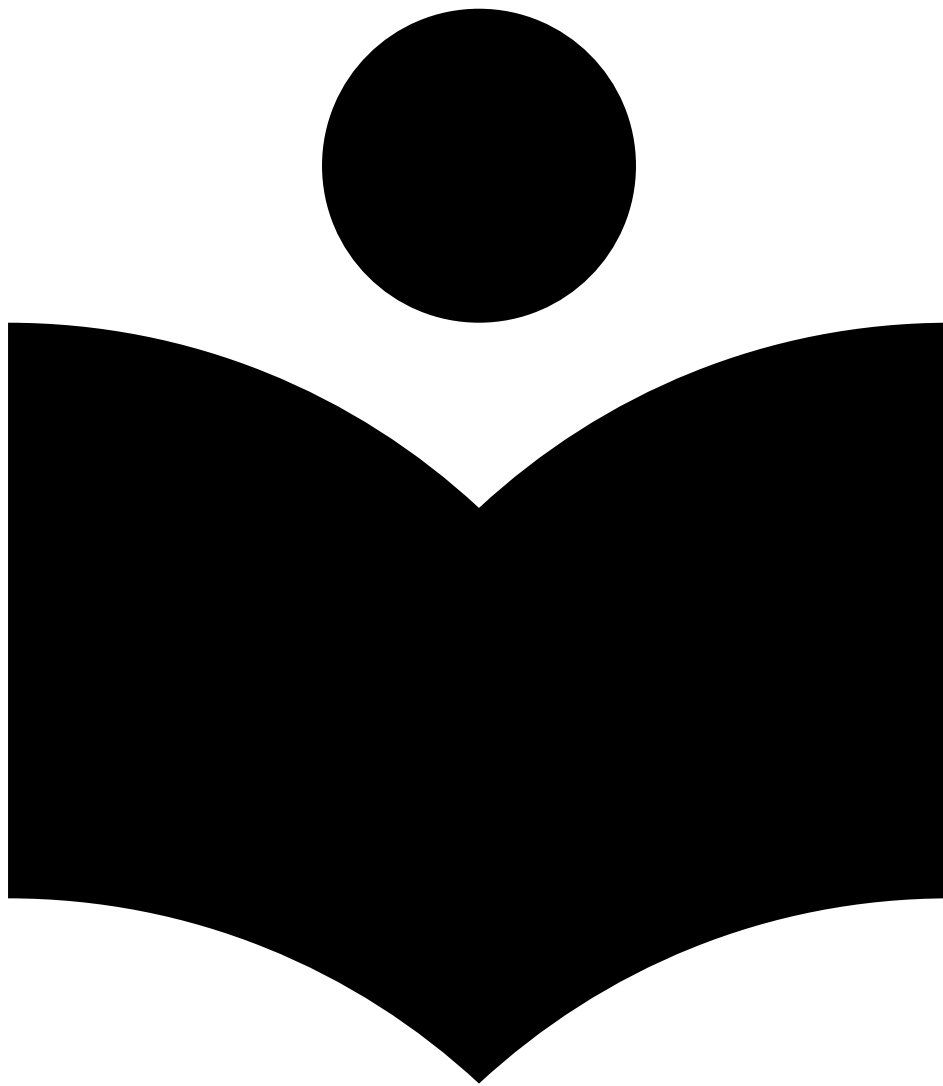






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- ☐ Ch 06: API Objects [Ch 06: API Objects](#)
 - Table of contents
 - [Overview](#)
 - [Lab 6.1 — Jobs](#)
 - [Create a basic Job](#)
 - [Add completions](#)
 - [Add parallelism](#)
 - [Add an active deadline](#)
 - [Clean up](#)
 - [Lab 6.2 — CronJobs](#)
 - [Add a job deadline](#)

- [Clean up](#)
 - [Lab 6.3 — ConfigMaps](#)
 - [Lab 6.4 — Secrets](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 6.1 — Jobs](#)
 - [Create a basic Job](#)
 - [Add completions](#)
 - [Add parallelism](#)
 - [Add an active deadline](#)
 - [Clean up](#)
- [Lab 6.2 — CronJobs](#)
 - [Add a job deadline](#)
 - [Clean up](#)
- [Lab 6.3 — ConfigMaps](#)
- [Lab 6.4 — Secrets](#)

Chapter 6: API Objects¶

Overview¶

In this lab you will work with:

- Jobs (one-shot, parallel, deadline-bounded)
- CronJobs (scheduled Jobs)
- ConfigMaps and Secrets as API objects

The YAML files are in `~/lfs458/ch06-api-objects/`. Chapters that require editing a file have a completed version in the `solutions/` subfolder.

Lab 6.1 — Jobs¶

Create a basic Job¶

```
cp ~/lfs458/ch06-api-objects/job.yaml ~/job.yaml
cat ~/job.yaml
```

Apply it and watch it complete:

```
kubectl apply -f ~/job.yaml
kubectl get jobs -w
```

Once complete, check the pod:

```
JOB_POD=$(kubectl get pod -l job-name=sleepy -o jsonpath='{.items[0].metadata.name}')
kubectl logs $JOB_POD
kubectl describe job sleepy
```

Add completions¶

Edit ~/job.yaml to run the job **5 times total** by setting spec.completions: 5. The solution is at ~/lfs458/ch06-api-objects/solutions/job-completions.yaml.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
kubectl get jobs -w
```

Add parallelism¶

Now also set spec.parallelism: 2 so 2 pods run at a time. The solution is at ~/lfs458/ch06-api-objects/solutions/job-parallelism.yaml.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
kubectl get pods -w # You should see 2 pods running simultaneously
```

Add an active deadline¶

Set spec.activeDeadlineSeconds: 15 so the job is killed if it hasn't finished in 15 seconds. The solution is at ~/lfs458/ch06-api-objects/solutions/job-deadline.yaml.

```
kubectl delete job sleepy
kubectl apply -f ~/job.yaml
watch kubectl get jobs
```

Observe the job being terminated early with reason DeadlineExceeded.

Clean up¶

```
kubectl delete job sleepy 2>/dev/null; true
```

Lab 6.2 — CronJobs¶

```
cp ~/lfs458/ch06-api-objects/cronjob.yaml ~/cronjob.yaml
cat ~/cronjob.yaml
```

Apply and observe:

```
kubectl apply -f ~/cronjob.yaml
kubectl get cronjobs -w
```

The CronJob runs every minute. Wait for the first execution:

```
kubectl get jobs --watch
```

Once a Job appears, check its pod's logs:

```
CRON_POD=$(kubectl get pod -l app=date-job --sort-by=.metadata.creationTime
-o jsonpath='{.items[-1].metadata.name}')
kubectl logs $CRON_POD
```

Add a job deadline¶

Edit ~/cronjob.yaml to add spec.jobTemplate.spec.activeDeadlineSeconds: 10 and change the sleep duration so the job exceeds the deadline. The solution is at ~/lfs458/ch06-api-objects/solutions/cronjob-deadline.yaml.

```
kubectl delete cronjob date-job
kubectl apply -f ~/cronjob.yaml
kubectl get jobs -w
```

Clean up¶

```
kubectl delete cronjob date-job 2>/dev/null; true
```

Lab 6.3 — ConfigMaps¶

Create a ConfigMap from literal values:

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
```

```
kubectl describe configmap app-config
kubectl get configmap app-config -o yaml
```

Use the ConfigMap in a pod as environment variables:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: config-test
spec:
  containers:
  - name: app
```

```
image: nginx
envFrom:
- configMapRef:
    name: app-config
EOF
```

Verify the env vars are present:

```
kubectl exec config-test -- env | grep -E "APP_ENV|LOG_LEVEL"
```

Clean up:

```
kubectl delete pod config-test
kubectl delete configmap app-config
```

Lab 6.4 — Secrets

Create a Secret:

```
kubectl create secret generic db-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=s3cr3t!
```

```
kubectl get secret db-secret -o yaml
```

Notice the data values are base64-encoded (not encrypted at rest by default). Decode one:

```
kubectl get secret db-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
echo # newline
```

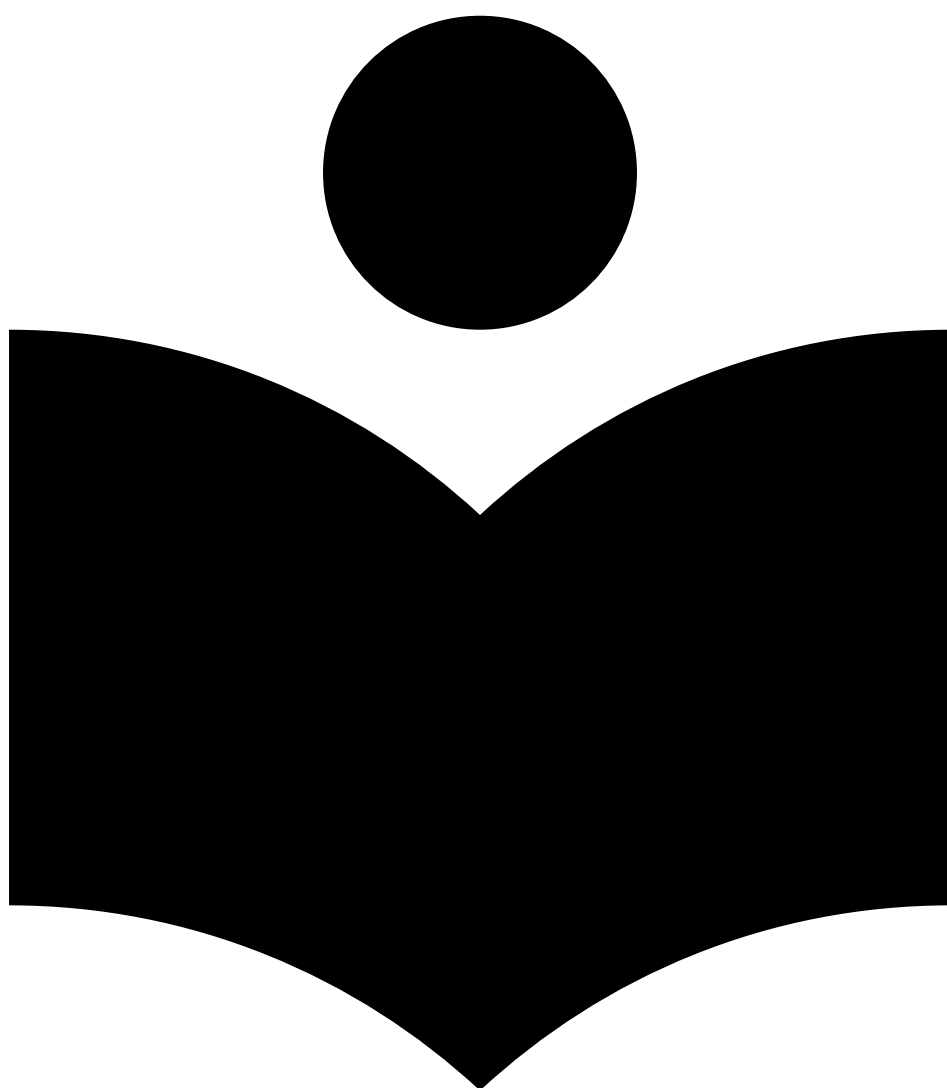
Clean up:

```
kubectl delete secret db-secret
```

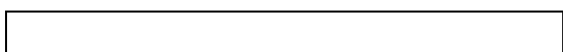
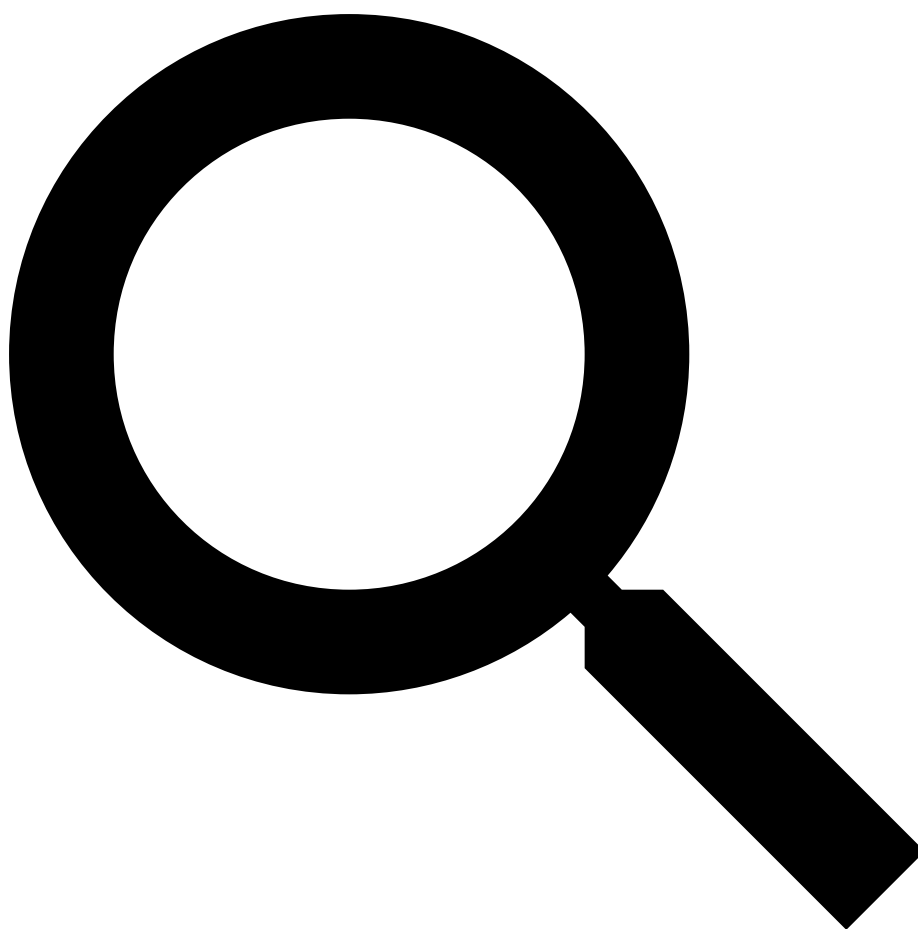
© 2025 VEGA TRAINING

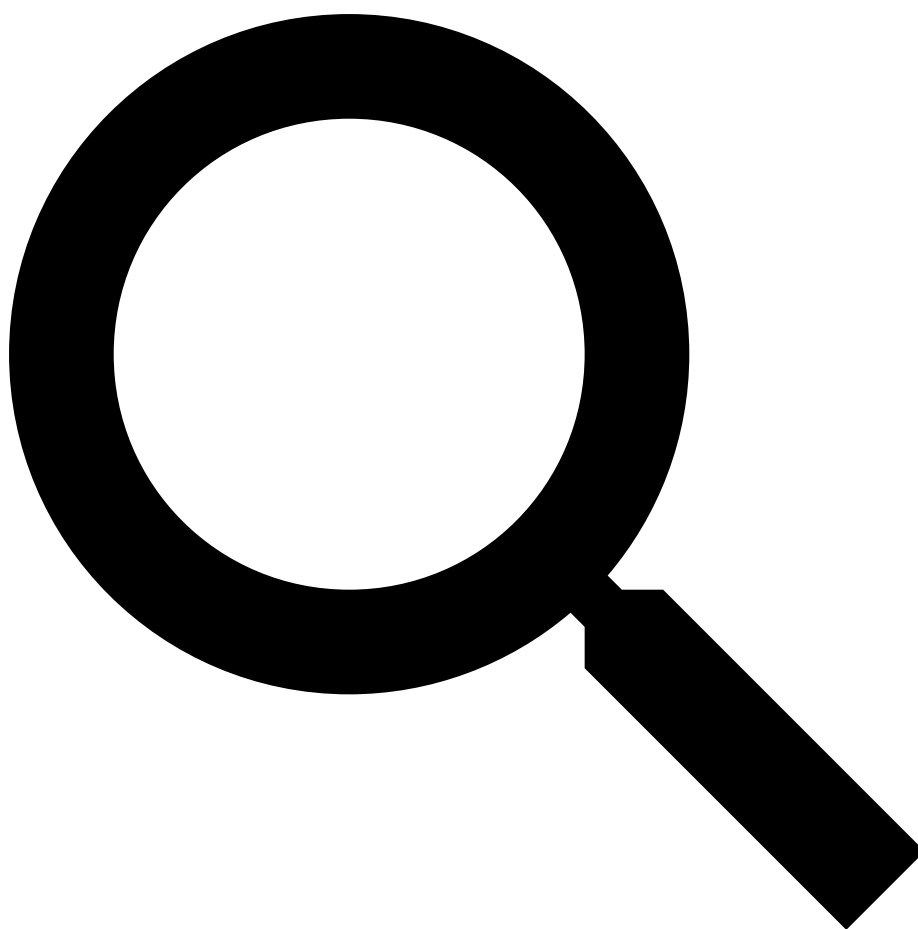


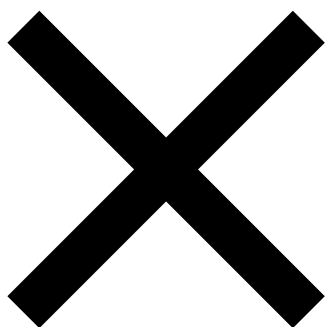
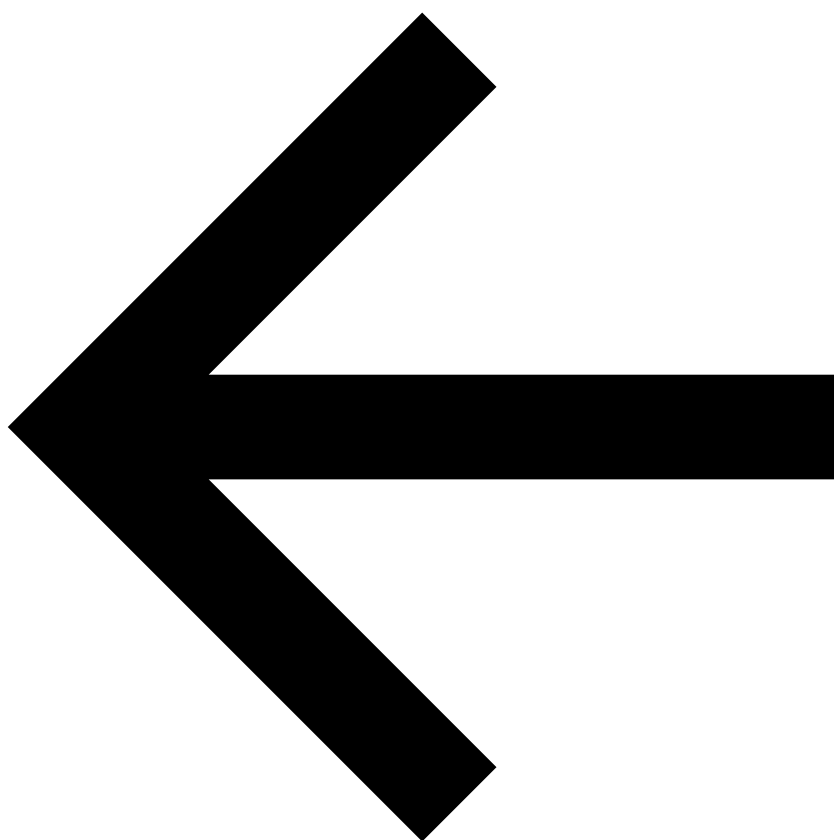
[Skip to content](#)





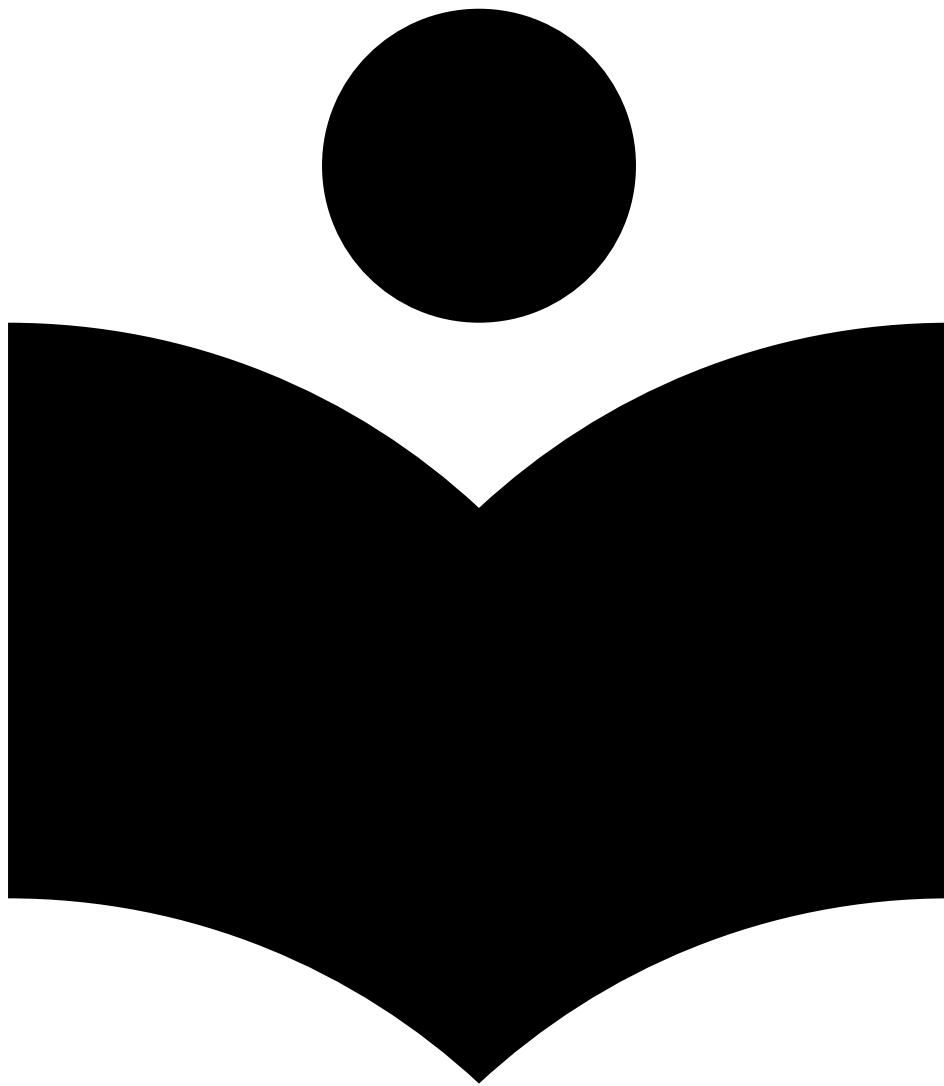






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
 - [Ch 03: Install Kubernetes](#)
 - [Ch 04: Architecture](#)
 - [Ch 05: API Access](#)
 - [Ch 06: API Objects](#)
 - ☐ Ch 07: Deployments [Ch 07: Deployments](#)
- Table of contents
- [Overview](#)
 - [Lab 7.1 — ReplicaSets](#)
 - [Clean up](#)
 - [Lab 7.2 — Deployments](#)
 - [Scale the Deployment](#)
 - [Update the image \(rolling update\)](#)
 - [Rollback](#)
 - [Pause and resume a rollout](#)

- [Clean up](#)
 - [Lab 7.3 — DaemonSets](#)
 - [Switch to RollingUpdate](#)
 - [Clean up](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 7.1 — ReplicaSets](#)
 - [Clean up](#)
- [Lab 7.2 — Deployments](#)
 - [Scale the Deployment](#)
 - [Update the image \(rolling update\)](#)
 - [Rollback](#)
 - [Pause and resume a rollout](#)
 - [Clean up](#)
- [Lab 7.3 — DaemonSets](#)
 - [Switch to RollingUpdate](#)
 - [Clean up](#)

Chapter 7: Deployments, ReplicaSets, and DaemonSets¶

Overview¶

In this lab you will:

- Work with ReplicaSets directly
- Create and manage Deployments (scale, rollout, rollback)
- Create a DaemonSet with OnDelete and RollingUpdate strategies

Lab files are in ~/lfs458/ch07-deployments/. The solutions/ subfolder contains completed manifests for files you need to edit.

Lab 7.1 — ReplicaSets¶

Create a ReplicaSet:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: webserver
  labels:
    app: webserver
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      labels:
        app: webserver
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
EOF
```

List pods managed by the ReplicaSet:

```
kubectl get rs webserver
kubectl get pods -l app=webserver
```

Delete one pod and watch the ReplicaSet recreate it:

```
RS_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $RS_POD
kubectl get pods -l app=webserver -w
```

Clean up

```
kubectl delete rs webserver
```

Lab 7.2 — Deployments

Create a Deployment:

```
kubectl create deployment webserver --image=nginx:1.25 --replicas=3
kubectl rollout status deployment webserver
kubectl get pods -l app=webserver
```

Inspect the Deployment:

```
WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

Scale the Deployment¶

```
kubectl scale deployment webserver --replicas=5
kubectl get pods -l app=webserver -w
```

Update the image (rolling update)¶

```
kubectl set image deployment/webserver nginx=nginx:1.26
kubectl rollout status deployment webserver
```

Watch the update progress — old pods are terminated one by one as new ones become ready:

```
kubectl get pods -l app=webserver -w
```

Verify the new image is running:

```
WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

Rollback¶

```
kubectl rollout history deployment webserver
kubectl rollout undo deployment webserver
kubectl rollout status deployment webserver
```

```
WEB_POD=$(kubectl get pod -l app=webserver -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $WEB_POD | grep Image:
```

The image should be back to nginx:1.25.

Pause and resume a rollout¶

```
# Start an update and immediately pause
kubectl set image deployment/webserver nginx=nginx:1.27
kubectl rollout pause deployment webserver
```

```
# Inspect – only some pods updated
kubectl get pods -l app=webserver
```

```
# Resume
kubectl rollout resume deployment webserver
kubectl rollout status deployment webserver
```

Clean up¶

```
kubectl delete deployment webserver
```

Lab 7.3 — DaemonSets

Create a DaemonSet with the `OnDelete` update strategy so you control when each node's pod is updated:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ds-one
  labels:
    app: ds-one
spec:
  updateStrategy:
    type: OnDelete
  selector:
    matchLabels:
      system: ds-one
  template:
    metadata:
      labels:
        system: ds-one
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
EOF
```

Verify the DaemonSet runs one pod per node:

```
kubectl get ds ds-one
kubectl get pods -l system=ds-one -o wide
```

Update the image:

```
kubectl set image ds/ds-one nginx=nginx:1.26
kubectl rollout status ds/ds-one
```

With `OnDelete`, pods are only updated when you delete them manually:

```
DS_POD=$(kubectl get pod -l system=ds-one -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $DS_POD
# A new pod is created with the updated image
kubectl get pods -l system=ds-one -o wide
```

Verify the updated pod runs the new image:

```
DS_POD=$(kubectl get pod -l system=ds-one -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $DS_POD | grep Image:
```


Switch to RollingUpdate¶

Edit the DaemonSet in place:

```
kubectl patch ds ds-one -p '{"spec":{"updateStrategy":{"type":"RollingUpdate"}}}'
kubectl get ds ds-one -o jsonpath='{.spec.updateStrategy.type}'
```

Now update the image — all pods roll over automatically:

```
kubectl set image ds/ds-one nginx=nginx:1.27
kubectl rollout status ds/ds-one
```

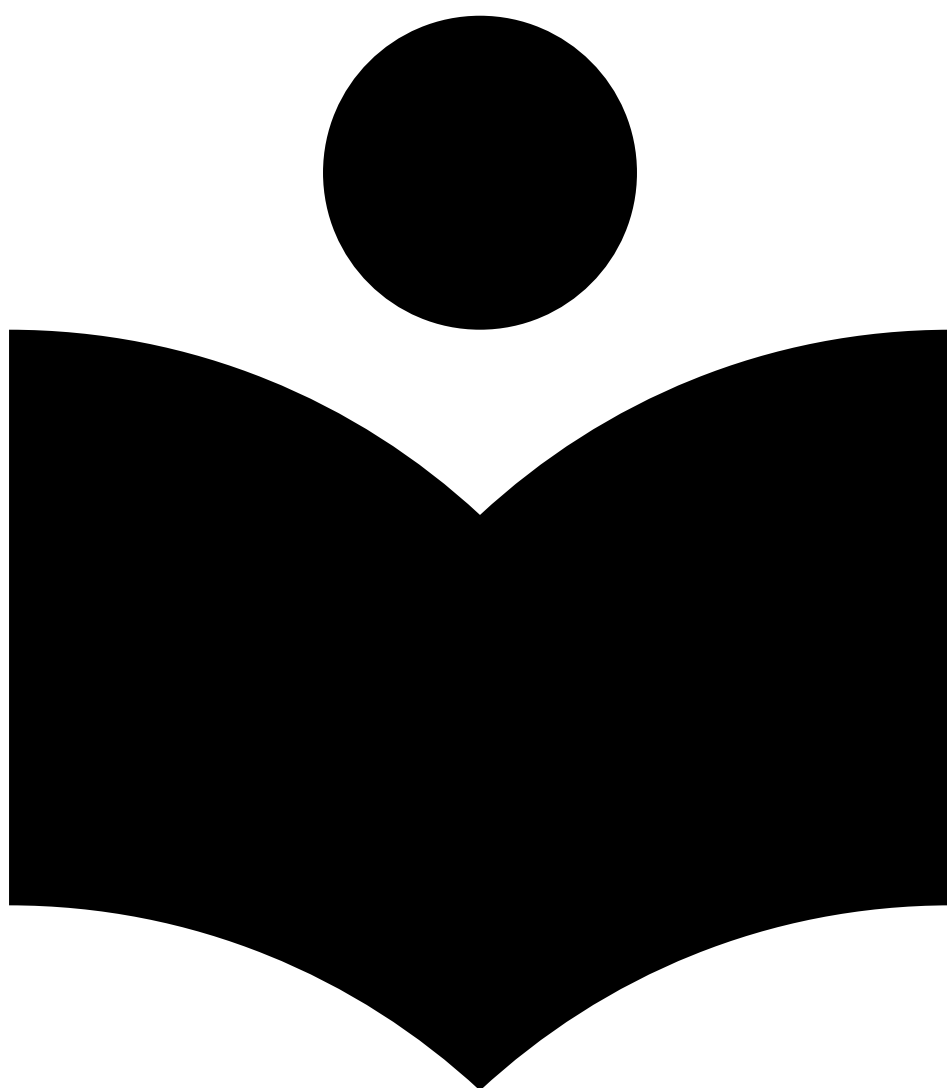
Clean up¶

```
kubectl delete ds ds-one
```

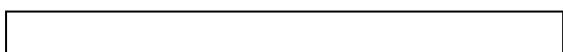
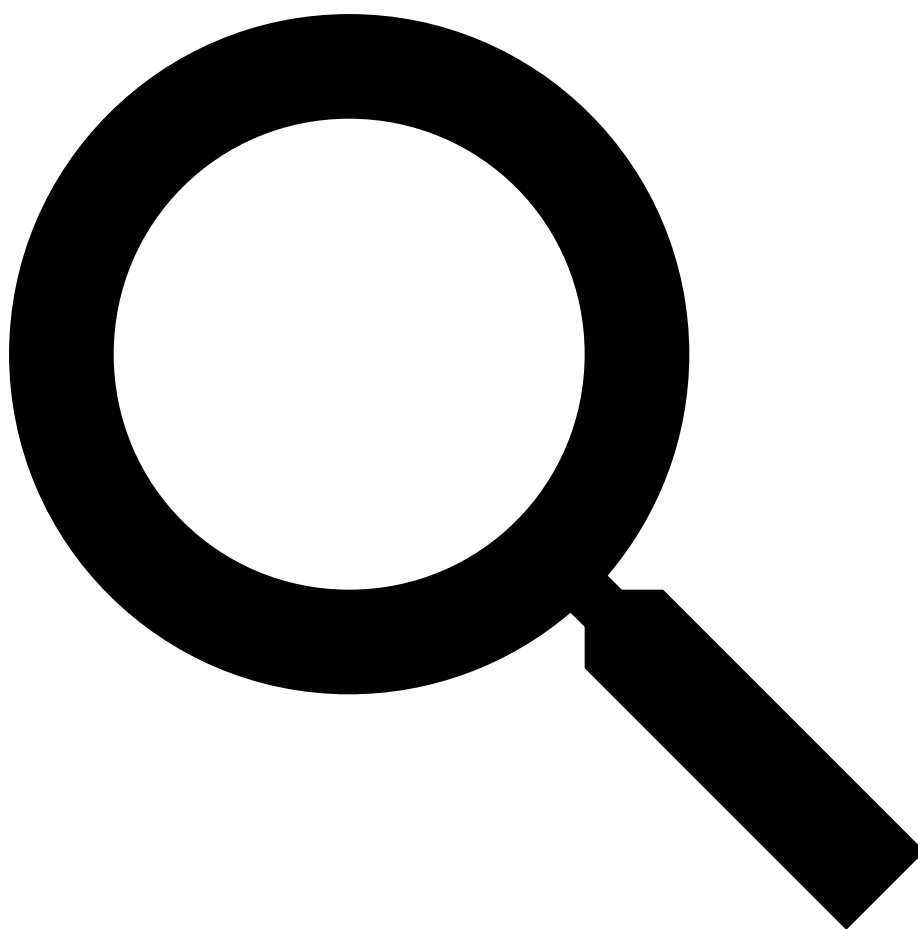
© 2025 VEGA TRAINING

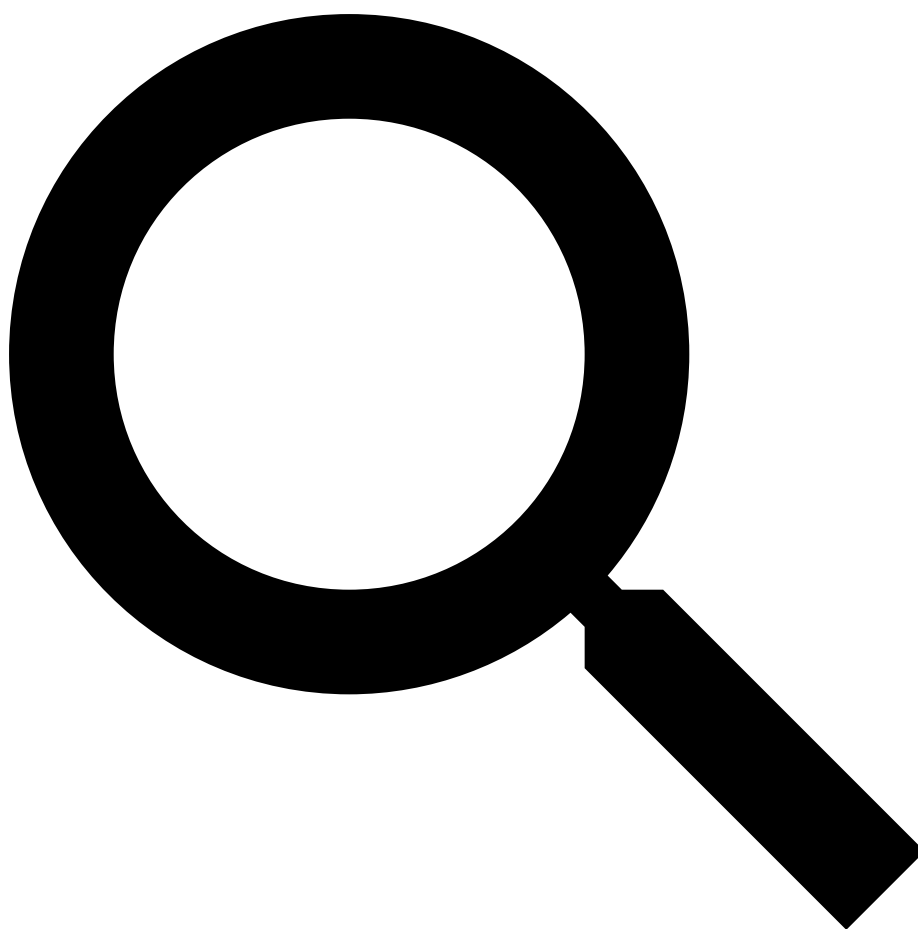


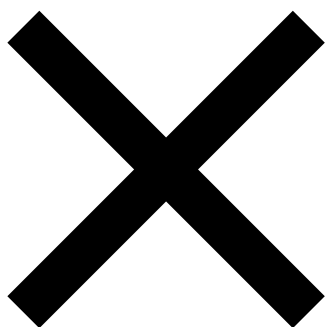
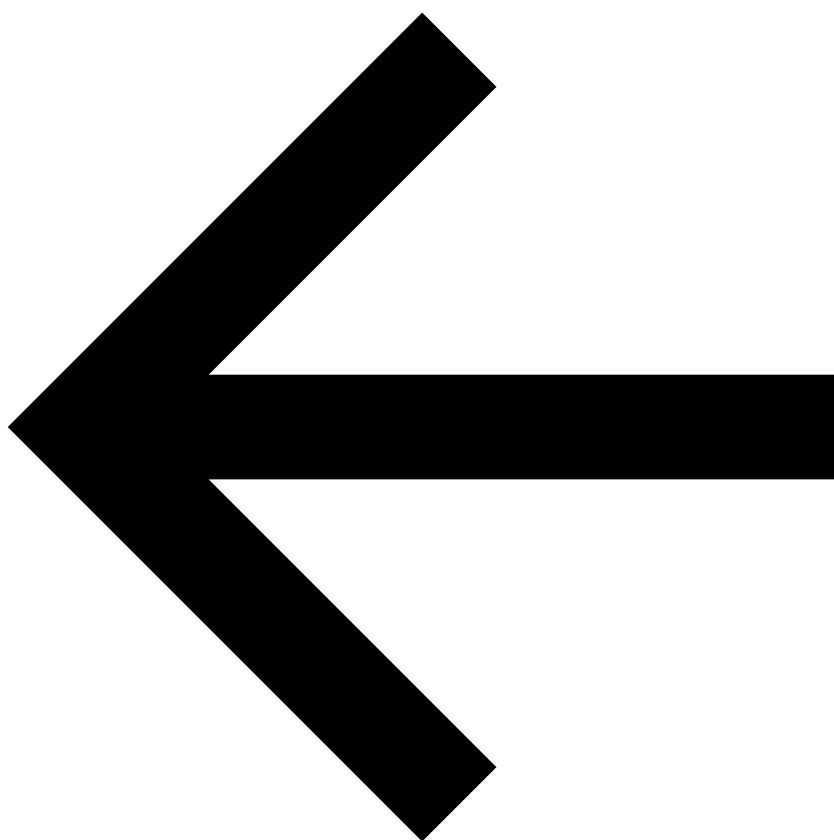
[Skip to content](#)





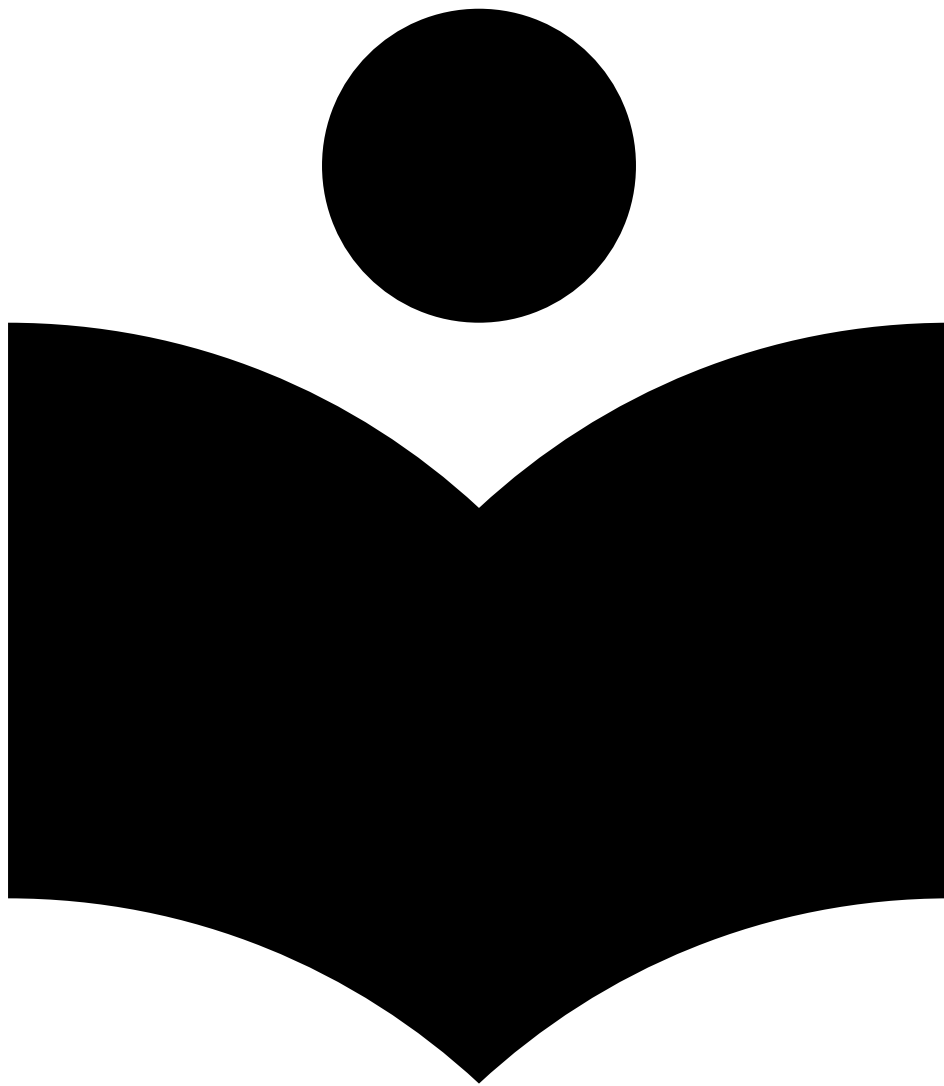






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
 - [Ch 03: Install Kubernetes](#)
 - [Ch 04: Architecture](#)
 - [Ch 05: API Access](#)
 - [Ch 06: API Objects](#)
 - [Ch 07: Deployments](#)
 - ☐ Ch 08: Helm & Kustomize [Ch 08: Helm & Kustomize](#)
- Table of contents
- [Overview](#)
 - [Lab 8.1 — Install Helm](#)
 - [Lab 8.2 — Deploy an Application with Helm](#)
 - [Lab 8.3 — Install metrics-server](#)
 - [Lab 8.4 — Horizontal Pod Autoscaler](#)
 - [Deploy a CPU-hungry application](#)
 - [Generate load with siege](#)

- [Clean up](#)
- [Lab 8.5 — Kustomize](#)
 - [Create the base](#)
 - [Create the dev overlay](#)
 - [Create the prod overlay](#)
 - [Preview and apply](#)
 - [Clean up](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 8.1 — Install Helm](#)
- [Lab 8.2 — Deploy an Application with Helm](#)
- [Lab 8.3 — Install metrics-server](#)
- [Lab 8.4 — Horizontal Pod Autoscaler](#)
 - [Deploy a CPU-hungry application](#)
 - [Generate load with siege](#)
 - [Clean up](#)
- [Lab 8.5 — Kustomize](#)
 - [Create the base](#)
 - [Create the dev overlay](#)
 - [Create the prod overlay](#)
 - [Preview and apply](#)
 - [Clean up](#)

Chapter 8: Helm and Kustomize¶

Overview¶

In this lab you will:

- Install and use Helm to deploy an application from a chart
 - Install metrics-server with Helm and configure it for insecure TLS (lab environment)
 - Configure Horizontal Pod Autoscaling (HPA) and generate load with siege
 - Use Kustomize to manage environment-specific configuration overlays
-

Lab 8.1 — Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
helm version
```

Add a repository and search it:

```
helm repo add stable https://charts.helm.sh/stable  
helm repo add bitnami https://charts.bitnami.com/bitnami  
helm repo update
```

```
helm search repo bitnami/nginx
```

Lab 8.2 — Deploy an Application with Helm

Install an nginx chart into its own namespace:

```
kubectl create namespace helm-nginx  
helm install my-nginx bitnami/nginx --namespace helm-nginx
```

Watch the pods come up:

```
kubectl get pods -n helm-nginx -w
```

Inspect what Helm created:

```
helm list -n helm-nginx  
helm status my-nginx -n helm-nginx  
kubectl get all -n helm-nginx
```

Clean up:

```
helm uninstall my-nginx -n helm-nginx  
kubectl delete namespace helm-nginx
```

Lab 8.3 — Install metrics-server

metrics-server is required for kubectl top and for HPA to function. In a lab environment, the kubelets use self-signed certificates, so you must pass --kubelet-insecure-tls.

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server  
helm repo update
```

```
helm upgrade --install metrics-server metrics-server/metrics-server \\  
  --namespace kube-system \\  
  --set args="{--kubelet-insecure-tls}"
```

Wait for it to be ready:

```
kubectl rollout status deployment metrics-server -n kube-system --timeout=
```

Verify:

```
kubectl top nodes
kubectl top pods -A
```

Lab 8.4 — Horizontal Pod Autoscaler

Deploy a CPU-hungry application

```
kubectl create deployment php-apache \
  --image=registry.k8s.io/hpa-example \
  --requests=cpu=200m
```

```
kubectl expose deployment php-apache --port=80
```

Create the HPA:

```
kubectl autoscale deployment php-apache \
  --cpu-percent=50 \
  --min=1 \
  --max=10
```

```
kubectl get hpa php-apache -w
```

Generate load with siege

```
sudo apt-get install -y siege
```

```
PHP_SVC_IP=$(kubectl get svc php-apache -o jsonpath='{.spec.clusterIP}')
siege -c 20 -t 2M http://$PHP_SVC_IP &
SIEGE_PID=$!
```

```
# Watch HPA scale up
kubectl get hpa php-apache -w
```

Stop the load:

```
kill $SIEGE_PID 2>/dev/null
# HPA will scale back down after ~5 minutes
kubectl get hpa php-apache -w
```

Clean up

```
kubectl delete hpa php-apache
kubectl delete deployment php-apache
kubectl delete svc php-apache
```

Lab 8.5 — Kustomize

Kustomize is built into kubectl. Create a base application and two overlays (dev and prod).

Create the base

```
mkdir -p ~/kustomize/base ~/kustomize/overlays/dev ~/kustomize/overlays/prod
```

```
cat <<EOF > ~/kustomize/base/deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.25
```

```
EOF
```

```
cat <<EOF > ~/kustomize/base/kustomization.yaml
```

```
resources:
- deployment.yaml
EOF
```

Create the dev overlay

```
cat <<EOF > ~/kustomize/overlays/dev/kustomization.yaml
```

```
bases:
- ../../base
patches:
- patch: |-
    - op: replace
      path: /spec/replicas
      value: 1
    - op: replace
      path: /spec/template/spec/containers/0/image
      value: nginx:1.25
target:
  kind: Deployment
  name: myapp
```

```
namePrefix: dev-  
EOF
```

Create the prod overlay¶

```
cat <<EOF > ~/kustomize/overlays/prod/kustomization.yaml  
bases:  
- ../../base  
patches:  
- patch: |-  
    - op: replace  
      path: /spec/replicas  
      value: 3  
    - op: replace  
      path: /spec/template/spec/containers/0/image  
      value: nginx:1.26  
target:  
  kind: Deployment  
  name: myapp  
namePrefix: prod-  
EOF
```

Preview and apply¶

```
# Preview what will be applied  
kubectl kustomize ~/kustomize/overlays/dev  
kubectl kustomize ~/kustomize/overlays/prod
```

```
# Apply dev  
kubectl apply -k ~/kustomize/overlays/dev  
kubectl get deployments | grep dev-myapp
```

```
# Apply prod  
kubectl apply -k ~/kustomize/overlays/prod  
kubectl get deployments | grep prod-myapp
```

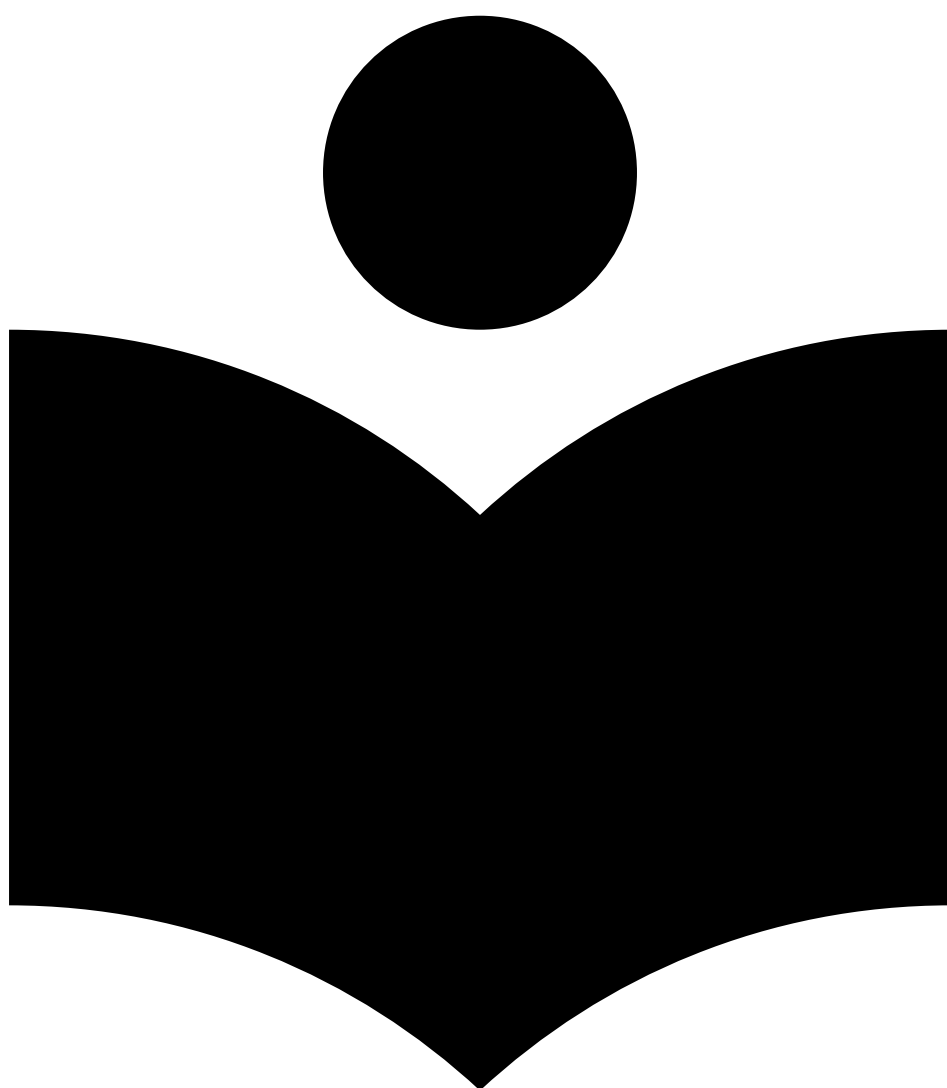
Clean up¶

```
kubectl delete -k ~/kustomize/overlays/dev  
kubectl delete -k ~/kustomize/overlays/prod
```

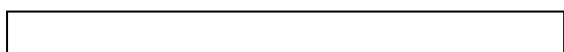
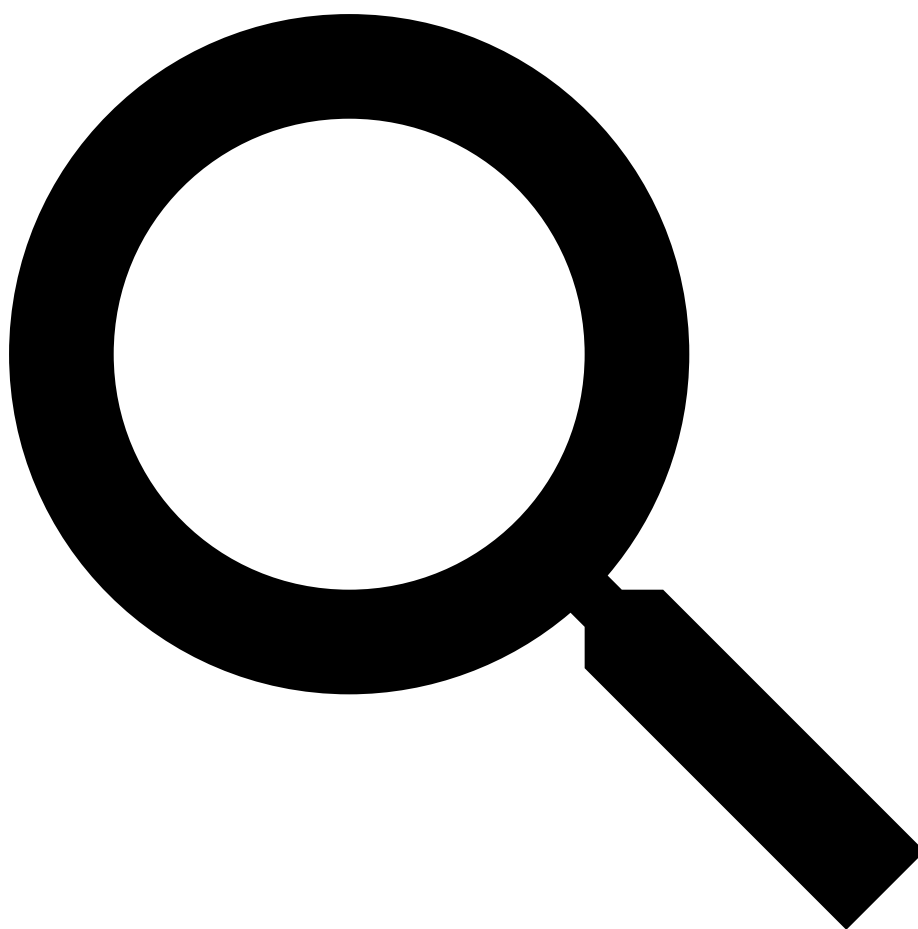
© 2025 VEGA TRAINING

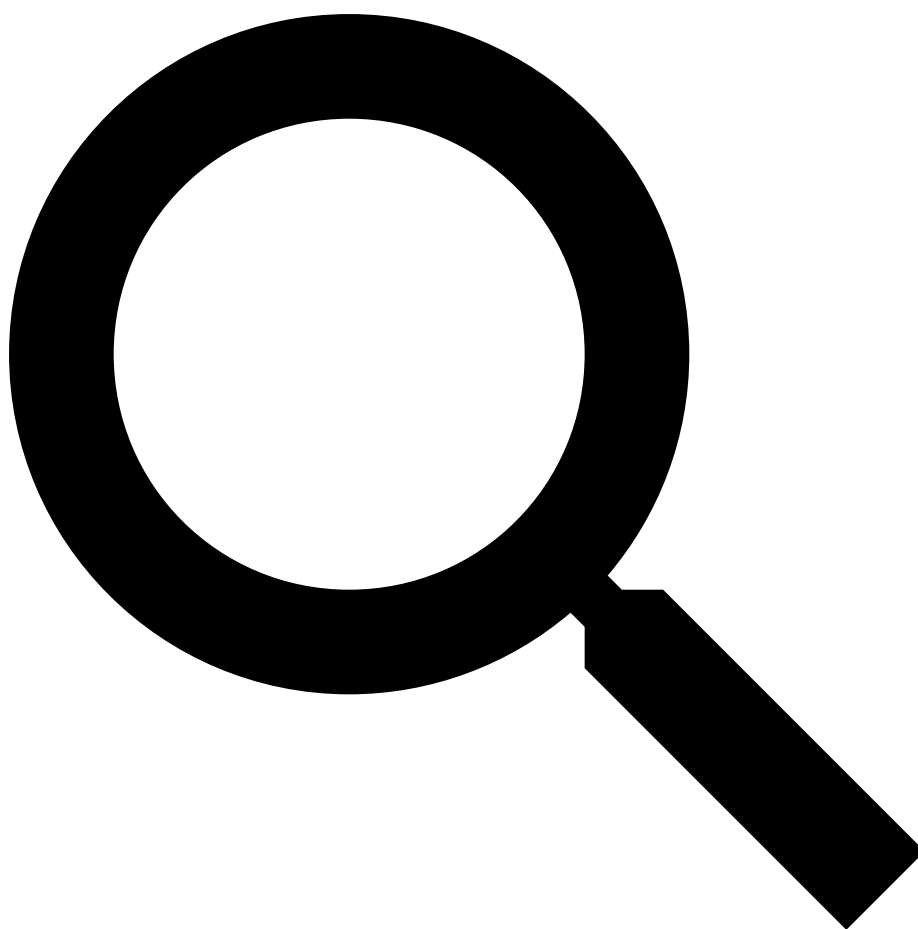


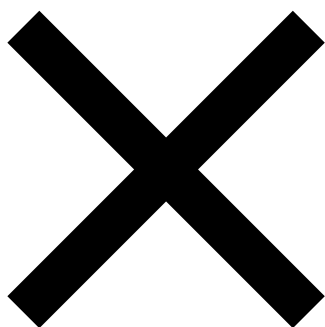
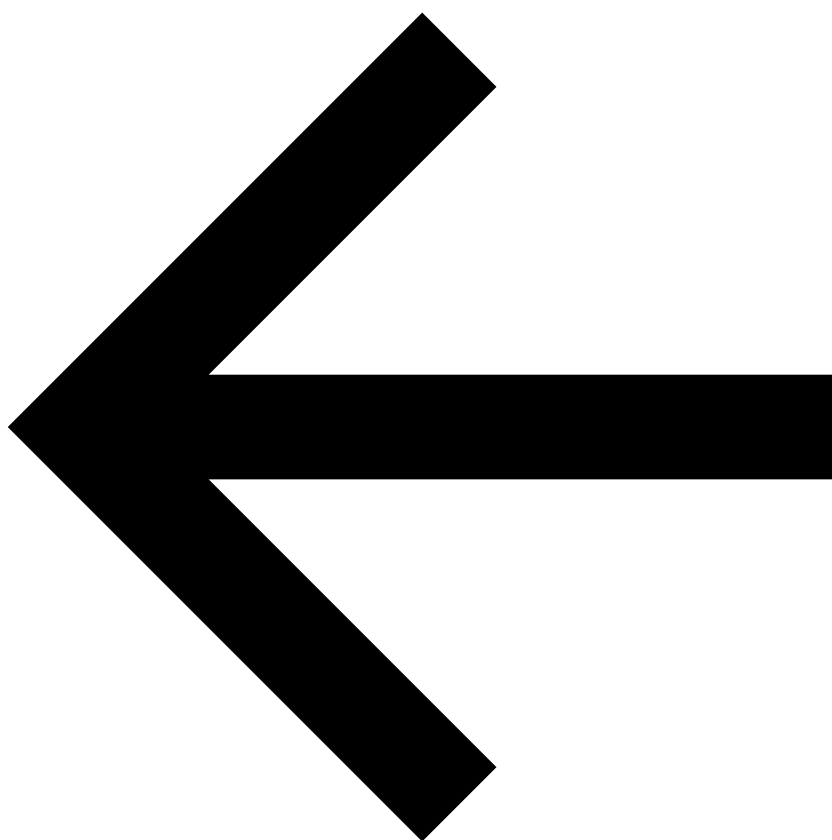
[Skip to content](#)





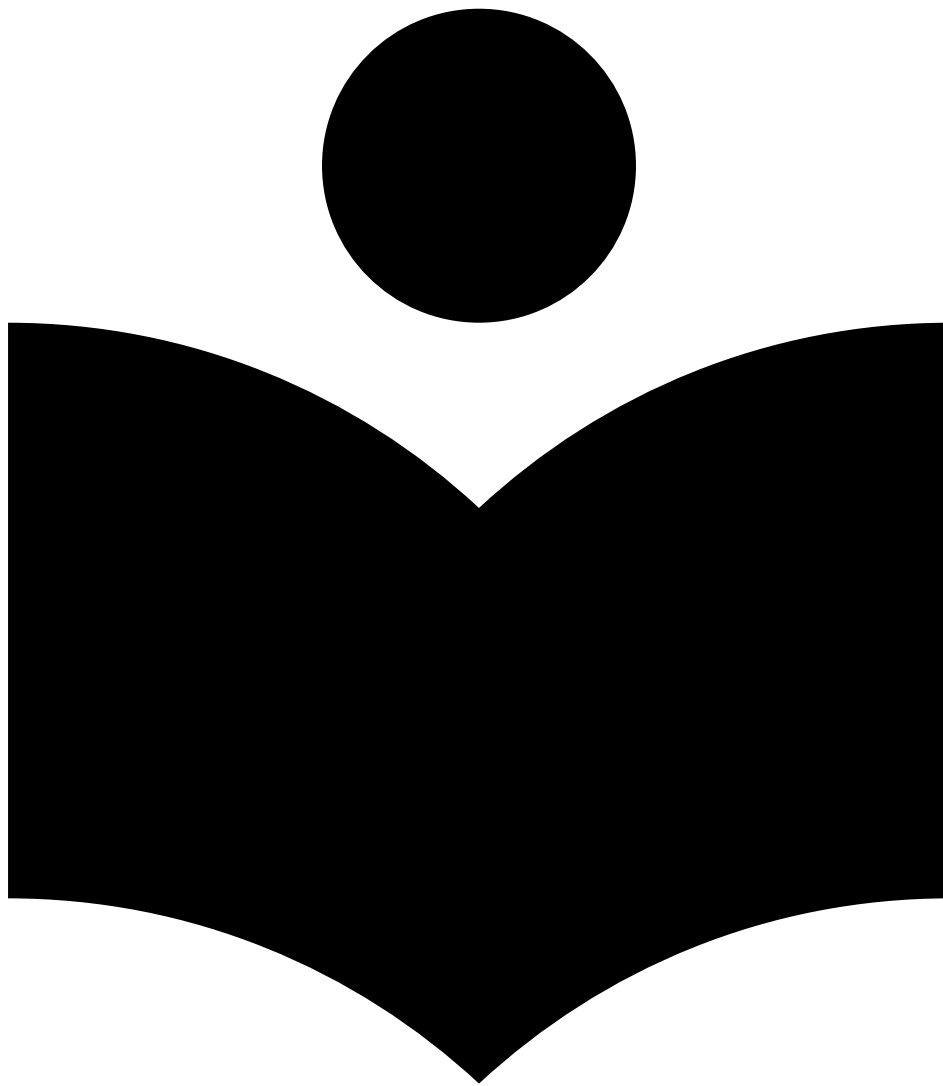






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
 - [Ch 03: Install Kubernetes](#)
 - [Ch 04: Architecture](#)
 - [Ch 05: API Access](#)
 - [Ch 06: API Objects](#)
 - [Ch 07: Deployments](#)
 - [Ch 08: Helm & Kustomize](#)
 - ☐ Ch 09: Volumes [Ch 09: Volumes](#)
- Table of contents
- [Overview](#)
 - [Lab 9.1 — ConfigMaps as Volumes](#)
 - [Lab 9.2 — Using simpleshell.yaml](#)
 - [Stage 1 — Add envFrom](#)
 - [Stage 2 — Add a volumeMount](#)

- [Lab 9.3 — NFS PersistentVolume](#)
 - [Set up NFS server \(controller only\)](#)
 - [Install NFS client on workers \(worker1 and worker2\)](#)
 - [Create the PersistentVolume](#)
 - [Create a PersistentVolumeClaim](#)
 - [Use the PVC in a pod](#)
 - [Clean up](#)
- [Lab 9.4 — ResourceQuota for Storage](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 9.1 — ConfigMaps as Volumes](#)
- [Lab 9.2 — Using simpleshell.yaml](#)
 - [Stage 1 — Add envFrom](#)
 - [Stage 2 — Add a volumeMount](#)
- [Lab 9.3 — NFS PersistentVolume](#)
 - [Set up NFS server \(controller only\)](#)
 - [Install NFS client on workers \(worker1 and worker2\)](#)
 - [Create the PersistentVolume](#)
 - [Create a PersistentVolumeClaim](#)
 - [Use the PVC in a pod](#)
 - [Clean up](#)
- [Lab 9.4 — ResourceQuota for Storage](#)

Chapter 9: Volumes and Storage¶

Overview¶

In this lab you will:

- Use ConfigMaps and Secrets as volumes
- Create a PersistentVolume backed by NFS
- Bind a PersistentVolumeClaim and use it in a pod
- Configure ResourceQuotas for storage
- Set up dynamic provisioning with a StorageClass

Lab files are in ~/lfs458/ch09-volumes/. Completed YAMLs for files you need to edit are in the solutions/ subfolder.

Lab 9.1 — ConfigMaps as Volumes

Create a ConfigMap containing an nginx config:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  nginx.conf: |
    server {
      listen 80;
      root /usr/share/nginx/html;
      index index.html;
    }
EOF
```

Mount it into a pod:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nginx-configmap
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: config-vol
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: config-vol
    configMap:
      name: nginx-conf
EOF
```

```
CM_POD=$(kubectl get pod nginx-configmap -o jsonpath='{.metadata.name}')
kubectl exec $CM_POD -- cat /etc/nginx/conf.d/nginx.conf
```

Clean up:

```
kubectl delete pod nginx-configmap
kubectl delete configmap nginx-conf
```

Lab 9.2 — Using simpleshell.yaml

The `simpleshell.yaml` file defines a basic container. You will modify it in two stages.


```
cp ~/lfs458/ch09-volumes/simpleshell.yaml ~/simpleshell.yaml
cat ~/simpleshell.yaml
```

Stage 1 — Add envFrom

Edit ~/simpleshell.yaml to add an envFrom section referencing the nginx-conf configmap (create a simple one first):

```
kubectl create configmap shell-config \
  --from-literal=MY_VAR=hello \
  --from-literal=ANOTHER_VAR=world
```

Add to ~/simpleshell.yaml under spec.containers[0]:

```
envFrom:
- configMapRef:
  name: shell-config
```

The solution is at ~/lfs458/ch09-volumes/solutions/simpleshell-envfrom.yaml.

Apply and verify:

```
kubectl apply -f ~/simpleshell.yaml
SS_POD=$(kubectl get pod simpleshell -o jsonpath='{.metadata.name}')
kubectl exec $SS_POD -- env | grep -E "MY_VAR|ANOTHER_VAR"
kubectl delete pod simpleshell
```

Stage 2 — Add a volumeMount

Edit ~/simpleshell.yaml further to mount a volume using a hostPath. The solution is at ~/lfs458/ch09-volumes/solutions/simpleshell-volume.yaml.

Apply and verify:

```
kubectl apply -f ~/simpleshell.yaml
SS_POD=$(kubectl get pod simpleshell -o jsonpath='{.metadata.name}')
kubectl exec $SS_POD -- ls /mnt/data
kubectl delete pod simpleshell
kubectl delete configmap shell-config
```

Lab 9.3 — NFS PersistentVolume

NFS Server

This lab assumes an NFS server is available. In the lab environment, the controller node can act as the NFS server.

Set up NFS server (controller only)

```
sudo apt-get install -y nfs-kernel-server
sudo mkdir -p /opt/sfw
sudo chmod 777 /opt/sfw
echo "/opt/sfw *(rw,sync,no_subtree_check,no_root_squash)" | sudo tee -a /etc/exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server
```

Verify the export:

```
showmount -e controller
```

Install NFS client on workers (worker1 and worker2)

```
sudo apt-get install -y nfs-common
```

Create the PersistentVolume

```
cp ~/lfs458/ch09-volumes/PVol.yaml ~/PVol.yaml
cat ~/PVol.yaml
```

Note the persistentVolumeReclaimPolicy. Edit ~/PVol.yaml to change it to Retain. The solution is at ~/lfs458/ch09-volumes/solutions/PVol-retain.yaml.

```
kubectl apply -f ~/PVol.yaml
kubectl get pv
```

Create a PersistentVolumeClaim

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 200Mi
  storageClassName: ""
EOF
```

```
kubectl get pvc nfs-pvc
```

Use the PVC in a pod

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
```

```
kind: Pod
metadata:
  name: nginx-nfs
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: nfs-vol
      mountPath: /usr/share/nginx/html
  volumes:
  - name: nfs-vol
    persistentVolumeClaim:
      claimName: nfs-pvc
EOF
```

```
NFS_POD=$(kubectl get pod nginx-nfs -o jsonpath='{.metadata.name}')
kubectl describe pod $NFS_POD | grep -A5 Volumes
```

Write a test file from the controller and read it in the pod:

```
sudo bash -c 'echo "<h1>Hello from NFS</h1>" > /opt/sfw/index.html'
kubectl exec $NFS_POD -- cat /usr/share/nginx/html/index.html
```

Clean up

```
kubectl delete pod nginx-nfs
kubectl delete pvc nfs-pvc
kubectl delete pv nfs-pv
```

Lab 9.4 — ResourceQuota for Storage

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ResourceQuota
metadata:
  name: storage-quota
  namespace: default
spec:
  hard:
    requests.storage: "500Mi"
    persistentvolumeclaims: "3"
EOF
```

```
kubectl describe resourcequota storage-quota
```

Try to create a PVC that exceeds the quota:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
```

```
metadata:
  name: big-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 600Mi
  storageClassName: ""
EOF
```

You should see a forbidden: exceeded quota error.

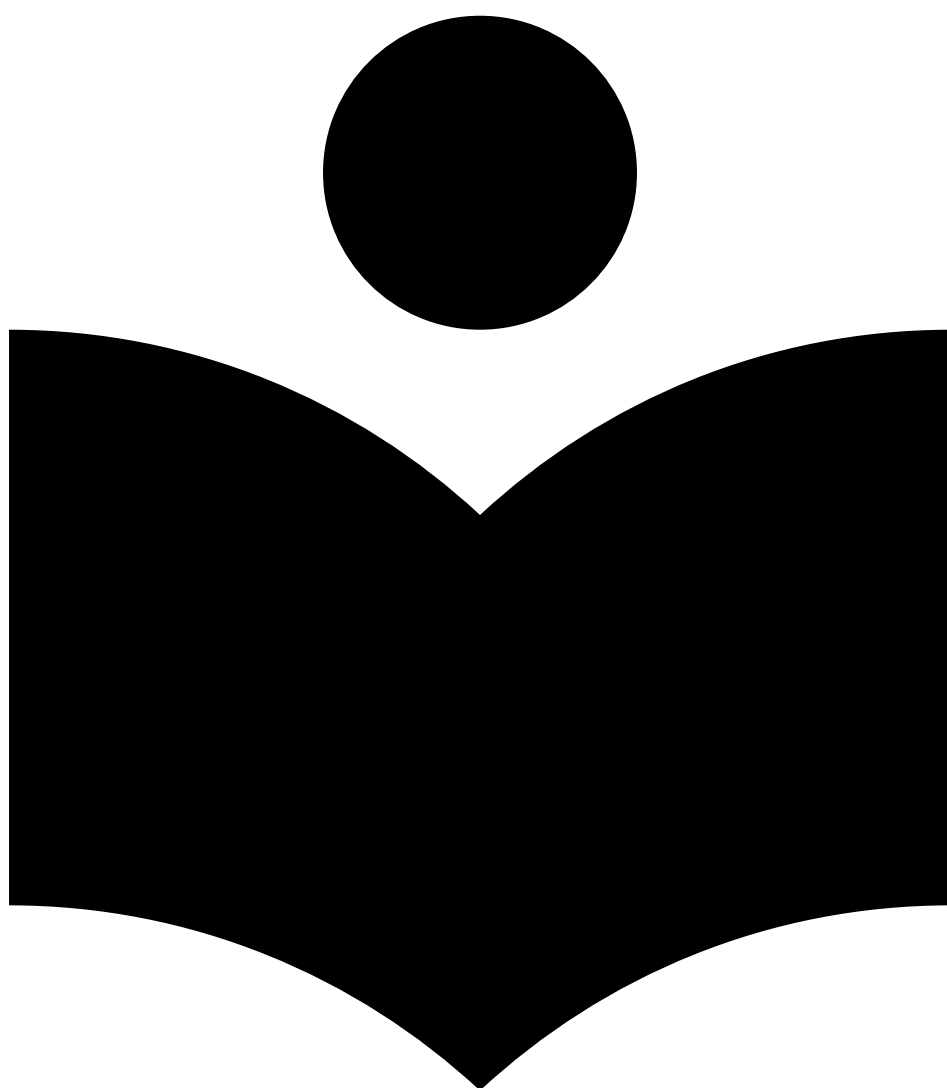
Clean up:

```
kubectl delete resourcequota storage-quota
kubectl delete pvc big-pvc 2>/dev/null; true
```

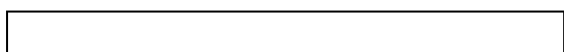
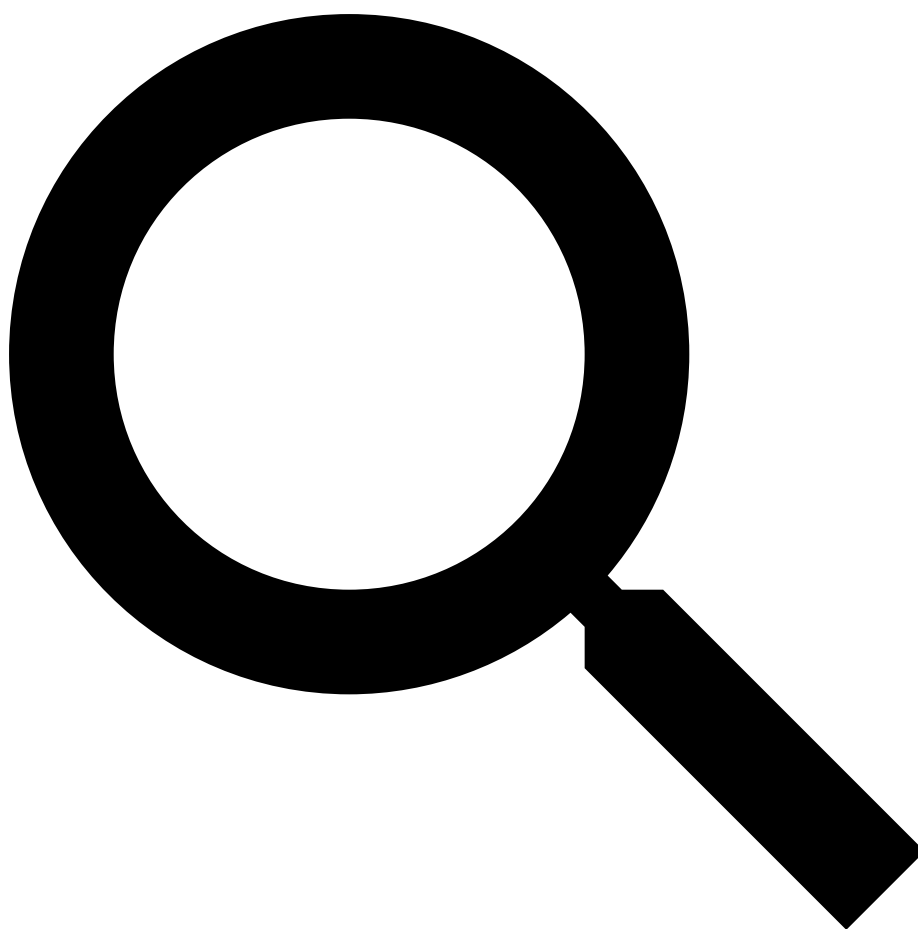
© 2025 VEGA TRAINING

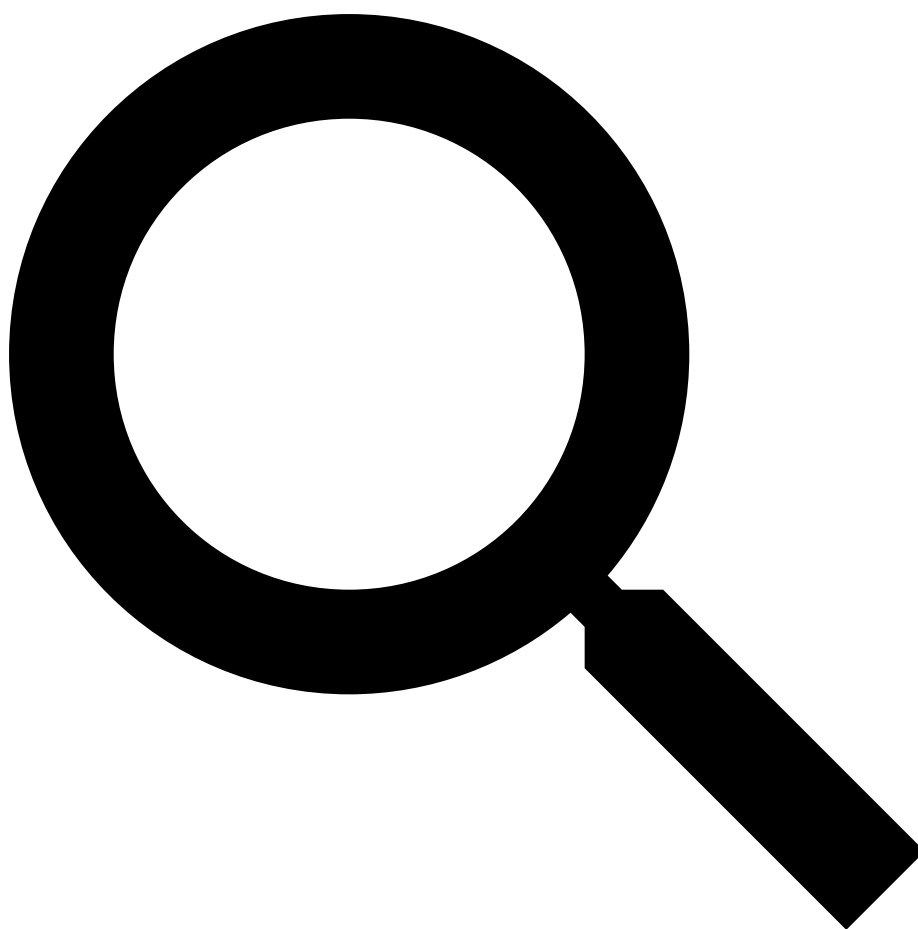


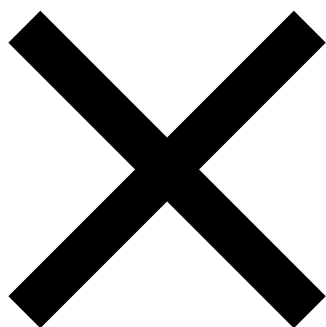
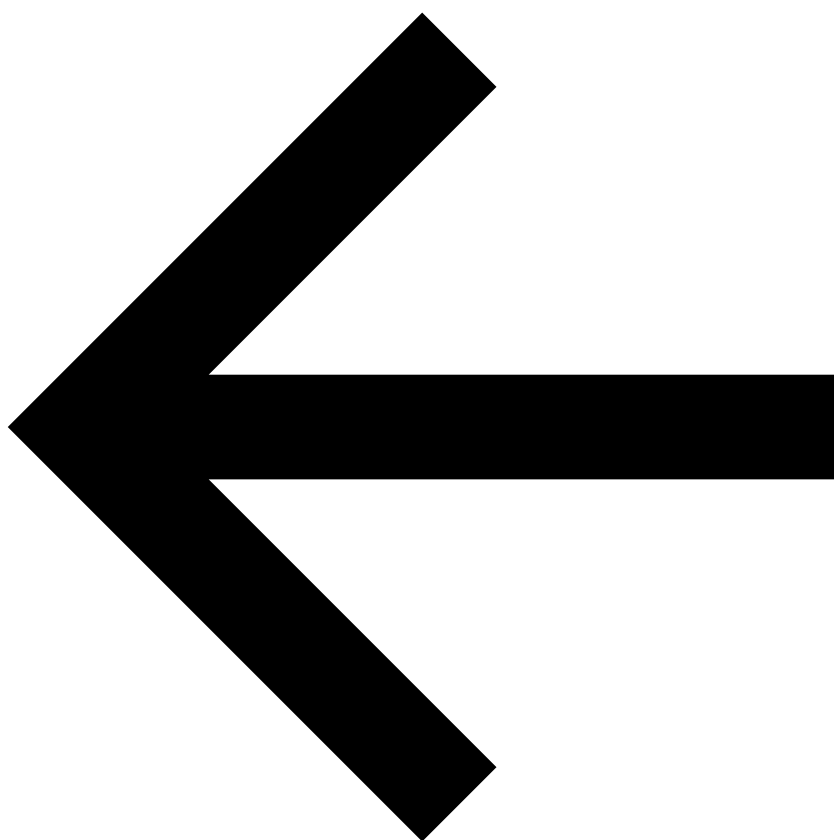
[Skip to content](#)





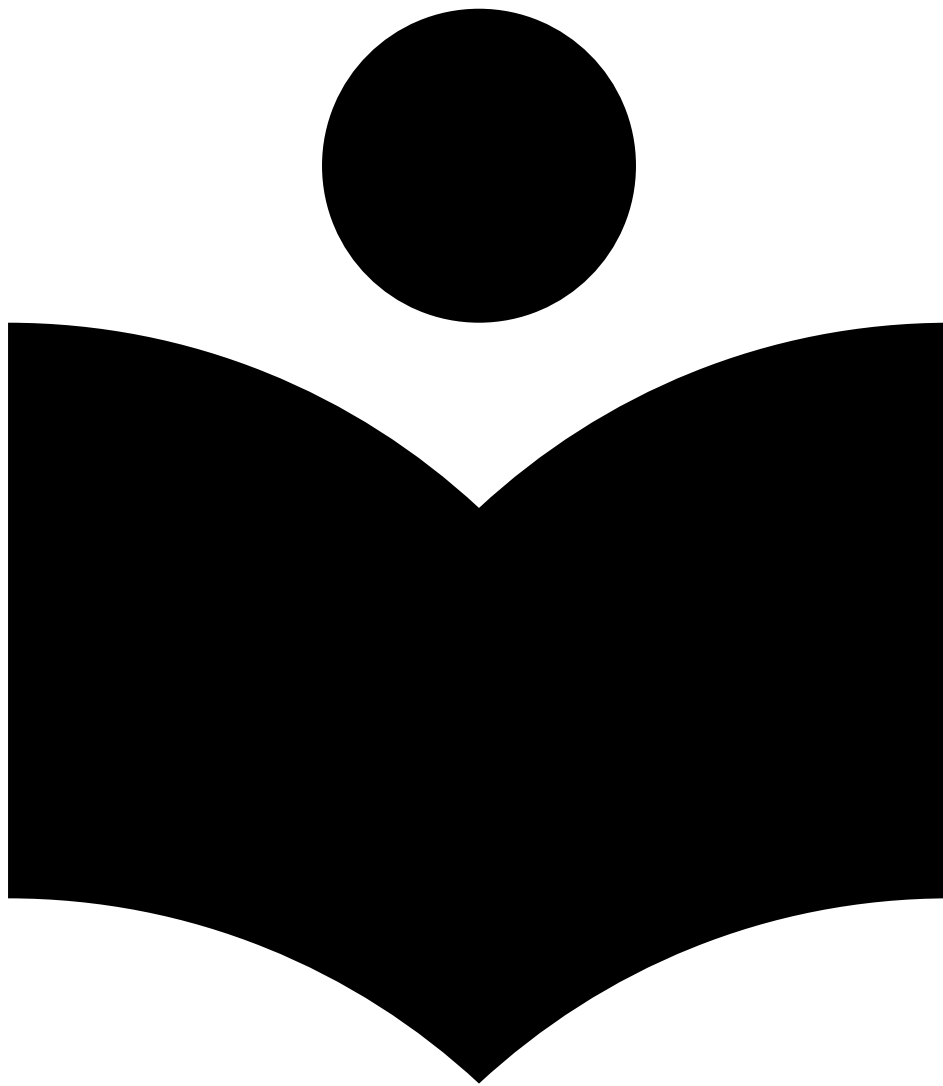






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
 - [Ch 03: Install Kubernetes](#)
 - [Ch 04: Architecture](#)
 - [Ch 05: API Access](#)
 - [Ch 06: API Objects](#)
 - [Ch 07: Deployments](#)
 - [Ch 08: Helm & Kustomize](#)
 - [Ch 09: Volumes](#)
 - ☐ Ch 10: Services [Ch 10: Services](#)
- Table of contents
- [Overview](#)
 - [Lab 10.1 — NodePort Service](#)
 - [Deploy the application](#)
 - [Expose as a NodePort Service](#)
 - [Inspect Endpoints](#)

- [Clean up](#)
- [Lab 10.2 — Multi-pod Service with Labels](#)
 - [Label-based traffic control](#)
 - [Clean up](#)
- [Lab 10.3 — CoreDNS Custom Domain Rewrite](#)
 - [Revert CoreDNS to the original config](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 10.1 — NodePort Service](#)
 - [Deploy the application](#)
 - [Expose as a NodePort Service](#)
 - [Inspect Endpoints](#)
 - [Clean up](#)
- [Lab 10.2 — Multi-pod Service with Labels](#)
 - [Label-based traffic control](#)
 - [Clean up](#)
- [Lab 10.3 — CoreDNS Custom Domain Rewrite](#)
 - [Revert CoreDNS to the original config](#)

Chapter 10: Services and Networking¶

Overview¶

In this lab you will:

- Create a NodePort Service and test pod-to-service connectivity
- Inspect endpoints and understand how kube-proxy routes traffic
- Configure CoreDNS with a rewrite rule for a custom domain
- Use labels to manage Service traffic

Lab files are in ~/lfs458/ch10-services/. The solutions/ subfolder contains completed manifests for files you need to edit.

Lab 10.1 — NodePort Service¶

Deploy the application¶

```
cp ~/lfs458/ch10-services/nginx-one.yaml ~/nginx-one.yaml
cat ~/nginx-one.yaml
```

Fix the containerPort

The manifest has containerPort: 8080 but nginx listens on **port 80**. Edit ~/nginx-one.yaml and change it to containerPort: 80 before applying. The solution is in ~/lfs458/ch10-services/solutions/nginx-one-port80.yaml.

```
kubectl apply -f ~/nginx-one.yaml
kubectl get pods -l app=nginx-one -w
```

Once running:

```
POD=$(kubectl get pod -l app=nginx-one -o jsonpath='{.items[0].metadata.name}')
kubectl describe pod $POD | grep -E "IP:|Port:|Node:"
```

Expose as a NodePort Service

```
kubectl expose pod $POD --name=nginx-one --port=80 --type=NodePort
```

Get the assigned port:

```
NODE_PORT=$(kubectl get svc nginx-one -o jsonpath='{.spec.ports[0].nodePort}')
echo "NodePort: $NODE_PORT"
```

Test from any node:

```
curl http://controller:$NODE_PORT
curl http://worker1:$NODE_PORT
curl http://worker2:$NODE_PORT
```

Inspect Endpoints

```
kubectl get endpoints nginx-one
```

```
ENDPOINT_IP=$(kubectl get endpoints nginx-one -o jsonpath='{.subsets[0].addresses[0].ip}')
echo "Pod endpoint IP: $ENDPOINT_IP"
curl http://$ENDPOINT_IP:80
```

Clean up

```
kubectl delete svc nginx-one
kubectl delete pod $POD
```

Lab 10.2 — Multi-pod Service with Labels

Create a Deployment and Service:

```
kubectl create deployment web --image=nginx --replicas=3
kubectl expose deployment web --port=80 --type=NodePort
```

```
NODE_PORT=$(kubectl get svc web -o jsonpath='{.spec.ports[0].nodePort}')
```

```
echo "NodePort: $NODE_PORT"
curl http://controller:$NODE_PORT
```

Watch endpoint changes as you scale:

```
kubectl get endpoints web -w &
EP_WATCH=$!
```

```
kubectl scale deployment web --replicas=5
sleep 5
```

```
kubectl scale deployment web --replicas=1
sleep 5
kill $EP_WATCH
```

Label-based traffic control

Add a second label to a specific pod and show how label selectors route traffic:

```
WEB_POD=$(kubectl get pod -l app=web -o jsonpath='{.items[0].metadata.name}')
```

```
# Add an extra label
kubectl label pod $WEB_POD tier=canary
```

```
# Create a separate service selecting only the canary pod
kubectl expose pod $WEB_POD --name=web-canary --port=80 --type=NodePort \
  --selector="app=web,tier=canary"
```

```
CANARY_PORT=$(kubectl get svc web-canary -o jsonpath='{.spec.ports[0].nodePort}')
echo "Canary NodePort: $CANARY_PORT"
curl http://controller:$CANARY_PORT
```

Clean up

```
kubectl delete deployment web
kubectl delete svc web web-canary
```

Lab 10.3 — CoreDNS Custom Domain Rewrite

This exercise adds a CoreDNS rewrite rule so that *.test.io resolves to your cluster services via a wildcard.

Inspect the current CoreDNS ConfigMap:

```
kubectl get configmap coredns -n kube-system -o yaml
```

Patch the Corefile to add a rewrite rule:

```
kubectl get configmap coredns -n kube-system -o jsonpath='{.data.Corefile}'
cat /tmp/Corefile.orig
```

Add the rewrite rule before the forward block:

```
cat <<'EOF' > /tmp/Corefile.new
.:53 {
    errors
    health {
        lameduck 5s
    }
    ready
    rewrite name suffix .test.io .default.svc.cluster.local answer auto
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
    }
    prometheus :9153
    forward . /etc/resolv.conf {
        max_concurrent 1000
    }
    cache 30
    loop
    reload
    loadbalance
}
EOF
```

```
kubectl create configmap coredns -n kube-system \
  --from-file=Corefile=/tmp/Corefile.new \
  --dry-run=client -o yaml | kubectl apply -f -
```

Force CoreDNS pods to reload by deleting them (they will be recreated by the Deployment):

```
COREDNS_POD=$(kubectl get pod -n kube-system -l k8s-app=kube-dns \
  -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $COREDNS_POD -n kube-system
```

```
# Wait for the new pod
kubectl rollout status deployment coredns -n kube-system
```

Test the rewrite from inside a pod:

```
kubectl run dnstest --image=busybox --rm -it --restart=Never -- \
  nslookup kubernetes.test.io
```

This should resolve to the kubernetes ClusterIP service.

Revert CoreDNS to the original config

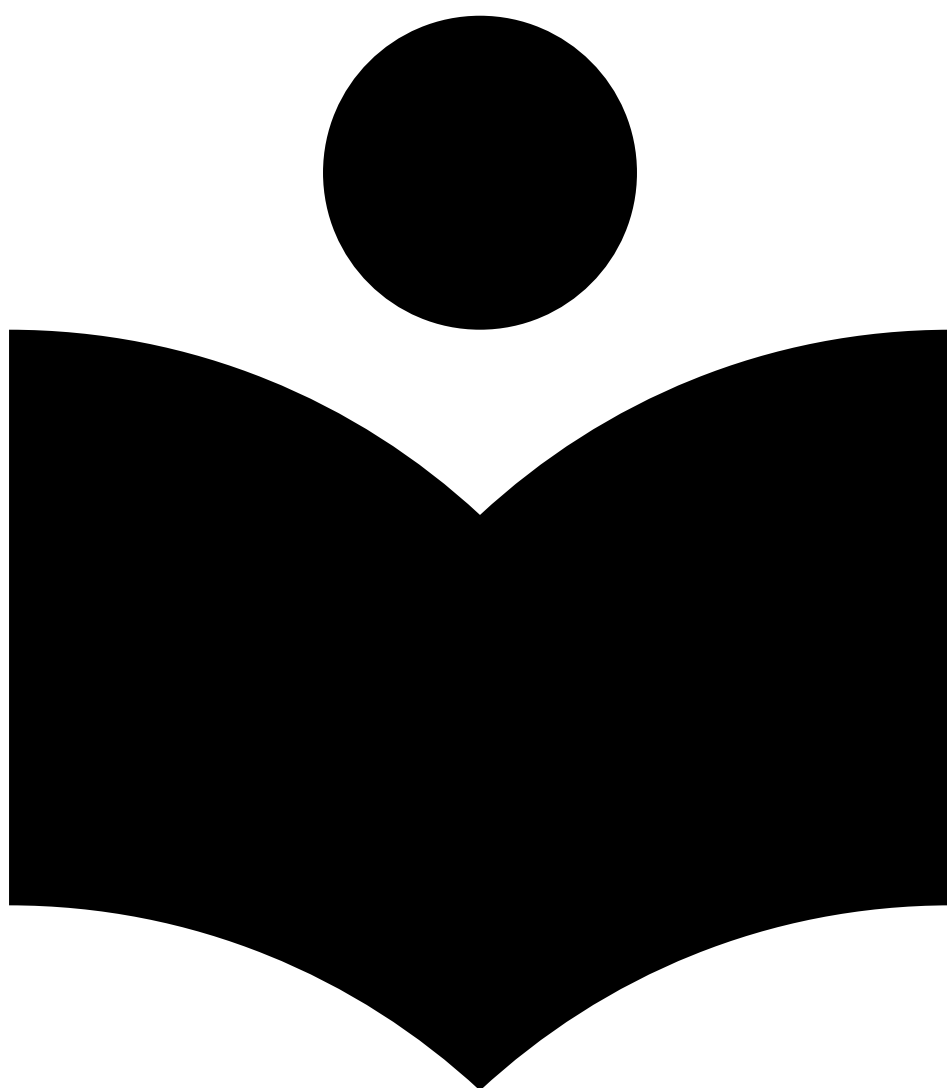
```
kubectl create configmap coredns -n kube-system \
  --from-file=Corefile=/tmp/Corefile.orig \
  --dry-run=client -o yaml | kubectl apply -f -

COREDNS_POD=$(kubectl get pod -n kube-system -l k8s-app=kube-dns \
  -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $COREDNS_POD -n kube-system
kubectl rollout status deployment coredns -n kube-system
```

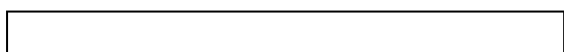
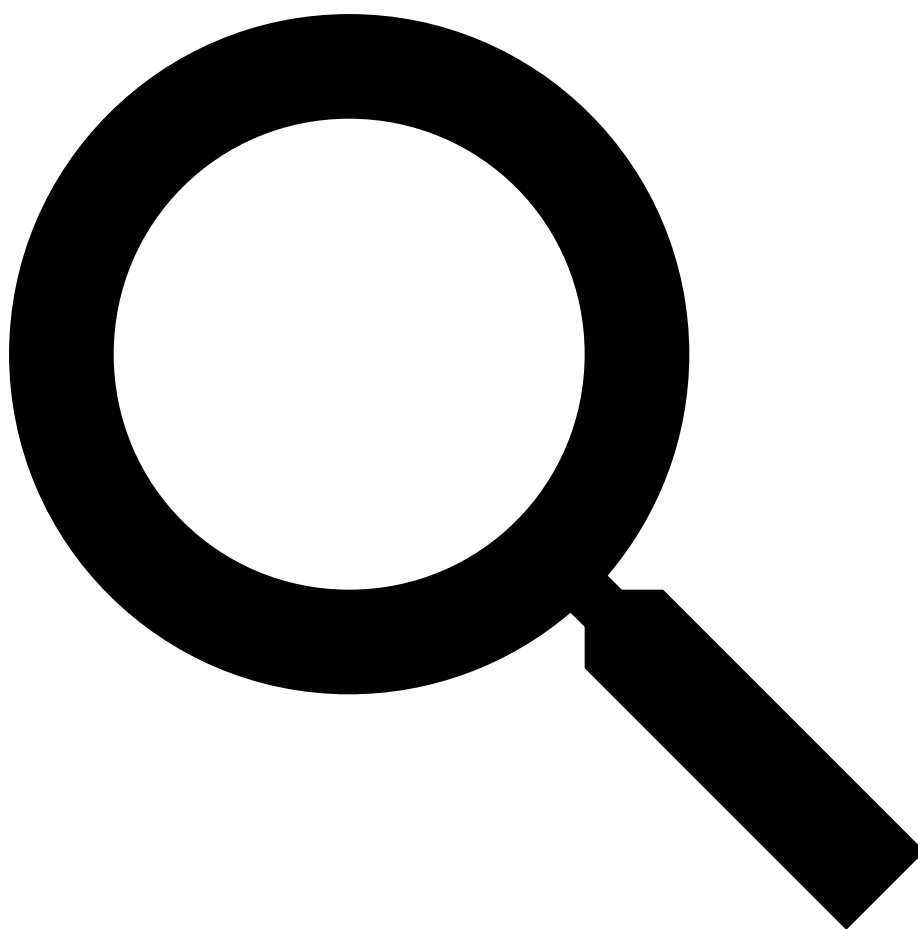
© 2025 VEGA TRAINING

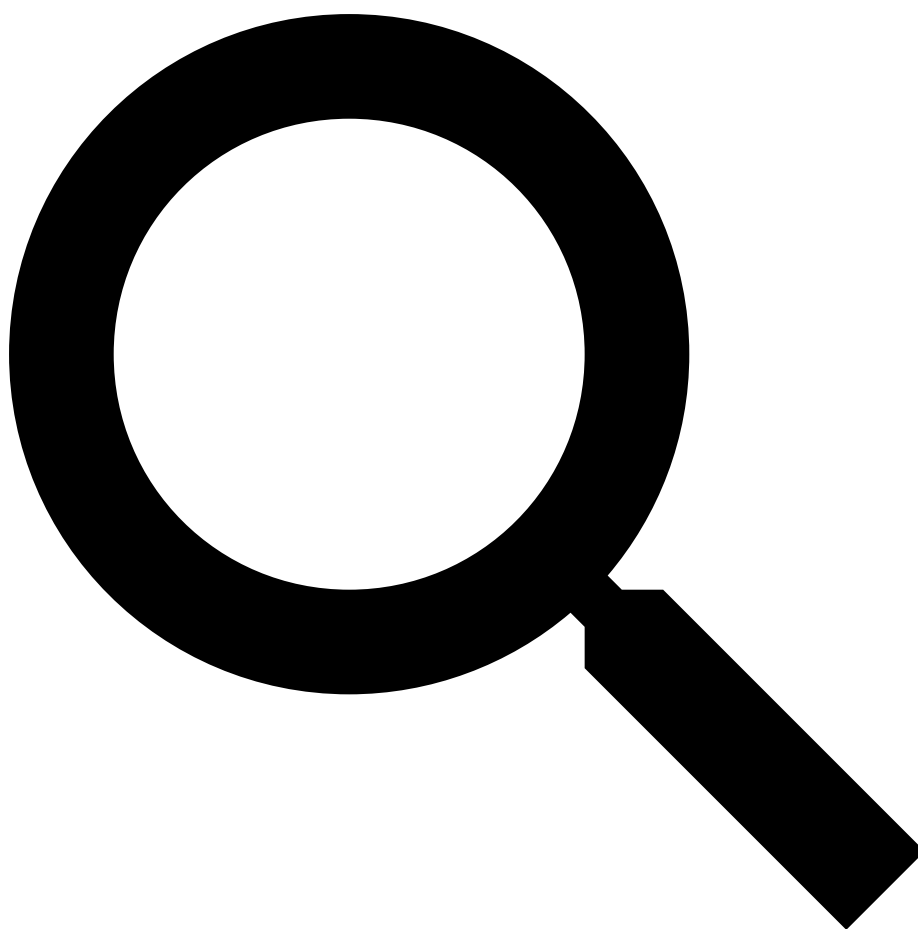


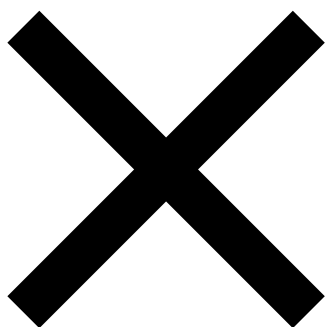
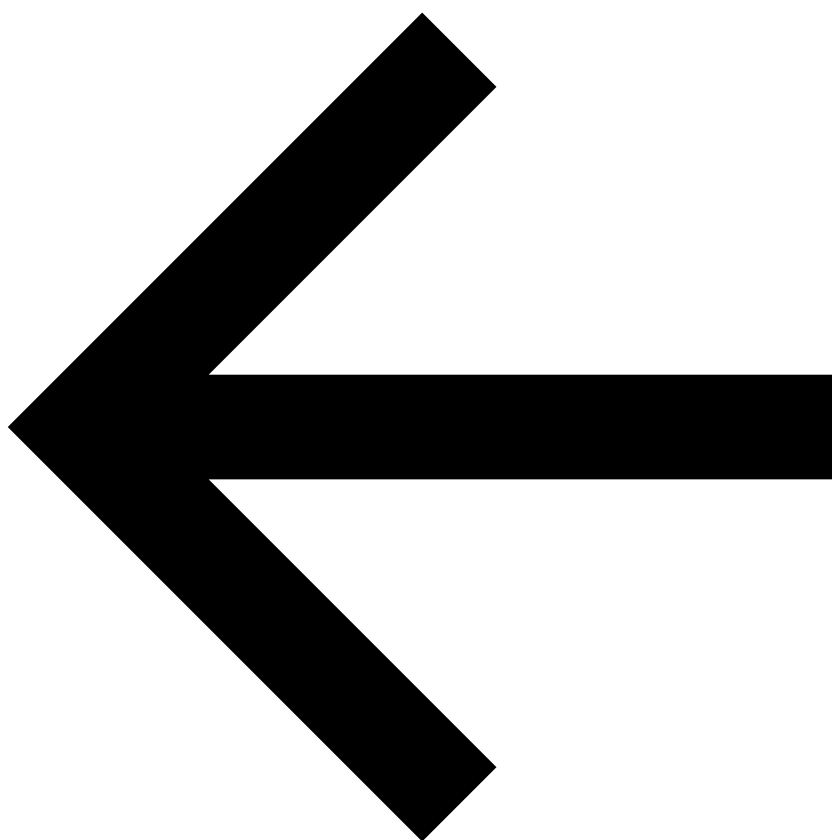
[Skip to content](#)





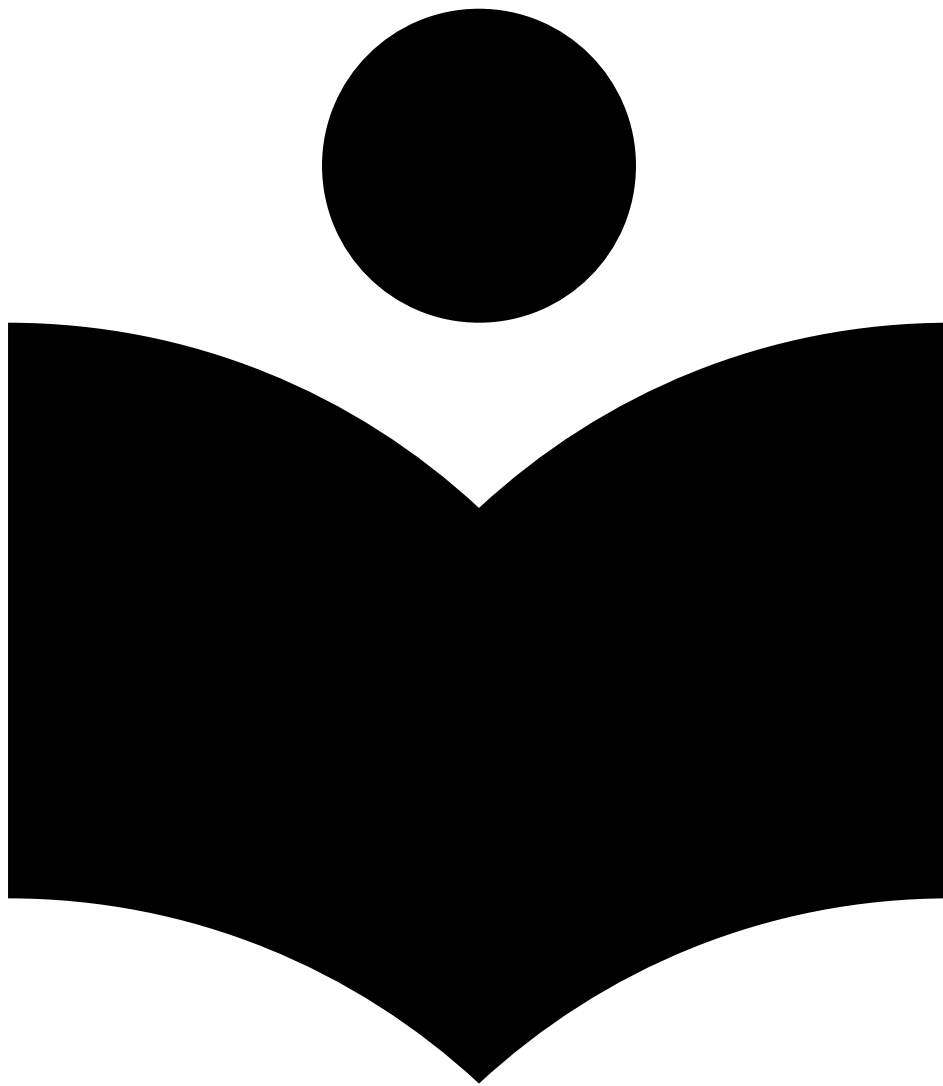






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- ☐ Ch 11: Ingress [Ch 11: Ingress](#)
 - Table of contents
 - [Overview](#)
 - [Lab 11.1 — Install NGINX Ingress Controller](#)
 - [Lab 11.2 — Create Ingress Rules](#)
 - [Deploy two backend applications](#)

- [Create an Ingress resource](#)
 - [Test the Ingress](#)
 - [Clean up](#)
- [Lab 11.3 — Gateway API](#)
 - [Install Gateway API CRDs](#)
 - [Install NGINX Gateway Fabric](#)
 - [Create a GatewayClass and Gateway](#)
 - [Create a backend and HTTPRoute](#)
 - [Clean up](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 11.1 — Install NGINX Ingress Controller](#)
- [Lab 11.2 — Create Ingress Rules](#)
 - [Deploy two backend applications](#)
 - [Create an Ingress resource](#)
 - [Test the Ingress](#)
 - [Clean up](#)
- [Lab 11.3 — Gateway API](#)
 - [Install Gateway API CRDs](#)
 - [Install NGINX Gateway Fabric](#)
 - [Create a GatewayClass and Gateway](#)
 - [Create a backend and HTTPRoute](#)
 - [Clean up](#)

Chapter 11: Ingress and Gateway API

Overview

In this lab you will:

- Install the NGINX Ingress Controller as a DaemonSet
- Create Ingress rules to route traffic to multiple backends
- Explore the Gateway API as the next-generation Ingress alternative

Lab 11.1 — Install NGINX Ingress Controller

Deploy NGINX Ingress as a DaemonSet so it runs on every node:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx
```

Wait for the controller to be ready:

```
kubectl rollout status deployment ingress-nginx-controller -n ingress-nginx
kubectl get pods -n ingress-nginx
```

Get the NodePort assigned to the ingress controller:

```
HTTP_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
-o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
HTTPS_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
-o jsonpath='{.spec.ports[?(@.name=="https")].nodePort}')
```

```
echo "HTTP NodePort: $HTTP_PORT"
echo "HTTPS NodePort: $HTTPS_PORT"
```

Lab 11.2 — Create Ingress Rules

Deploy two backend applications

```
kubectl create deployment app1 --image=nginx --replicas=2
kubectl expose deployment app1 --port=80
```

```
kubectl create deployment app2 --image=httpd --replicas=2
kubectl expose deployment app2 --port=80
```

Create an Ingress resource

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: apps-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app1.vega.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1
            port:
              number: 80
  - host: app2.vega.local
    http:
      paths:
```

```
- path: /
  pathType: Prefix
  backend:
    service:
      name: app2
      port:
        number: 80
```

EOF

Test the Ingress

Add entries to /etc/hosts on the controller (or wherever you're running curl):

```
CONTROLLER_IP=$(kubectl get node controller -o jsonpath='{.status.addresses[0].address}' | sed 's/10.0.0.1/10.0.0.2/')
echo "$CONTROLLER_IP app1.vega.local app2.vega.local" | sudo tee -a /etc/hosts
```

Test:

```
HTTP_PORT=$(kubectl get svc ingress-nginx-controller -n ingress-nginx \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')

```

```
curl http://app1.vega.local:$HTTP_PORT
curl http://app2.vega.local:$HTTP_PORT
```

app1 should return the nginx welcome page; app2 should return the Apache (httpd) page.

Clean up

```
kubectl delete ingress apps-ingress
kubectl delete deployment app1 app2
kubectl delete svc app1 app2
```

Lab 11.3 — Gateway API

The Gateway API is the successor to Ingress. It separates infrastructure configuration (Gateway) from routing rules (HTTPRoute).

Install Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/gateway-api.yaml
kubectl get crd | grep gateway
```

Install NGINX Gateway Fabric

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/main/deployments/kubernetes/nginx-gateway-fabric.yaml
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/main/deployments/kubernetes/nginx-gateway-fabric-nginx.yaml
kubectl rollout status deployment ngf-nginx-gateway-fabric -n nginx-gateway-fabric
```

Get the Gateway ports:

```
GW_HTTP=$(kubectl get svc ngf-nginx-gateway-fabric -n nginx-gateway \
-o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
echo "Gateway HTTP NodePort: $GW_HTTP"
```

Create a GatewayClass and Gateway

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
  namespace: default
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP
EOF
```

Create a backend and HTTPRoute

```
kubectl create deployment gw-app --image=nginx
kubectl expose deployment gw-app --port=80
```

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: gw-app-route
spec:
  parentRefs:
  - name: main-gateway
  hostnames:
  - "gw-app.vega.local"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: gw-app
```

port: 80
EOF

Test:

```
echo "$CONTROLLER_IP gw-app.vega.local" | sudo tee -a /etc/hosts  
curl http://gw-app.vega.local:$GW_HTTP
```

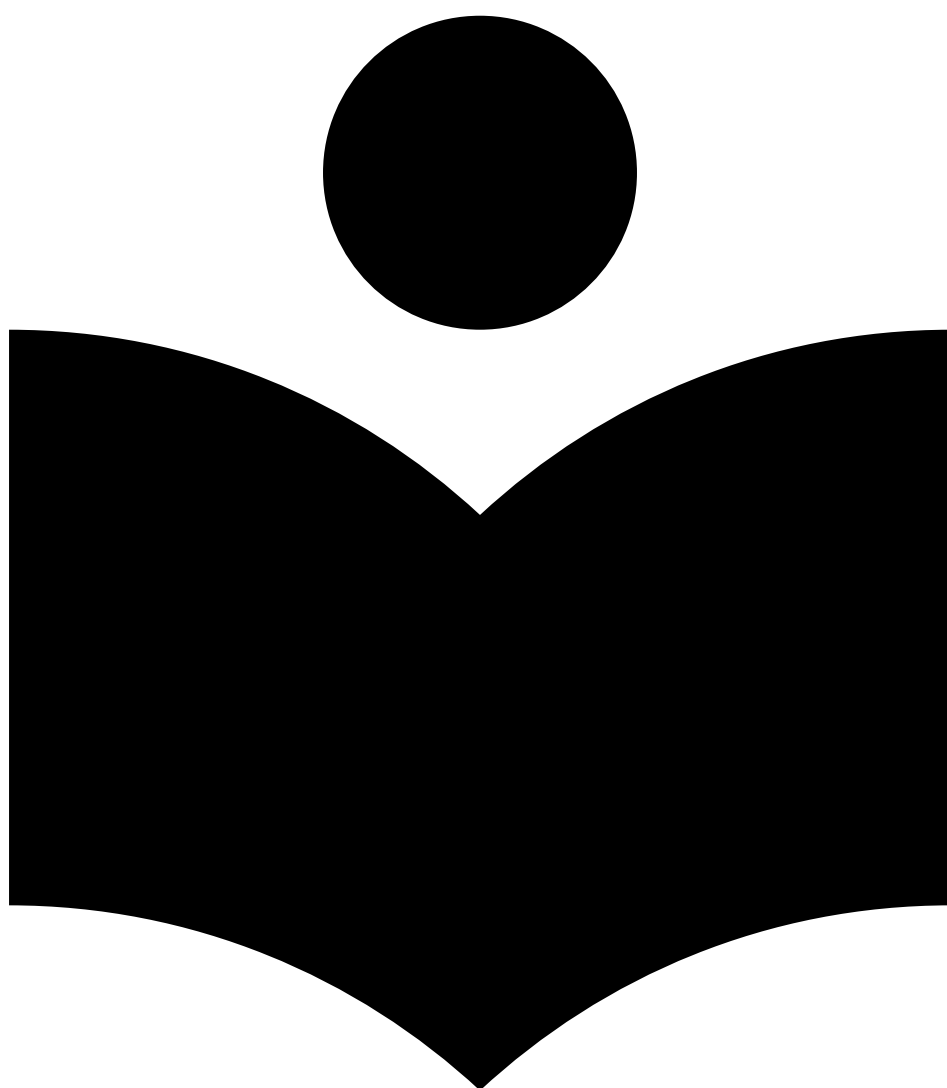
Clean up

```
kubectl delete httproute gw-app-route  
kubectl delete gateway main-gateway  
kubectl delete gatewayclass nginx  
kubectl delete deployment gw-app  
kubectl delete svc gw-app
```

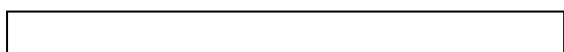
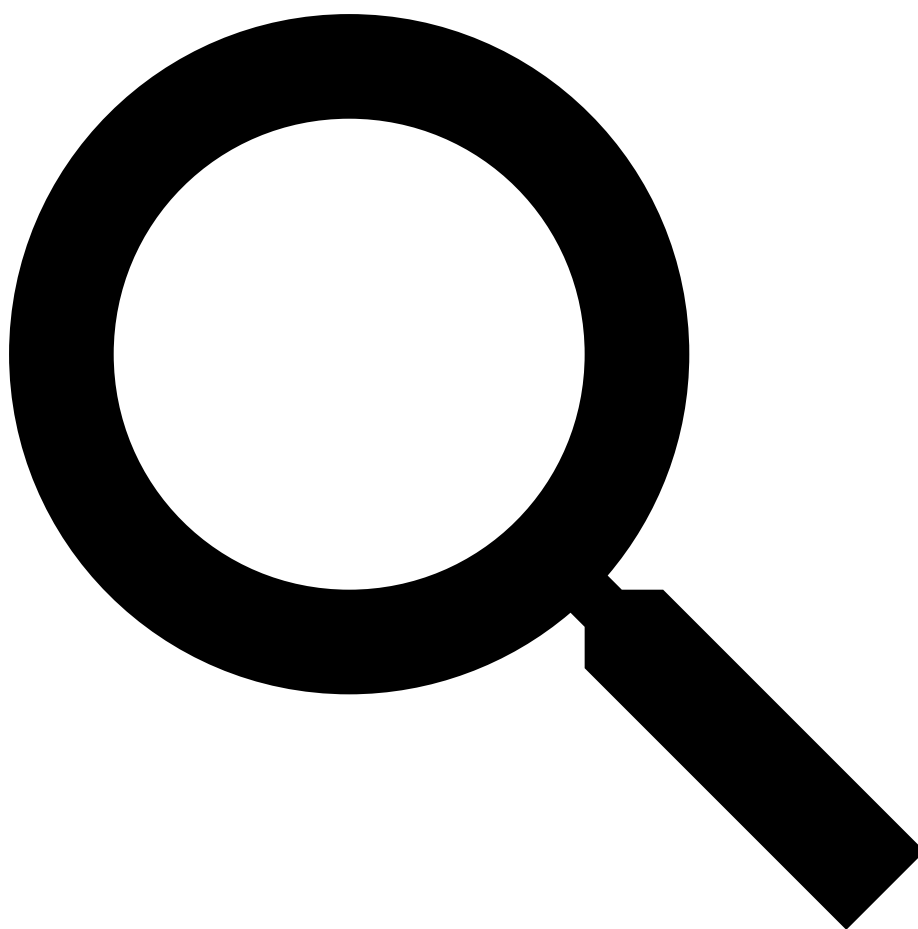
© 2025 VEGA TRAINING

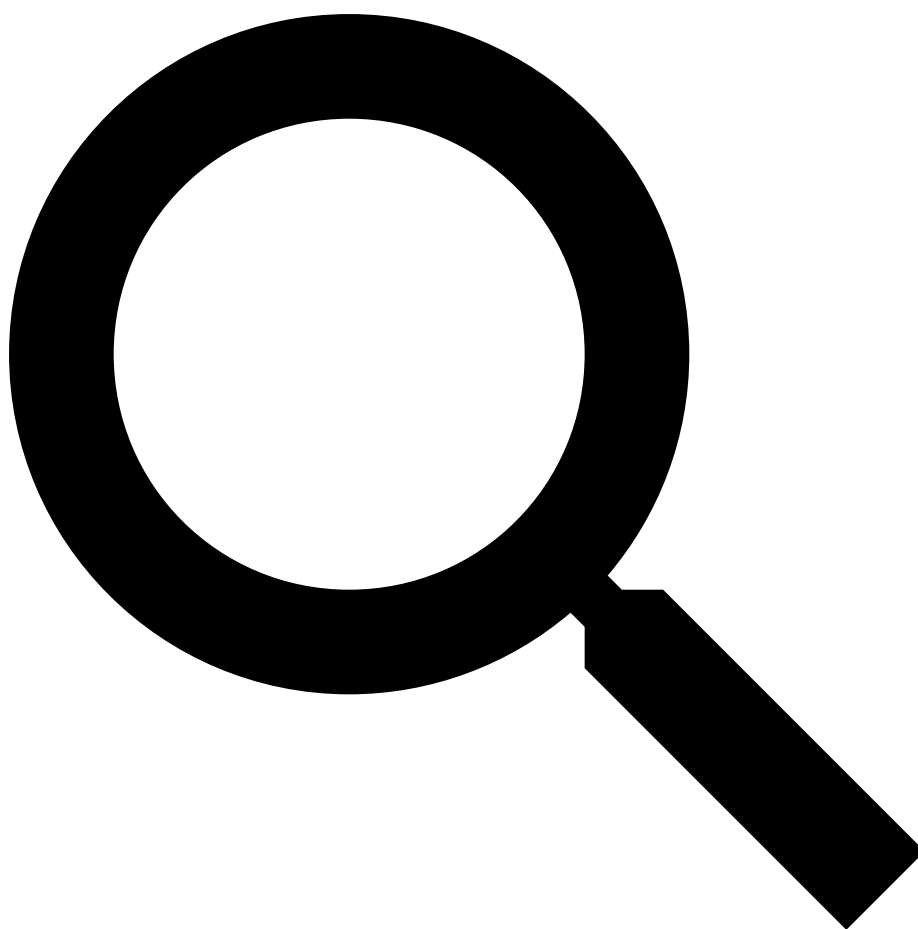


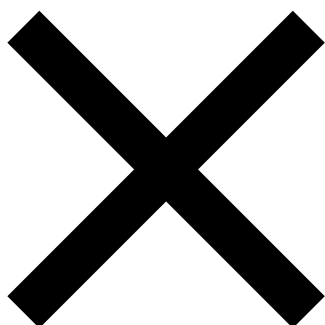
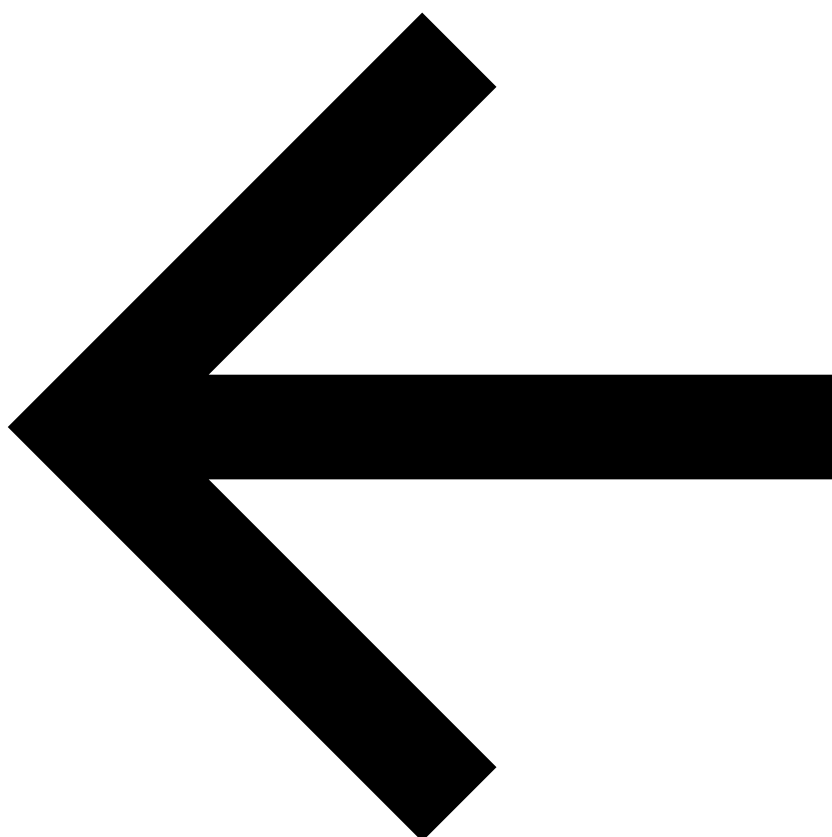
[Skip to content](#)





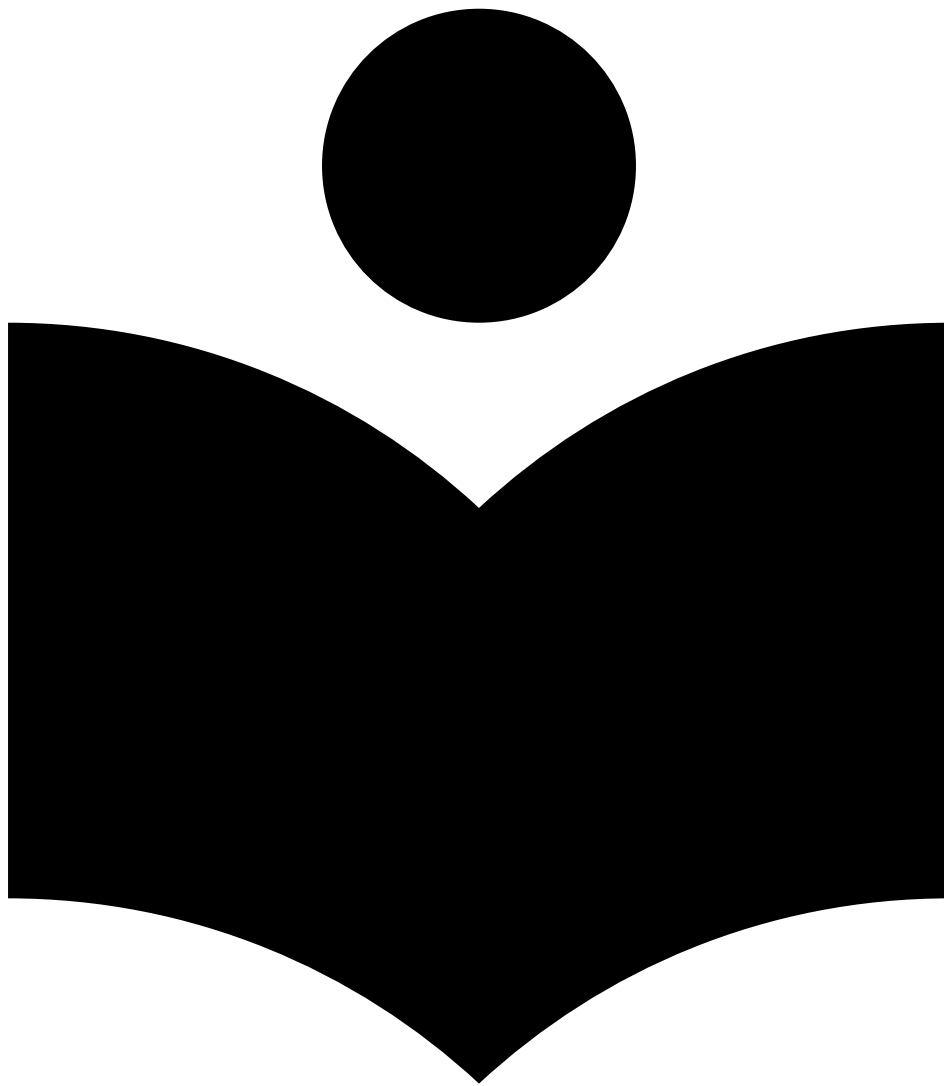






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- ☐ [Ch 12: Scheduling](#) [Ch 12: Scheduling](#)
Table of contents
 - [Overview](#)
 - [Lab 12.1 — nodeSelector](#)
 - [Lab 12.2 — Node Affinity](#)

- [Lab 12.3 — Taints and Tolerations](#)
 - [PreferNoSchedule](#)
 - [NoSchedule](#)
 - [NoExecute](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 12.1 — nodeSelector](#)
- [Lab 12.2 — Node Affinity](#)
- [Lab 12.3 — Taints and Tolerations](#)
 - [PreferNoSchedule](#)
 - [NoSchedule](#)
 - [NoExecute](#)

Chapter 12: Scheduling¶

Overview¶

In this lab you will:

- Use nodeSelector to pin pods to specific nodes
- Work with node labels
- Apply Taints and Tolerations (PreferNoSchedule, NoSchedule, NoExecute)
- Understand how the scheduler makes placement decisions

Lab 12.1 — nodeSelector¶

Label a node:

```
kubectl label node worker1 disktype=ssd
kubectl get nodes --show-labels | grep disktype
```

Create a pod that requires disktype=ssd:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disktype: ssd
  containers:
```

```
- name: nginx
  image: nginx
EOF
```

Verify it landed on worker1:

```
kubectl get pod ssd-pod -o wide
```

Delete and remove the label:

```
kubectl delete pod ssd-pod
kubectl label node worker1 disktype-
```

Lab 12.2 — Node Affinity¹

Node Affinity is a more expressive replacement for nodeSelector.

Label nodes:

```
kubectl label node worker1 zone=west
kubectl label node worker2 zone=east
```

Create a pod with preferredDuringSchedulingIgnoredDuringExecution (soft requirement):

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: affinity-pod
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
              - key: zone
                operator: In
                values:
                  - west
  containers:
    - name: nginx
      image: nginx
EOF
```

```
kubectl get pod affinity-pod -o wide
```

Clean up:


```
kubectl delete pod affinity-pod
kubectl label node worker1 zone-
kubectl label node worker2 zone-
```

Lab 12.3 — Taints and Tolerations¶

PreferNoSchedule¶

Apply a PreferNoSchedule taint to worker2 — the scheduler will try to avoid it but will use it if necessary:

```
kubectl taint node worker2 gpu=true:PreferNoSchedule
kubectl describe node worker2 | grep -A3 Taints
```

Deploy many replicas and observe where they land:

```
kubectl create deployment no-pref --image=nginx --replicas=6
kubectl get pods -l app=no-pref -o wide
```

Most pods should land on controller and worker1; some may land on worker2.

Remove the taint:

```
kubectl taint node worker2 gpu=true:PreferNoSchedule-
kubectl delete deployment no-pref
```

NoSchedule¶

NoSchedule is a hard rule — no new pods will be scheduled on the tainted node unless they tolerate the taint.

```
kubectl taint node worker2 gpu=true:NoSchedule
```

Deploy without a toleration — pods should NOT land on worker2:

```
kubectl create deployment no-sched --image=nginx --replicas=4
kubectl get pods -l app=no-sched -o wide
```

Now deploy with a toleration:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tolerates-gpu
spec:
  replicas: 4
  selector:
    matchLabels:
      app: tolerates-gpu
```

```
template:
  metadata:
    labels:
      app: tolerates-gpu
  spec:
    tolerations:
      - key: gpu
        operator: Equal
        value: "true"
        effect: NoSchedule
    containers:
      - name: nginx
        image: nginx
```

EOF

```
kubectl get pods -l app=tolerates-gpu -o wide
```

These pods can land on worker2.

Remove the taint and clean up:

```
kubectl taint node worker2 gpu=true:NoSchedule-
kubectl delete deployment no-sched tolerates-gpu
```

NoExecute

NoExecute evicts already-running pods that lack the toleration.

```
kubectl create deployment evict-test --image=nginx --replicas=4
kubectl get pods -l app=evict-test -o wide
```

Note some pods on worker2. Now taint it with NoExecute:

```
kubectl taint node worker2 maintenance=true:NoExecute
kubectl get pods -l app=evict-test -o wide -w
```

Pods on worker2 will be evicted immediately and rescheduled on the other nodes.

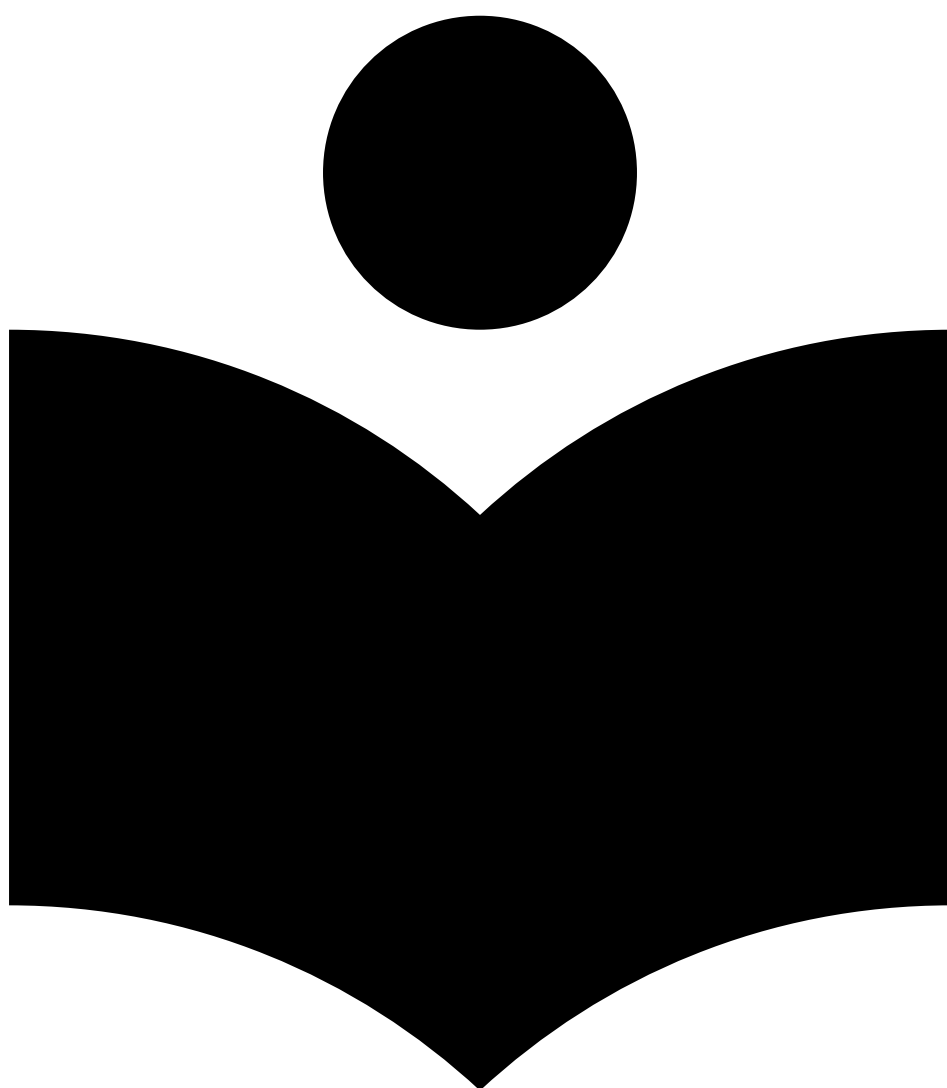
Remove the taint and clean up:

```
kubectl taint node worker2 maintenance=true:NoExecute-
kubectl delete deployment evict-test
```

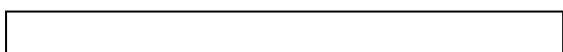
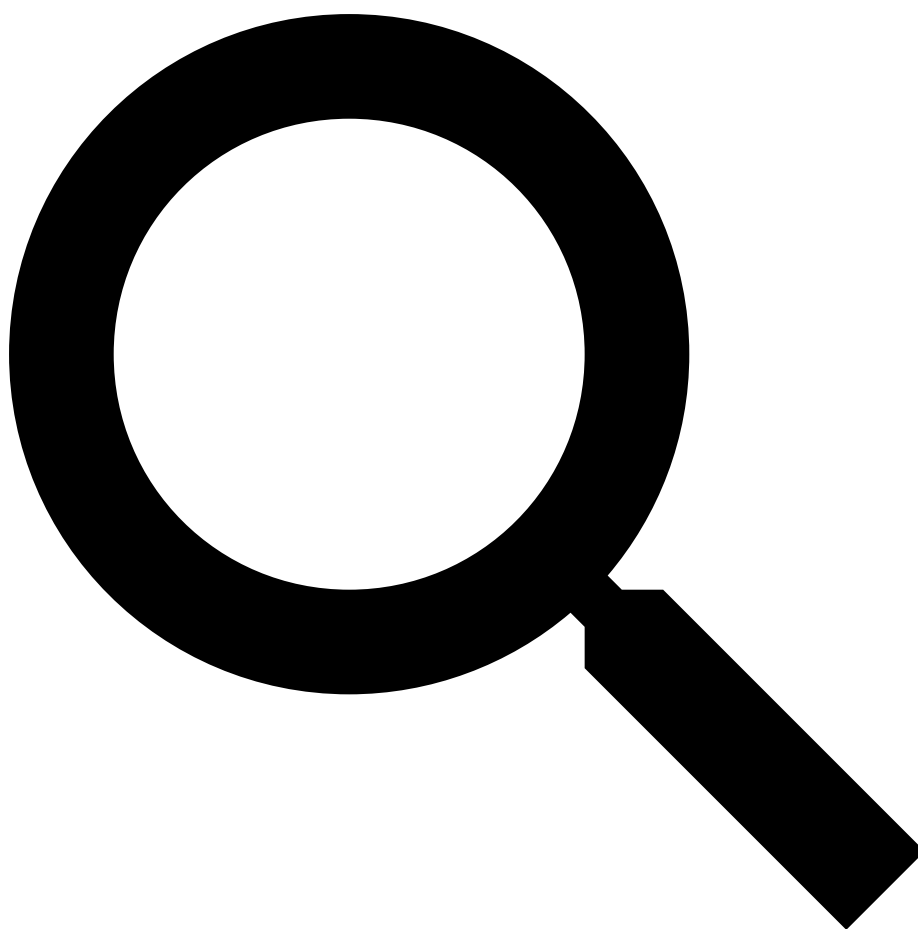
© 2025 VEGA TRAINING

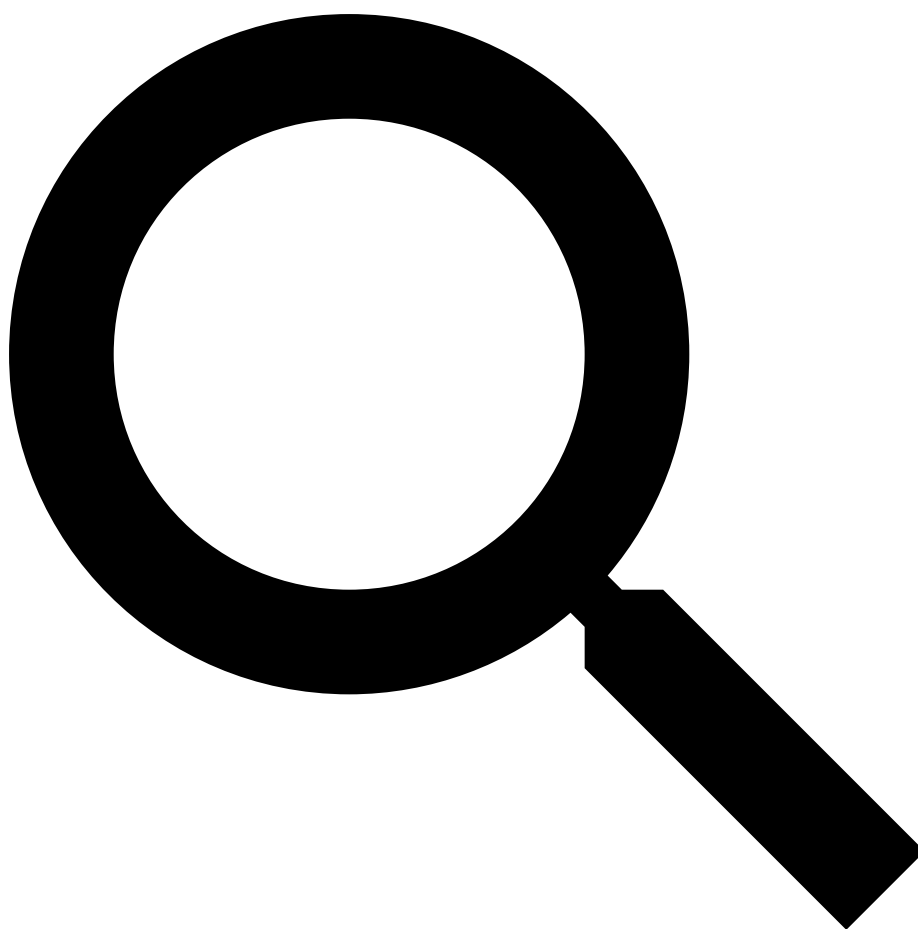


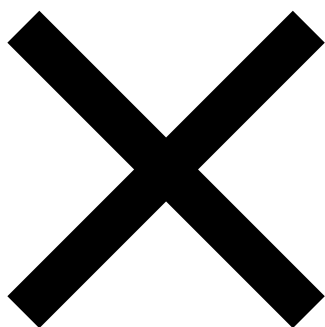
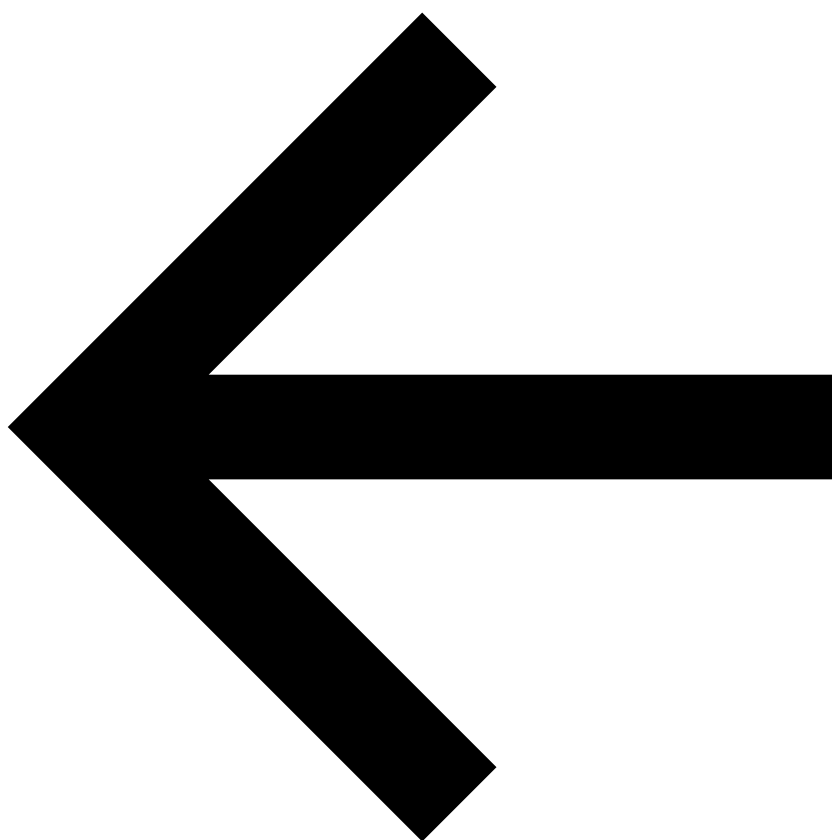
[Skip to content](#)





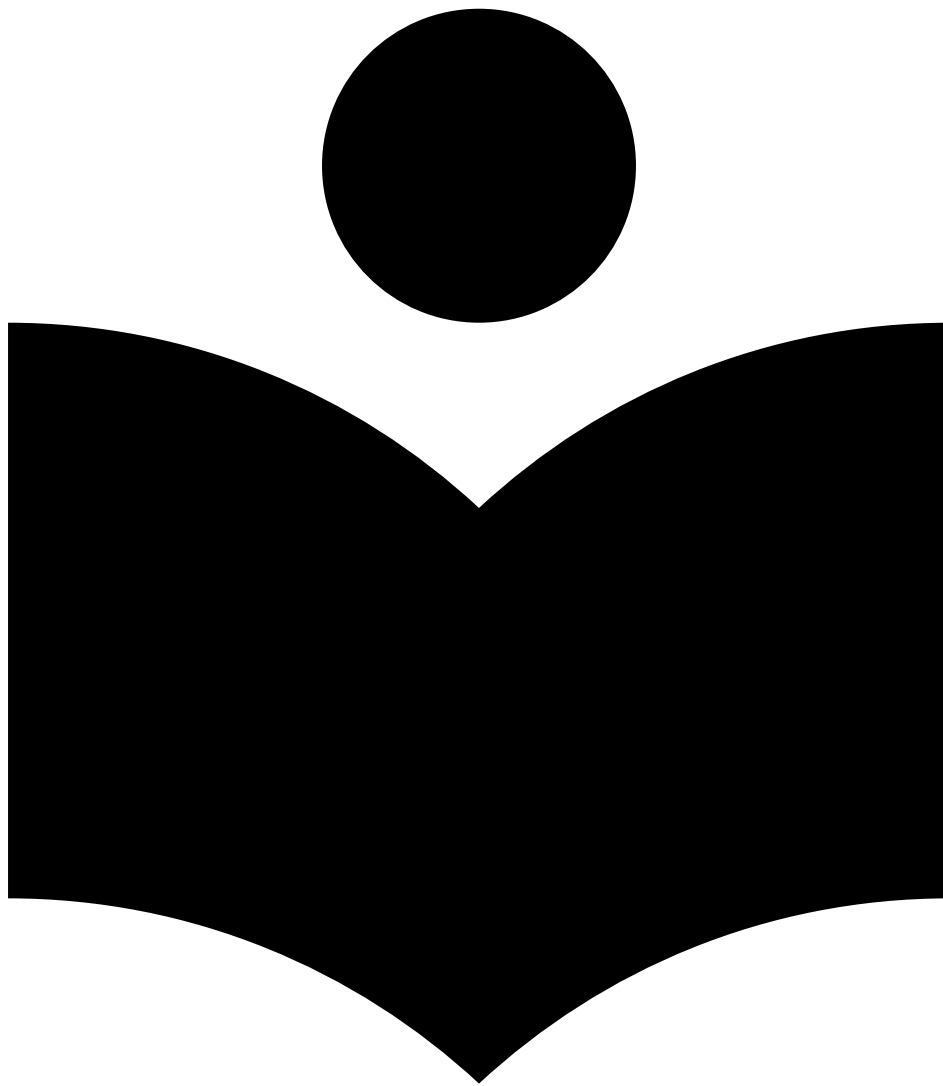






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- ☐ Ch 13: Troubleshooting [Ch 13: Troubleshooting](#)
- Table of contents
 - [Overview](#)

- [Lab 13.1 — Component Logs](#)
 - [API Server logs](#)
 - [Scheduler logs](#)
 - [Controller Manager logs](#)
 - [etcd logs](#)
 - [kubelet logs \(host systemd unit\)](#)
 - [Locate log files](#)
- [Lab 13.2 — kubectl top](#)
- [Lab 13.3 — Troubleshooting a Broken Pod](#)
- [Lab 13.4 — Kubernetes Dashboard](#)
 - [Clean up](#)
- [Lab 13.5 — Common Troubleshooting Patterns](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 13.1 — Component Logs](#)
 - [API Server logs](#)
 - [Scheduler logs](#)
 - [Controller Manager logs](#)
 - [etcd logs](#)
 - [kubelet logs \(host systemd unit\)](#)
 - [Locate log files](#)
- [Lab 13.2 — kubectl top](#)
- [Lab 13.3 — Troubleshooting a Broken Pod](#)
- [Lab 13.4 — Kubernetes Dashboard](#)
 - [Clean up](#)
- [Lab 13.5 — Common Troubleshooting Patterns](#)

Chapter 13: Troubleshooting¶

Overview¶

In this lab you will:

- Locate and read Kubernetes component logs
- Use `kubectl top` and `metrics-server`
- Deploy and access the Kubernetes Dashboard
- Practice common troubleshooting patterns

Lab 13.1 — Component Logs¶

Kubernetes control plane components run as static pods (manifests in `/etc/kubernetes/manifests/`). Their logs are accessible via `kubectl` or `journalctl`.

API Server logs

```
API_POD=$(kubectl get pod -n kube-system -l component=kube-apiserver \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs $API_POD -n kube-system | tail -20
```

Scheduler logs

```
SCHED_POD=$(kubectl get pod -n kube-system -l component=kube-scheduler \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs $SCHED_POD -n kube-system | tail -20
```

Controller Manager logs

```
CM_POD=$(kubectl get pod -n kube-system -l component=kube-controller-manag
-o jsonpath='{.items[0].metadata.name}')
kubectl logs $CM_POD -n kube-system | tail -20
```

etcd logs

```
ETCD_POD=$(kubectl get pod -n kube-system -l component=etcd \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs $ETCD_POD -n kube-system | tail -20
```

kubelet logs (host systemd unit)

On each node, kubelet runs as a systemd service (not a pod):

```
sudo journalctl -u kubelet --since "10 minutes ago" | tail -30
```

Locate log files

Control plane pod logs are also written to disk:

```
sudo ls /var/log/pods/ | grep kube-system
```

Lab 13.2 — kubectl top

metrics-server must be installed (covered in Ch08). If it's running:

```
kubectl top nodes
kubectl top pods -A --sort-by=cpu | head -20
```

Watch a specific namespace:

```
kubectl top pods -n kube-system --sort-by=memory
```

Lab 13.3 — Troubleshooting a Broken Pod

Create a pod with an intentional error:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: broken-pod
spec:
  containers:
  - name: app
    image: nginx:doesnotexist
EOF
```

Observe the failure:

```
kubectl get pod broken-pod
kubectl describe pod broken-pod | grep -A10 Events
```

Look at the image pull error in the Events section. Fix it:

```
kubectl patch pod broken-pod -p '{"spec":{"containers":[{"name":"app","ima
# Patching running pods has limitations; for a real fix, delete and recrea
kubectl delete pod broken-pod
```

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: broken-pod
spec:
  containers:
  - name: app
    image: nginx:latest
EOF
```

```
kubectl get pod broken-pod -w
```

Clean up:

```
kubectl delete pod broken-pod
```

Lab 13.4 — Kubernetes Dashboard

Deploy the Dashboard:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2
kubectl rollout status deployment/kubernetes-dashboard -n kubernetes-dashb
```

Create a ServiceAccount and ClusterRoleBinding to access the dashboard:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dashboard-admin
  namespace: kubernetes-dashboard
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: dashboard-admin-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: dashboard-admin
  namespace: kubernetes-dashboard
EOF
```

Get the login token:

```
kubectl create token dashboard-admin -n kubernetes-dashboard
```

Access the dashboard via kubectl proxy:

```
kubectl proxy --address=0.0.0.0 --accept-hosts='.*' &
echo "Dashboard URL: http://localhost:8001/api/v1/namespaces/kubernetes-da
```

Open the URL in a browser and log in with the token.

Clean up

```
kubectl delete -f https://raw.githubusercontent.com/kubernetes/dashboard/v
kubectl delete clusterrolebinding dashboard-admin-binding
kubectl delete serviceaccount dashboard-admin -n kubernetes-dashboard 2>/d
```

Lab 13.5 — Common Troubleshooting Patterns

Symptom

Pod stuck in Pending

Pod in
CrashLoopBackOff

Pod in
ImagePullBackOff

Node NotReady

Where to look

kubectl describe pod → Events; check resource requests, taints, node capacity

kubectl logs <pod> and kubectl logs <pod> --previous

kubectl describe pod → check image name/tag and registry credentials

Symptom**Where to look**

	<code>sudo journalctl -u kubelet</code> on the node; check CNI health
Service not routing	<code>kubectl get endpoints <svc></code> — if empty, label selector doesn't match any pods
DNS not resolving	<code>kubectl exec <pod> -- nslookup</code> kubernetes.default; check CoreDNS pods

Quick cluster health overview

```
kubectl get nodes
```

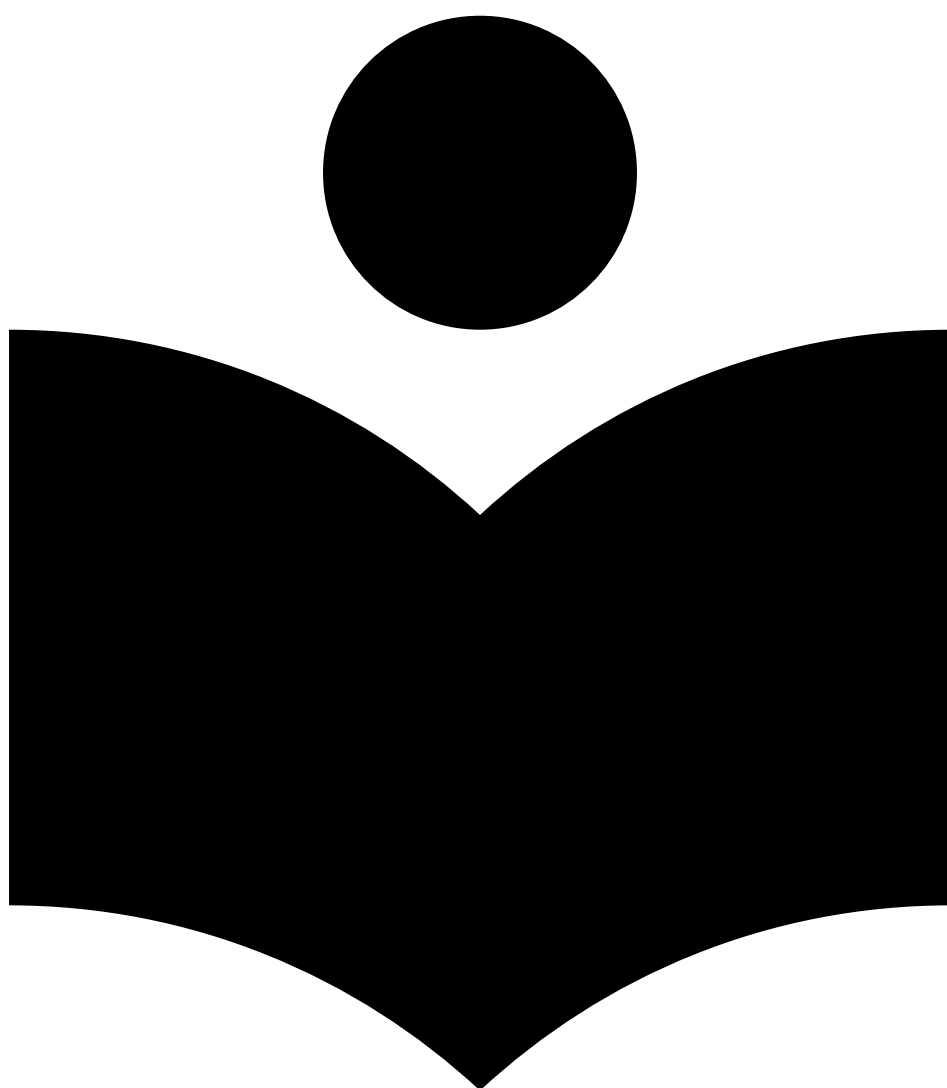
```
kubectl get pods -A | grep -v Running | grep -v Completed
```

```
kubectl get events -A --sort-by='.lastTimestamp' | tail -20
```

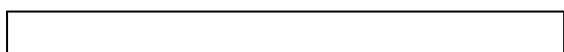
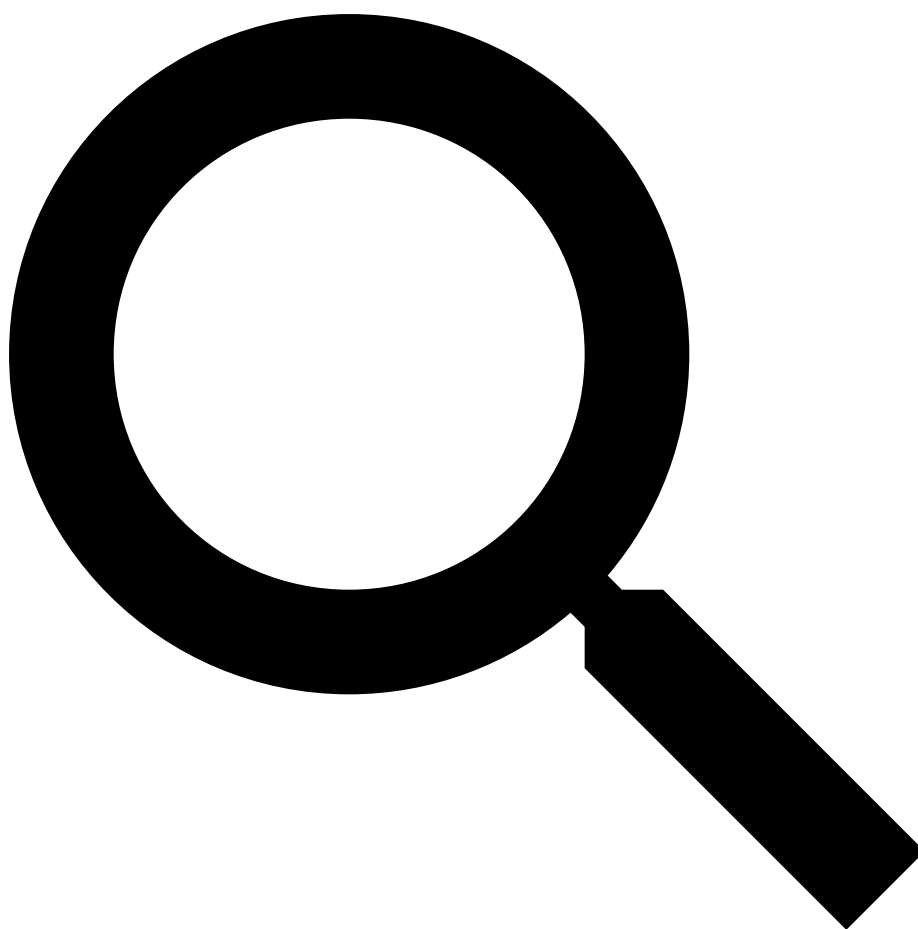
© 2025 VEGA TRAINING

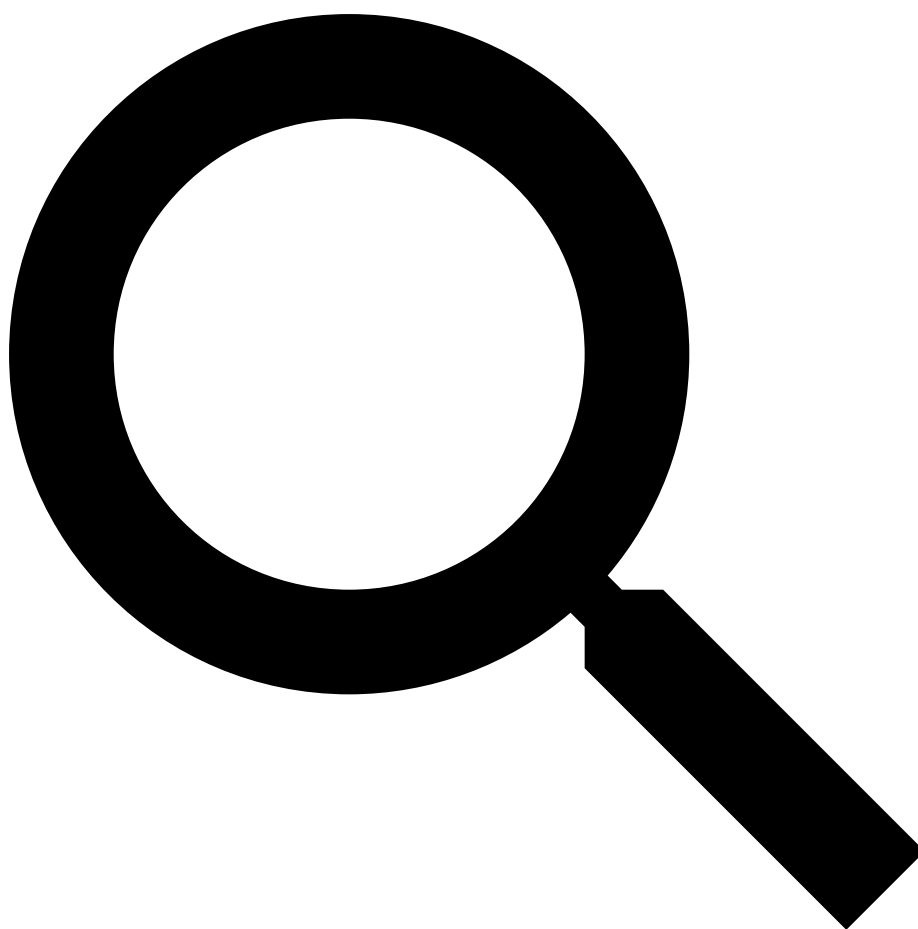


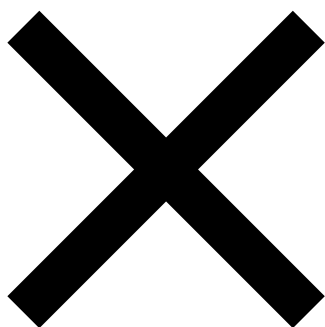
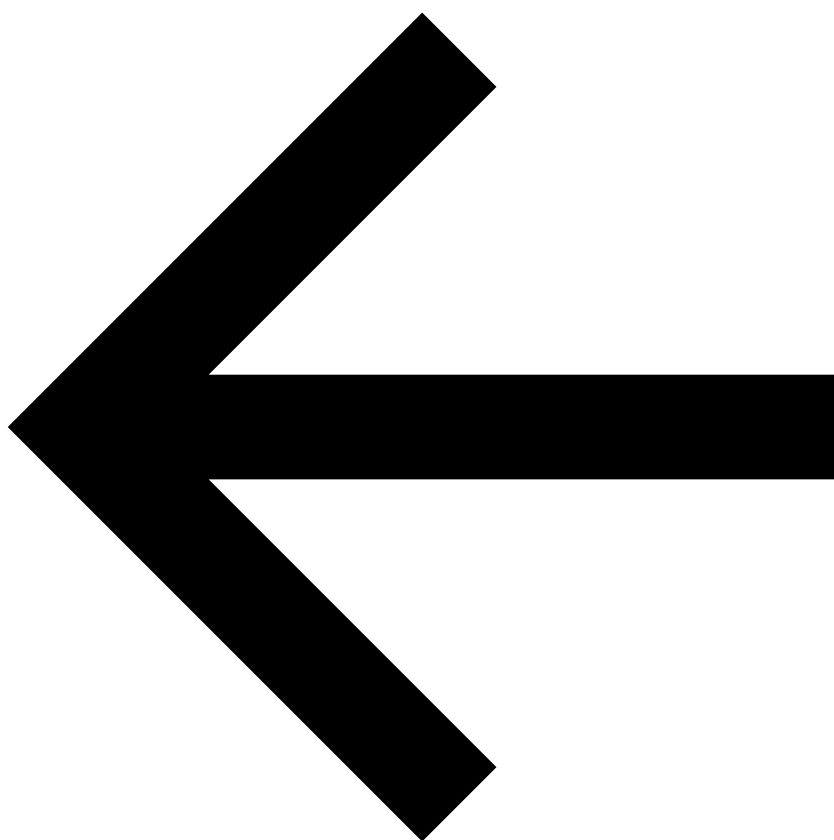
[Skip to content](#)





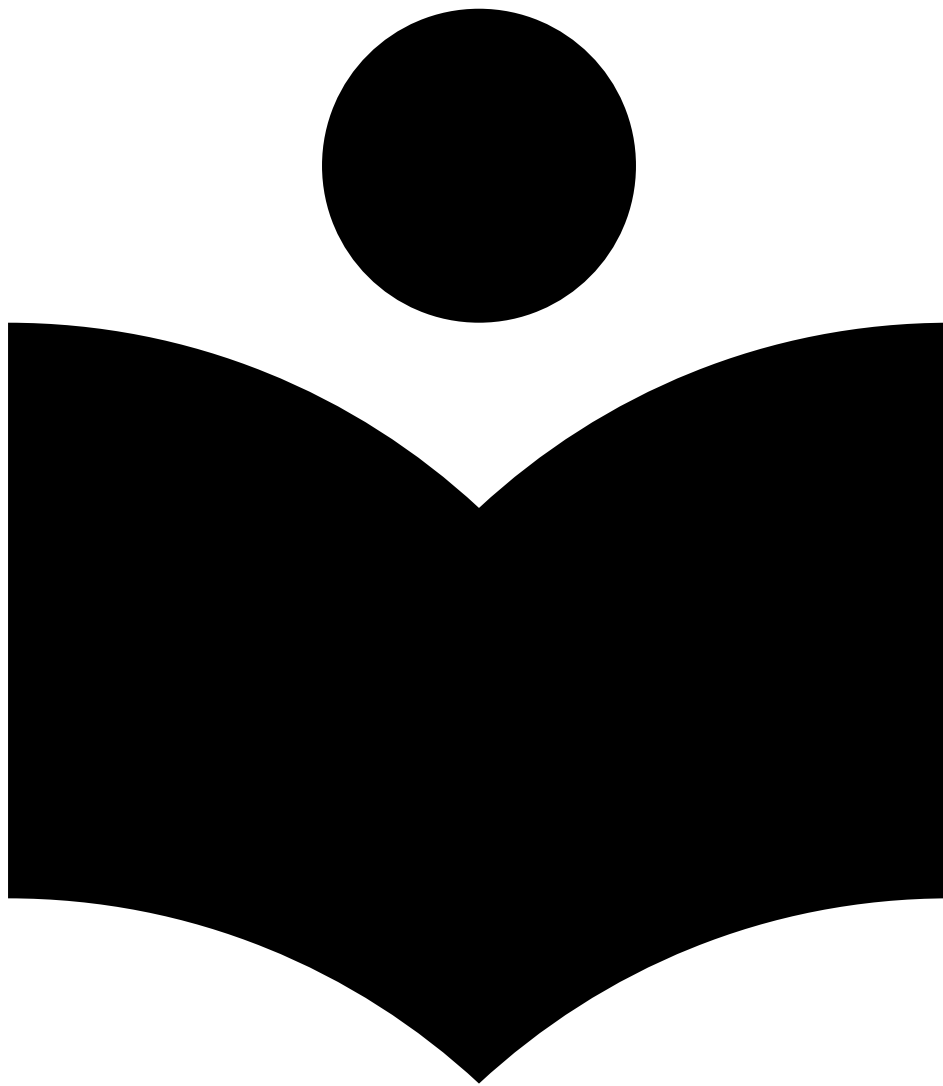






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
 - [Ch 03: Install Kubernetes](#)
 - [Ch 04: Architecture](#)
 - [Ch 05: API Access](#)
 - [Ch 06: API Objects](#)
 - [Ch 07: Deployments](#)
 - [Ch 08: Helm & Kustomize](#)
 - [Ch 09: Volumes](#)
 - [Ch 10: Services](#)
 - [Ch 11: Ingress](#)
 - [Ch 12: Scheduling](#)
 - [Ch 13: Troubleshooting](#)
 - ☐ Ch 14: Custom Resources [Ch 14: Custom Resources](#)
- Table of contents
- [Overview](#)

- [Lab 14.1 — Create a CRD](#)
- [Lab 14.2 — Create Custom Resource Instances](#)
- [Lab 14.3 — CRD Validation](#)
- [Lab 14.4 — Watch and List Custom Resources](#)
- [Lab 14.5 — Clean Up](#)
- [Understanding Operators](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 14.1 — Create a CRD](#)
- [Lab 14.2 — Create Custom Resource Instances](#)
- [Lab 14.3 — CRD Validation](#)
- [Lab 14.4 — Watch and List Custom Resources](#)
- [Lab 14.5 — Clean Up](#)
- [Understanding Operators](#)

Chapter 14: Custom Resources¶

Overview¶

In this lab you will:

- Create a Custom Resource Definition (CRD)
- Create instances of the custom resource
- Explore CRD validation and versioning
- Understand how operators use CRDs

Lab 14.1 — Create a CRD¶

Define a CRD for a fictional BackupPolicy resource:

```
cat <<EOF | kubectl apply -f -
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backuppolicies.vega.training
spec:
  group: vega.training
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
```



```

spec:
  type: object
  required:
  - schedule
  - retentionDays
  properties:
    schedule:
      type: string
      description: "Cron expression for backup schedule"
    retentionDays:
      type: integer
      minimum: 1
      maximum: 365
      description: "Number of days to retain backups"
    target:
      type: string
      description: "Target resource or namespace"
scope: Namespaced
names:
  plural: backuppolicies
  singular: backuppolicy
  kind: BackupPolicy
  shortNames:
  - bp
EOF

```

Verify the CRD is registered:

```

kubectl get crd backuppolicies.vega.training
kubectl api-resources | grep vega

```

Lab 14.2 — Create Custom Resource Instances

Create a BackupPolicy object:

```

cat <<EOF | kubectl apply -f -
apiVersion: vega.training/v1
kind: BackupPolicy
metadata:
  name: daily-backup
  namespace: default
spec:
  schedule: "0 2 * * *"
  retentionDays: 30
  target: "default"
EOF

```

Work with it using standard kubectl commands:

```
kubectl get backuppolicies
kubectl get bp
kubectl describe backuppolicy daily-backup
kubectl get backuppolicy daily-backup -o yaml
```

Create another instance:

```
cat <<EOF | kubectl apply -f -
apiVersion: vega.training/v1
kind: BackupPolicy
metadata:
  name: hourly-backup
  namespace: default
spec:
  schedule: "0 * * * *"
  retentionDays: 7
  target: "kube-system"
EOF
```

```
kubectl get bp
```

Lab 14.3 — CRD Validation¶

Try to create an invalid resource (retentionDays exceeds the maximum of 365):

```
cat <<EOF | kubectl apply -f -
apiVersion: vega.training/v1
kind: BackupPolicy
metadata:
  name: invalid-backup
  namespace: default
spec:
  schedule: "0 0 * * *"
  retentionDays: 999
EOF
```

You should see a validation error from the API server.

Try one missing the required schedule field:

```
cat <<EOF | kubectl apply -f -
apiVersion: vega.training/v1
kind: BackupPolicy
metadata:
  name: missing-schedule
  namespace: default
spec:
  retentionDays: 7
EOF
```

Again, a validation error.

Lab 14.4 — Watch and List Custom Resources

```
# List with label selector (works the same as built-in resources)
kubectl get bp -l environment=prod 2>/dev/null || echo "No labeled resources"

# Add a label and filter
kubectl label backuppolicy daily-backup environment=prod
kubectl get bp -l environment=prod

# Use jsonpath to extract fields
kubectl get bp daily-backup -o jsonpath='{.spec.schedule}'
echo ""
```

Lab 14.5 — Clean Up

```
kubectl delete bp --all
kubectl delete crd backuppolicies.vega.training
```

Verify the CRD and all instances are gone:

```
kubectl get crd | grep vega
kubectl api-resources | grep vega
```

Understanding Operators

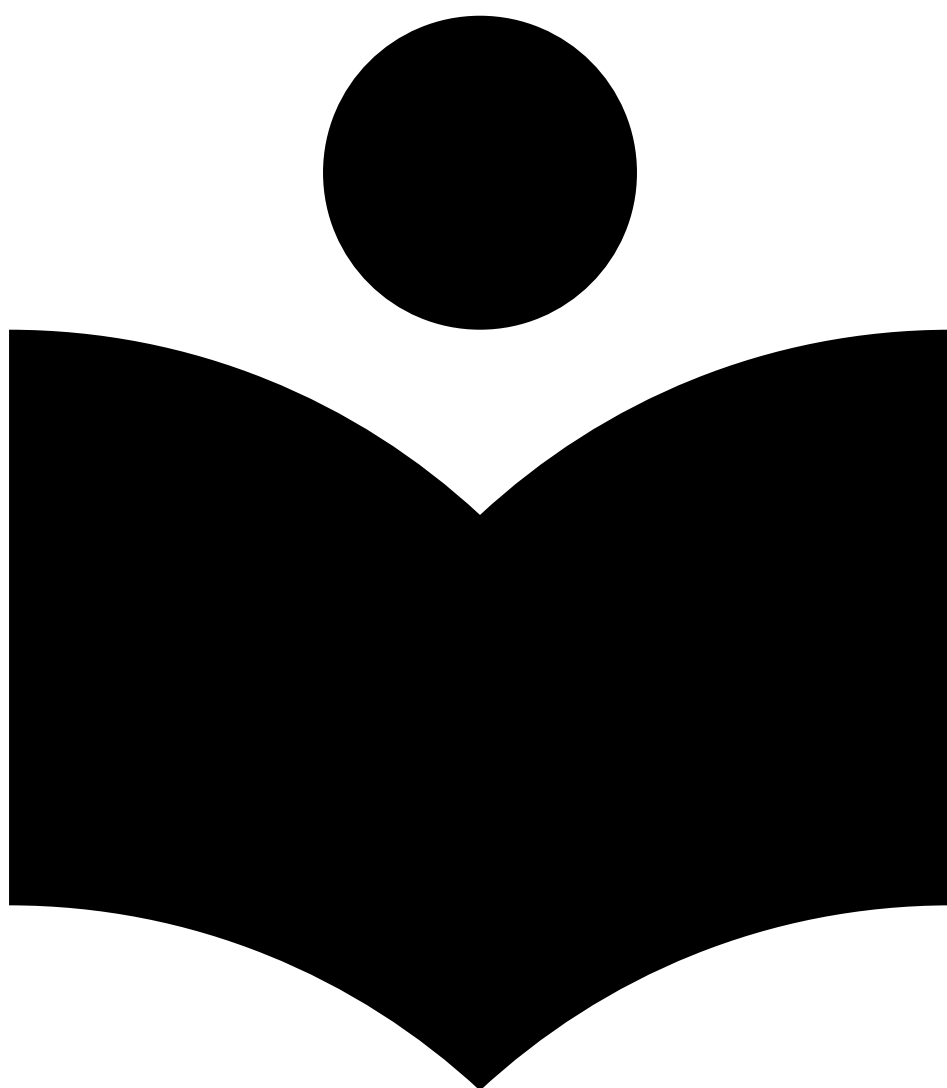
A **Kubernetes Operator** extends the control plane by:

1. Defining one or more CRDs that represent the desired state of an application
2. Running a controller (a Deployment) that watches those CRDs
3. Reconciling the actual state of the cluster to match the desired state declared in the custom resources

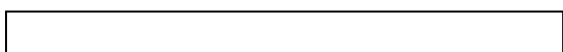
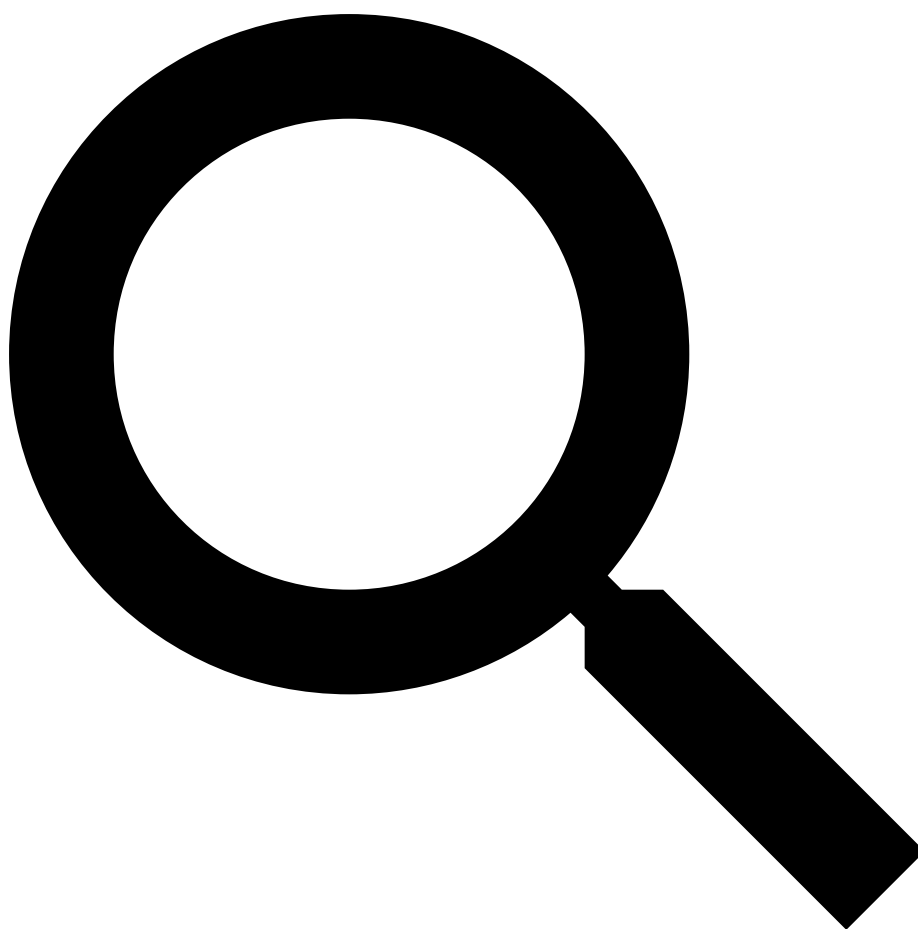
Popular operators include the Prometheus Operator, Cert-Manager, and the CloudNativePG PostgreSQL Operator. The CRD + controller pattern you practiced in this lab is exactly what they use internally.

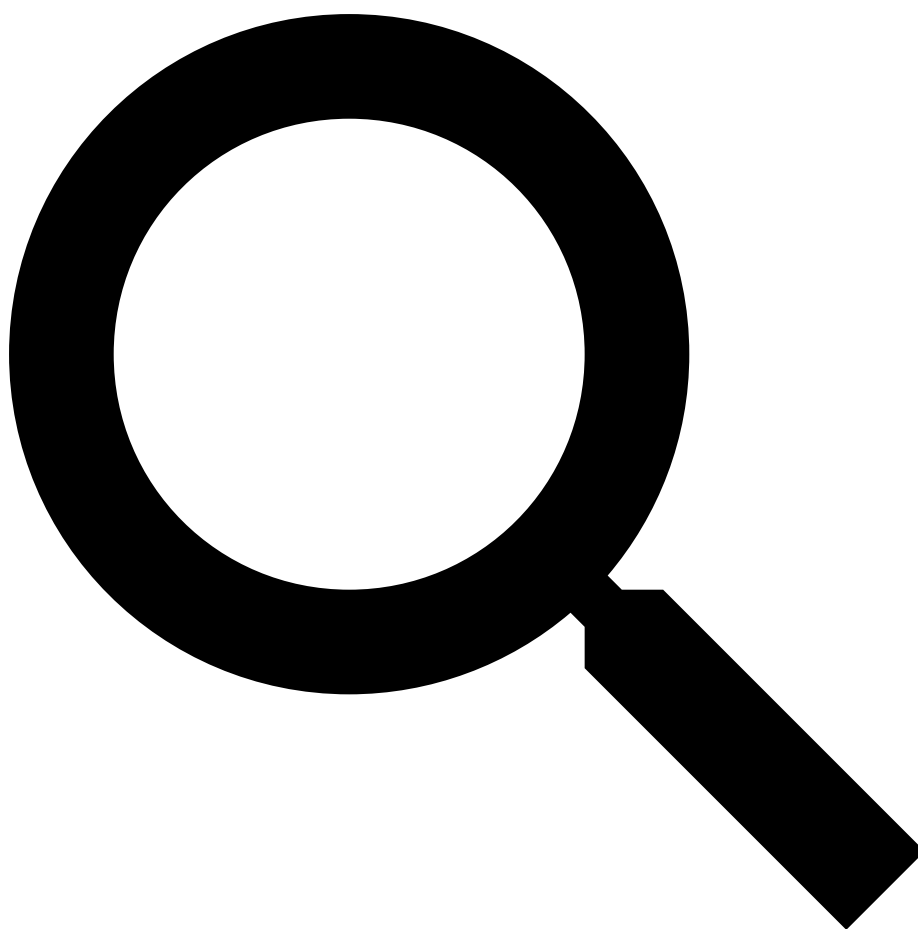


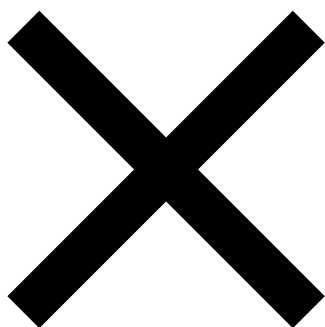
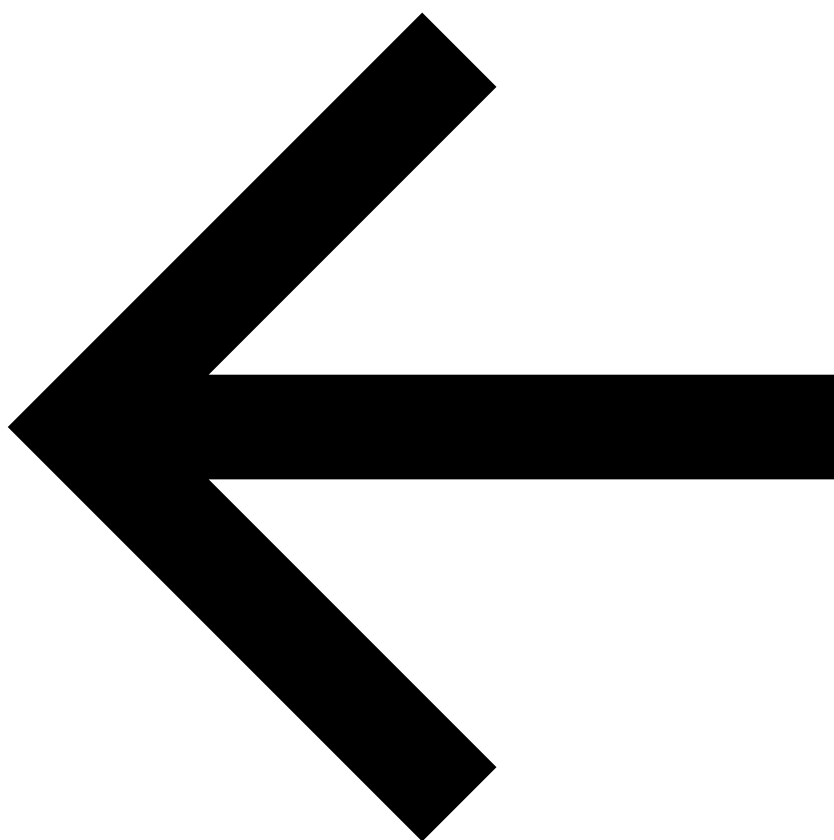
[Skip to content](#)





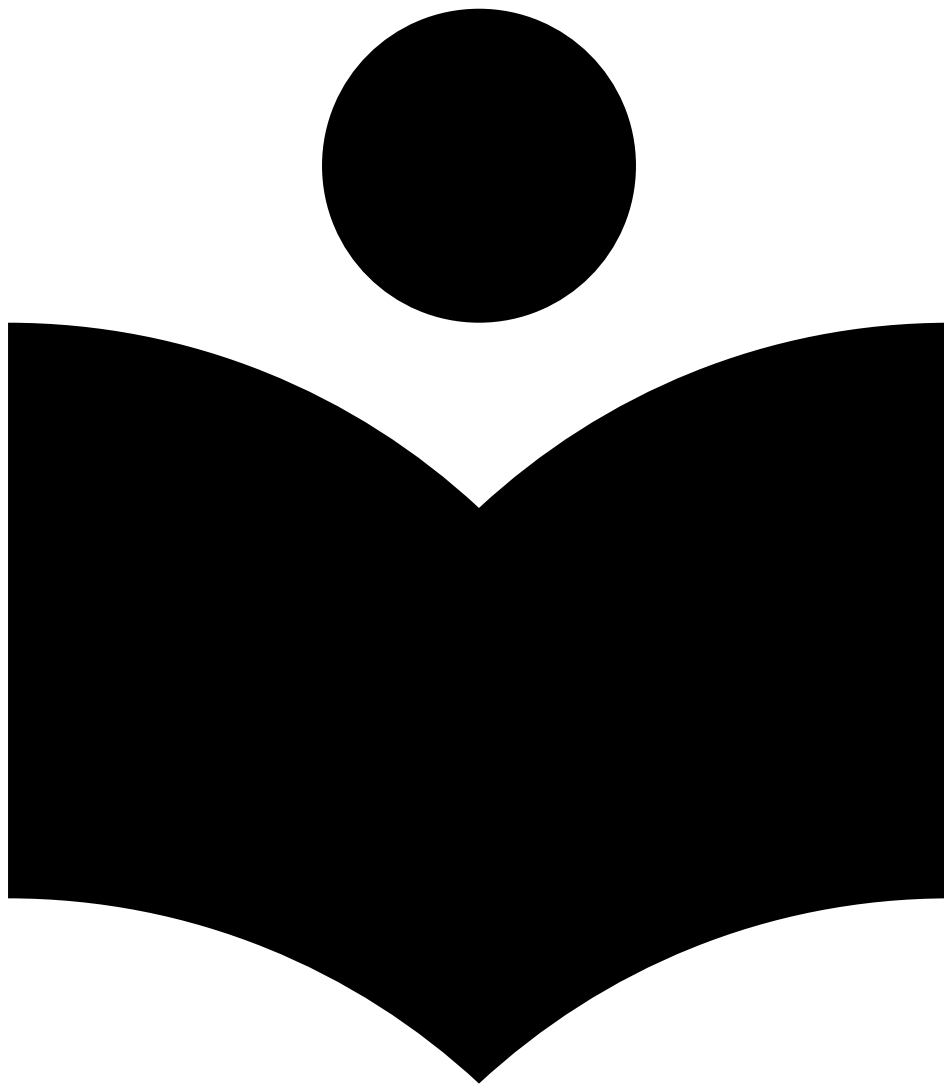






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- ☐ Ch 15: Security [Ch 15: Security](#)

Table of contents

- [Overview](#)
- [Lab 15.1 — Create a User with TLS Certificates](#)
 - [Generate a key and CSR for DevDan](#)
 - [Submit a CertificateSigningRequest to Kubernetes](#)
 - [Add DevDan to kubeconfig](#)
- [Lab 15.2 — RBAC: Role and RoleBinding](#)
 - [Create a namespace for DevDan](#)
 - [Create a Role in the prod namespace](#)
 - [Bind the Role to DevDan](#)
 - [Test DevDan's access](#)
 - [Grant more permissions \(edit the Role\)](#)
- [Lab 15.3 — ClusterRole and ClusterRoleBinding](#)
 - [Check permissions with auth can-i](#)
- [Lab 15.4 — NetworkPolicy](#)
 - [Clean up](#)
- [Lab 15.5 — Clean Up](#)
- [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 15.1 — Create a User with TLS Certificates](#)
 - [Generate a key and CSR for DevDan](#)
 - [Submit a CertificateSigningRequest to Kubernetes](#)
 - [Add DevDan to kubeconfig](#)
- [Lab 15.2 — RBAC: Role and RoleBinding](#)
 - [Create a namespace for DevDan](#)
 - [Create a Role in the prod namespace](#)
 - [Bind the Role to DevDan](#)
 - [Test DevDan's access](#)
 - [Grant more permissions \(edit the Role\)](#)
- [Lab 15.3 — ClusterRole and ClusterRoleBinding](#)
 - [Check permissions with auth can-i](#)
- [Lab 15.4 — NetworkPolicy](#)
 - [Clean up](#)
- [Lab 15.5 — Clean Up](#)

Chapter 15: Security¶

Overview¶

In this lab you will:

- Create a new user with TLS certificates (DevDan)
- Configure RBAC: Roles, ClusterRoles, RoleBindings
- Restrict a namespace with a NetworkPolicy
- Explore Admission Controllers

Lab files are in ~/lfs458/ch15-security/. The solutions/ subfolder contains completed manifests.

Lab 15.1 — Create a User with TLS Certificates

Kubernetes doesn't manage user accounts directly — users are identified by the Common Name (CN) in their TLS certificate, signed by the cluster CA.

Generate a key and CSR for DevDan

```
mkdir -p ~/devdan
openssl genrsa -out ~/devdan/devdan.key 2048
```

```
openssl req -new \
  -key ~/devdan/devdan.key \
  -out ~/devdan/devdan.csr \
  -subj "/CN=devdan/O=developers"
```

Submit a CertificateSigningRequest to Kubernetes

```
CSR_BASE64=$(cat ~/devdan/devdan.csr | base64 | tr -d '\n')
```

```
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: devdan
spec:
  request: $CSR_BASE64
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages:
    - client auth
EOF
```

Approve the CSR:

```
kubectl certificate approve devdan
kubectl get csr devdan
```

Retrieve the signed certificate:

```
kubectl get csr devdan -o jsonpath='{.status.certificate}' | base64 -d > ~
```

Add DevDan to kubeconfig

```
kubectl config set-credentials devdan \
  --client-certificate=~/devdan/devdan.crt \
  --client-key=~/devdan/devdan.key
```

```
kubectl config set-context devdan-context \
  --cluster=$(kubectl config current-context | xargs kubectl config view -
  --user=devdan
```

```
kubectl config get-contexts
```

Test that DevDan currently has no permissions:

```
kubectl get pods --context=devdan-context
```

You should see a Forbidden error.

Lab 15.2 — RBAC: Role and RoleBinding¶

Create a namespace for DevDan¶

```
kubectl create namespace prod
```

Create a Role in the prod namespace¶

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: prod
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
EOF
```

Bind the Role to DevDan¶

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devdan-pod-reader
  namespace: prod
subjects:
- kind: User
  name: devdan
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
EOF
```

Test DevDan's access

```
# Update the devdan context to use the prod namespace
kubectl config set-context devdan-context \
  --namespace=prod \
  --cluster=$(kubectl config view -o jsonpath='{.clusters[0].name}') \
  --user=devdan
```

```
# DevDan can list pods in prod
kubectl get pods --context=devdan-context -n prod
```

```
# DevDan cannot create pods
kubectl run test --image=nginx --context=devdan-context -n prod
```

The last command should return Forbidden.

Grant more permissions (edit the Role)

Edit the role to also allow creating pods. The solution is at `~/lfs458/ch15-security/solutions/role-prod-create.yaml`.

```
kubectl edit role pod-reader -n prod
# Add "create", "delete" to the verbs list
```

Lab 15.3 — ClusterRole and ClusterRoleBinding

Create a read-only ClusterRole and bind it to DevDan cluster-wide:

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-viewer
rules:
- apiGroups: [""]
  resources: ["nodes", "namespaces", "persistentvolumes"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: devdan-cluster-viewer
subjects:
- kind: User
  name: devdan
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-viewer
```

```
  apiGroup: rbac.authorization.k8s.io
EOF
```

Test:

```
kubectl get nodes --context=devdan-context
kubectl get namespaces --context=devdan-context
kubectl get secrets --context=devdan-context -n default # Should be Forbidden
```

Check permissions with auth can-i

```
kubectl auth can-i list pods --as=devdan -n prod
kubectl auth can-i create secrets --as=devdan -n prod
kubectl auth can-i get nodes --as=devdan
```

Lab 15.4 — NetworkPolicy

By default, pods can communicate freely across namespaces. NetworkPolicies restrict this.

Create two namespaces and deploy pods:

```
kubectl create namespace frontend
kubectl create namespace backend

kubectl run frontend-pod --image=nginx -n frontend --labels=app=frontend
kubectl run backend-pod --image=nginx -n backend --labels=app=backend
```

Allow only the frontend namespace to reach backend on port 80:

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: frontend
    ports:
    - protocol: TCP
      port: 80
EOF
```


Test connectivity:

```
BACKEND_IP=$(kubectl get pod backend-pod -n backend -o jsonpath='{.status.podIP}')  
echo "Backend pod IP: $BACKEND_IP"
```

```
# From frontend – should work
```

```
kubectl exec -n frontend frontend-pod -- curl -s --max-time 5 http://$BACKEND_IP
```

```
# From default namespace – should be blocked
```

```
kubectl run testpod --image=busybox --rm -it --restart=Never -- \\\n  wget -O- --timeout=5 http://$BACKEND_IP
```

Clean up

```
kubectl delete networkpolicy allow-frontend -n backend
```

```
kubectl delete pod frontend-pod -n frontend
```

```
kubectl delete pod backend-pod -n backend
```

```
kubectl delete namespace frontend backend
```

Lab 15.5 — Clean Up

```
kubectl delete rolebinding devdan-pod-reader -n prod
```

```
kubectl delete role pod-reader -n prod
```

```
kubectl delete clusterrolebinding devdan-cluster-viewer
```

```
kubectl delete clusterrole cluster-viewer
```

```
kubectl delete namespace prod
```

```
kubectl delete csr devdan
```

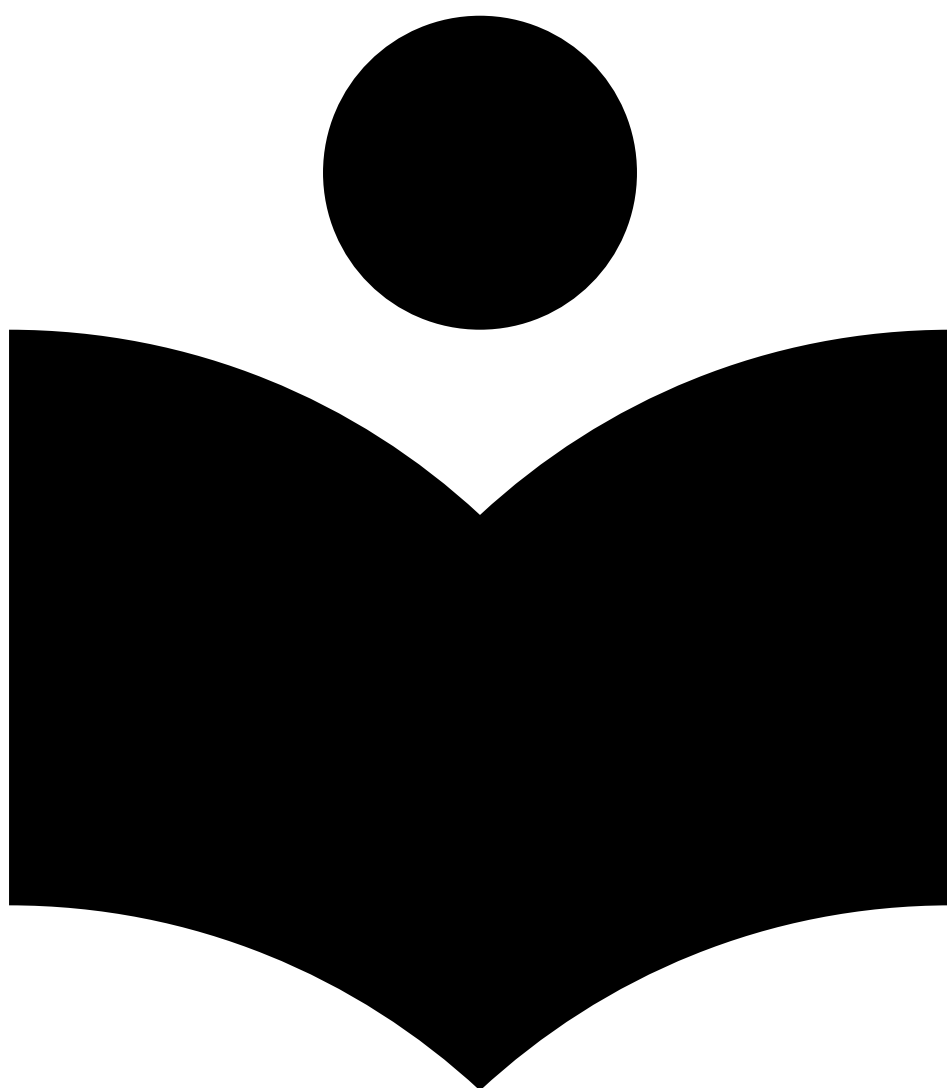
```
kubectl config delete-context devdan-context
```

```
kubectl config delete-user devdan
```

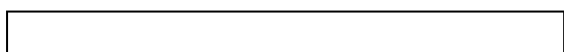
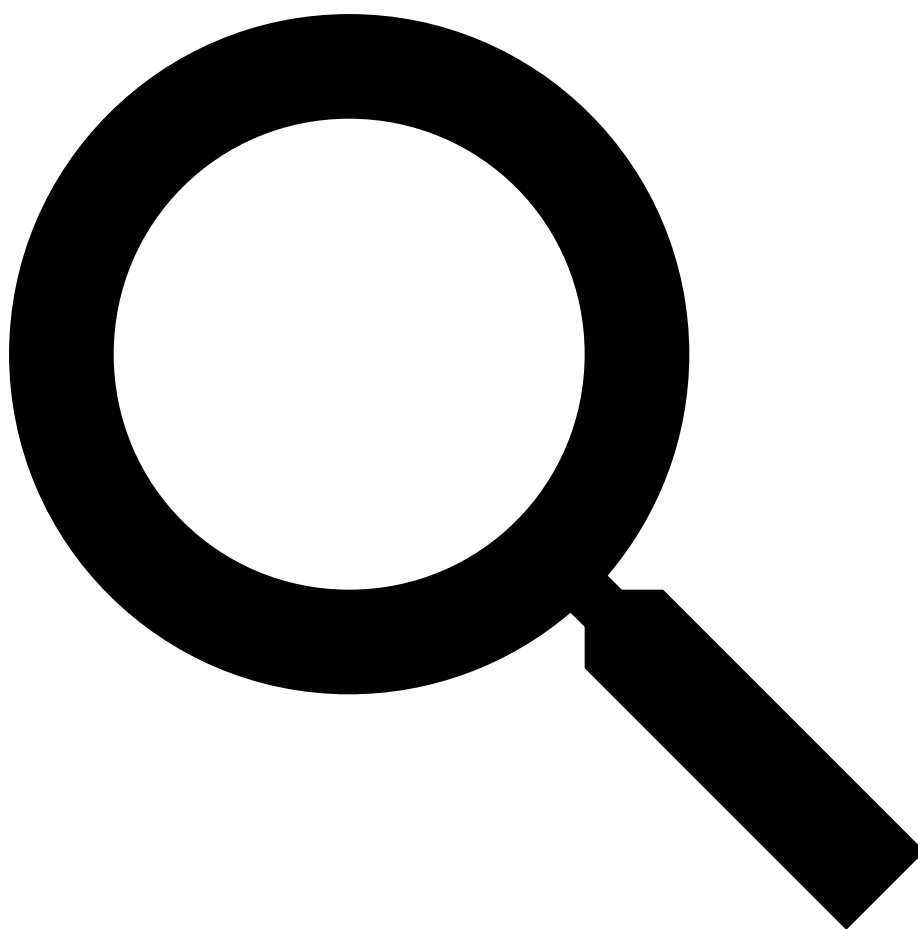
© 2025 VEGA TRAINING

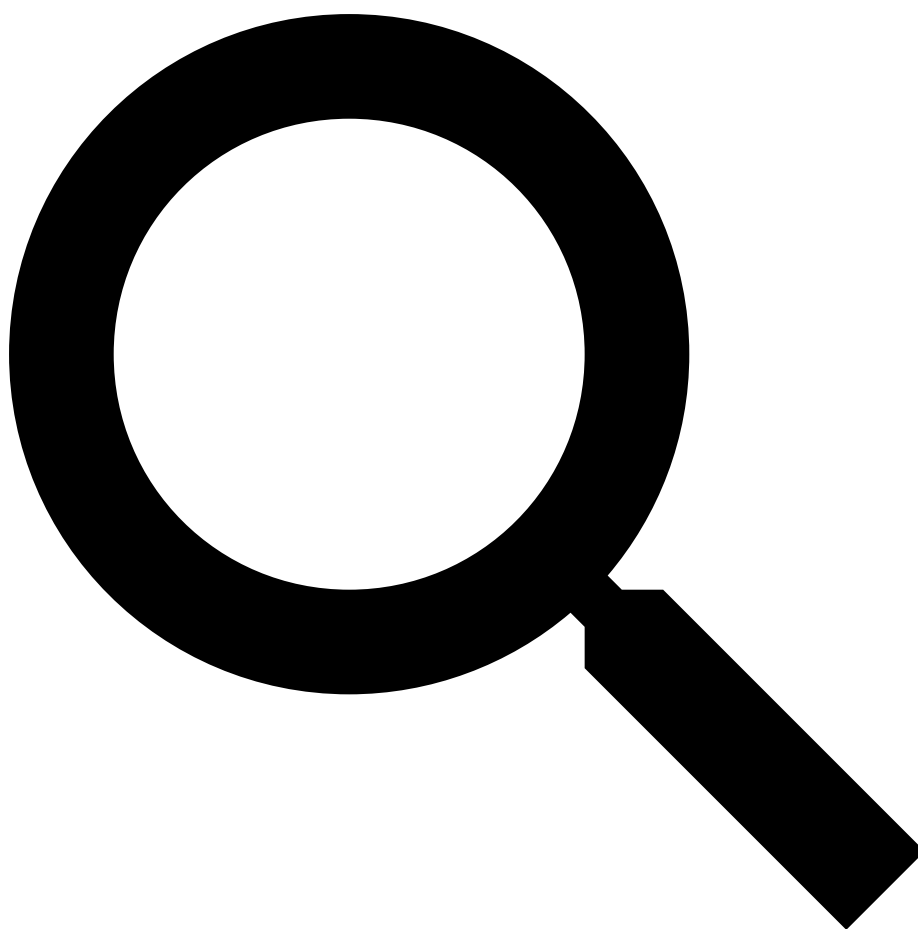


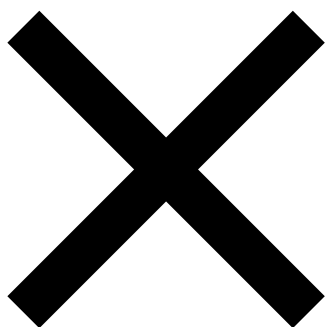
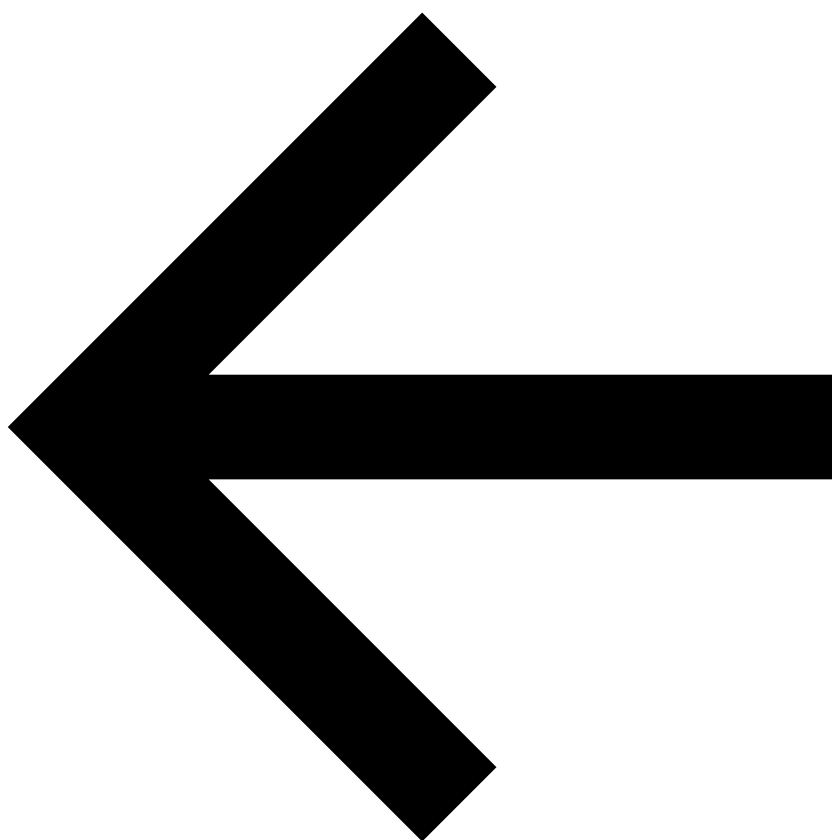
[Skip to content](#)





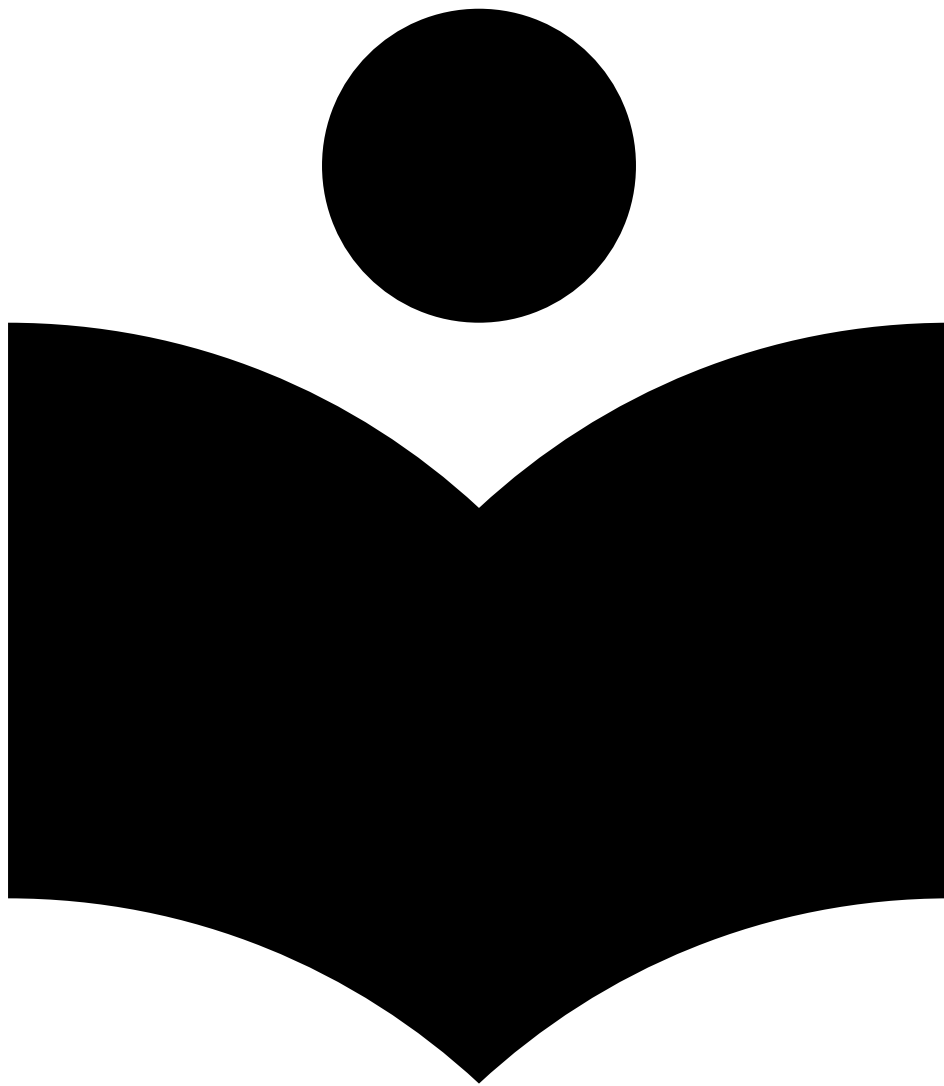






Initializing search

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- [Ch 16: High Availability](#)



LFS458 - Kubernetes Administration Lab Guide

- [Home](#)
- [Ch 03: Install Kubernetes](#)
- [Ch 04: Architecture](#)
- [Ch 05: API Access](#)
- [Ch 06: API Objects](#)
- [Ch 07: Deployments](#)
- [Ch 08: Helm & Kustomize](#)
- [Ch 09: Volumes](#)
- [Ch 10: Services](#)
- [Ch 11: Ingress](#)
- [Ch 12: Scheduling](#)
- [Ch 13: Troubleshooting](#)
- [Ch 14: Custom Resources](#)
- [Ch 15: Security](#)
- ☐ Ch 16: High Availability [Ch 16: High Availability](#)

Table of contents

- [Overview](#)
- [Lab 16.1 — HAProxy Load Balancer](#)
 - [Install HAProxy \(on a dedicated LB node or the controller\)](#)
 - [Configure HAProxy](#)
- [Lab 16.2 — Add a Second Control Plane Node](#)
- [Lab 16.3 — etcd Leader Election](#)
- [Lab 16.4 — Control Plane Failover Test](#)
 - [Deploy a workload](#)
 - [Simulate a control plane failure](#)
 - [Restore the controller](#)
 - [Clean up](#)
- [Lab 16.5 — kube-vip \(Optional\)](#)

Table of contents

- [Overview](#)
- [Lab 16.1 — HAProxy Load Balancer](#)
 - [Install HAProxy \(on a dedicated LB node or the controller\)](#)
 - [Configure HAProxy](#)
- [Lab 16.2 — Add a Second Control Plane Node](#)
- [Lab 16.3 — etcd Leader Election](#)
- [Lab 16.4 — Control Plane Failover Test](#)
 - [Deploy a workload](#)
 - [Simulate a control plane failure](#)
 - [Restore the controller](#)
 - [Clean up](#)
- [Lab 16.5 — kube-vip \(Optional\)](#)

Chapter 16: High Availability¶

Overview¶

In this lab you will:

- Set up HAProxy as a load balancer in front of multiple control plane nodes
- Join a second control plane node to the cluster
- Test etcd leader election
- Simulate a control plane failure and verify workloads continue running

Lab topology

This chapter assumes you have access to an additional VM for a second control plane node. If not, you can still follow the HAProxy and etcd sections on the existing 3-node cluster.

Lab 16.1 — HAProxy Load Balancer

HAProxy will distribute API server traffic across all control plane nodes on port 6443.

Install HAProxy (on a dedicated LB node or the controller)

```
sudo apt-get install -y haproxy
```

Configure HAProxy

Replace /etc/haproxy/haproxy.cfg with:

```
sudo tee /etc/haproxy/haproxy.cfg > /dev/null <<'EOF'
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon

defaults
    log          global
    mode         tcp
    option       tcplog
    option       dontlognull
    timeout     connect 5s
    timeout     client  50s
    timeout     server  50s

frontend kubernetes-api
    bind *:6443
    mode tcp
    default_backend kubernetes-masters

backend kubernetes-masters
    mode tcp
    balance roundrobin
    option tcp-check
    server controller controller:6443 check
    # Add additional control plane nodes here:
    # server controller2 controller2:6443 check
EOF
```

```
sudo systemctl restart haproxy
sudo systemctl enable haproxy
sudo systemctl status haproxy
```

Verify HAProxy is listening:

```
ss -tlnp | grep 6443
```

Lab 16.2 — Add a Second Control Plane Node

On the **controller**, generate a new join command with the `--control-plane` flag:

```
# Generate a new certificate key
CERT_KEY=$(kubeadm certs certificate-key)
echo "Certificate key: $CERT_KEY"

# Upload certs
sudo kubeadm init phase upload-certs --upload-certs --certificate-key $CERT_KEY

# Generate the join command
kubeadm token create --print-join-command --certificate-key $CERT_KEY
```

On the **second control plane node** (e.g., controller2), run the join command with `--control-plane`:

```
sudo kubeadm join <load-balancer-ip>:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Set up `kubectl` on the new node:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Back on the **original controller**, verify:

```
kubectl get nodes
kubectl get pods -n kube-system | grep etcd
```

You should now see two `etcd` pods and two controller nodes.

Lab 16.3 — etcd Leader Election

Inspect the `etcd` cluster members:

```
ETCD_POD=$(kubectl get pod -n kube-system -l component=etcd \
  -o jsonpath='{.items[0].metadata.name}')

kubectl exec $ETCD_POD -n kube-system -- \
  etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --write-out=table
```

Identify the leader:

```
kubectl exec $ETCD_POD -n kube-system -- \
  etcdctl endpoint status \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  --write-out=table
```

The IS LEADER column shows the current etcd leader.

Lab 16.4 — Control Plane Failover Test

This test shows that workloads continue running even if the primary control plane is unavailable.

Deploy a workload

```
kubectl create deployment failover-test --image=nginx --replicas=4
kubectl get pods -l app=failover-test -o wide
```

Simulate a control plane failure

Stop the kubelet on the primary controller:

```
# On the controller node:
sudo systemctl stop kubelet
```

From another machine (or controller2), verify the workload still runs:

```
kubectl get pods -l app=failover-test -o wide
kubectl get nodes
```

The pods should still be Running. The controller node will show NotReady after the node lease expires (~40 seconds).

Restore the controller

```
# On the controller node:
sudo systemctl start kubelet
```

Verify it rejoins:

```
kubectl get nodes -w
```

Clean up

```
kubectl delete deployment failover-test
```

Lab 16.5 — kube-vip (Optional)[¶](#)

For a production-grade virtual IP (VIP) approach instead of HAProxy, kube-vip provides a Layer 2 VIP for the control plane endpoint. It runs as a DaemonSet on control plane nodes and elects a leader that holds the VIP.

```
# Generate kube-vip manifest (example – adjust VIP and interface to your e
VIP=<your-vip-address>
IFACE=eth0
```

```
kubectl apply -f https://kube-vip.io/manifests/rbac.yaml
```

```
docker run --rm ghcr.io/kube-vip/kube-vip:latest manifest daemonset \
  --interface $IFACE \
  --address $VIP \
  --inCluster \
  --taint \
  --controlplane \
  --services \
  --arp \
  --leaderElection | kubectl apply -f -
```

This is advanced and environment-specific — refer to the kube-vip documentation for your network setup.

© 2025 VEGA TRAINING